Java Systems Programming: A Practical Study Guide

Ajay Kumar

August 2026

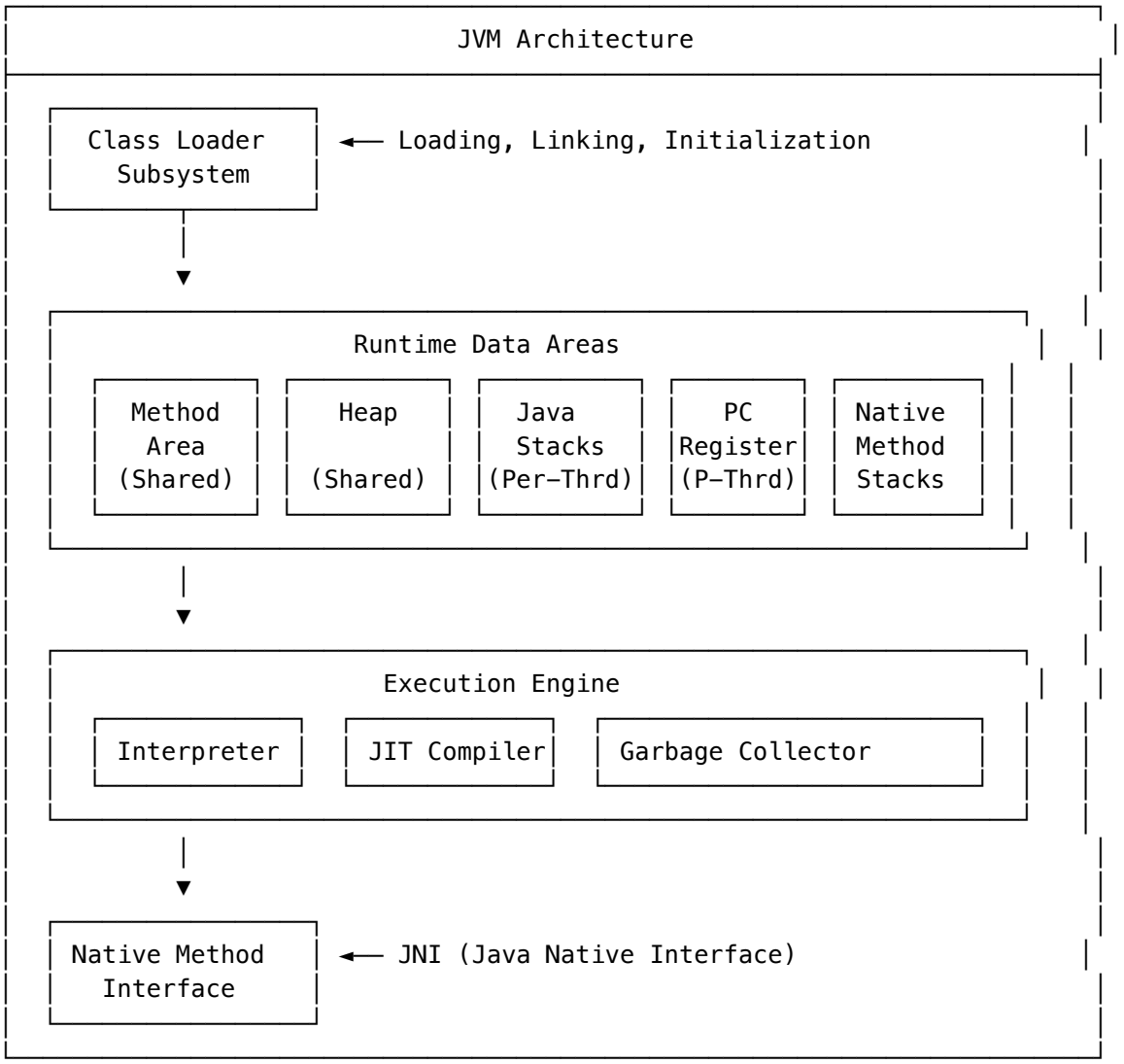
Contents

1 JVM Internals and Java Memory Model - Deep Dive

1.1 JVM Architecture Overview

The Java Virtual Machine is an abstract computing machine that enables a computer to run Java programs. Understanding its internals is crucial for writing efficient code and debugging complex issues.

1.1.1 JVM Components



1.2 Class Loading Mechanism

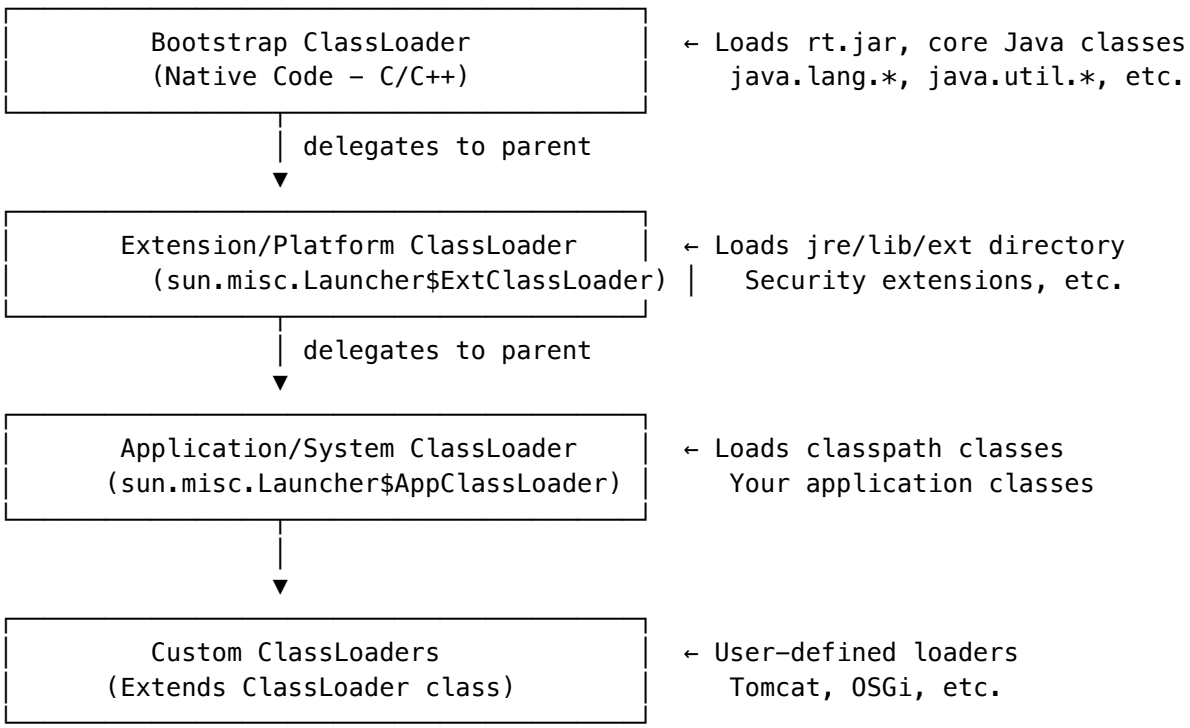
1.2.1 The Class Loading Process

Class loading follows a three-phase process: **Loading → Linking → Initialization**

1.2.1.1 Phase 1: Loading The class loader reads the `.class` file and creates a binary representation in the Method Area.

```
// Example: Understanding when classes are loaded
public class ClassLoadingDemo {
    public static void main(String[] args) {
        // Class A is loaded when first referenced
        System.out.println("Before A reference");
        A a = new A();  // Class A loaded here
        System.out.println("After A instantiation");
    }
}

class A {
    static {
        System.out.println("Class A static initializer");
    }

    {
        System.out.println("Class A instance initializer");
    }
}
// Output:
// Before A reference
// Class A static initializer
// Class A instance initializer
// After A instantiation
```

1.2.2 Class Loader Hierarchy (Delegation Model)



1.2.2.1 Parent Delegation Model - How It Works

```
// Simplified pseudocode of class loading
protected Class<?> loadClass(String name, boolean resolve) {
    // 1. Check if class already loaded
    Class<?> c = findLoadedClass(name);

    if (c == null) {
        try {
            // 2. Delegate to parent first (Parent Delegation)
            if (parent != null) {
                c = parent.loadClass(name, false);
            } else {
                c = findBootstrapClassOrNull(name);
            }
        } catch (ClassNotFoundException e) {
            // Parent couldn't find it
        }

        if (c == null) {
            // 3. If parent fails, try to load ourselves
            c = findClass(name);
        }
    }

    if (resolve) {
        resolveClass(c);
    }
    return c;
}
```

1.2.2.2 Why Parent Delegation?

1. **Security**: Prevents malicious code from replacing core Java classes
2. **Uniqueness**: Ensures class is loaded only once
3. **Visibility**: Child can see parent's classes, not vice versa

1.2.2.3 Phase 2: Linking (Verification → Preparation → Resolution)

```
// Linking has three sub-phases:

// 1. VERIFICATION
// - Ensures bytecode is valid and secure
// - Checks: magic number (0xCAFEBABE), version, constant pool
// - Verifies: type safety, stack operations, branch targets
// Errors: VerifyError, ClassFormatError

// 2. PREPARATION
// - Allocates memory for static fields
// - Initializes to DEFAULT values (not assigned values!)
class Example {
    static int count = 10;  // Preparation: count = 0
                            // Initialization: count = 10
    static Object obj = new Object();  // Preparation: obj = null
}

// 3. RESOLUTION
// - Converts symbolic references to direct references
// - Symbolic: "java.lang.String" (name)
// - Direct: actual memory address
// Can be eager or lazy (JVM implementation dependent)
```

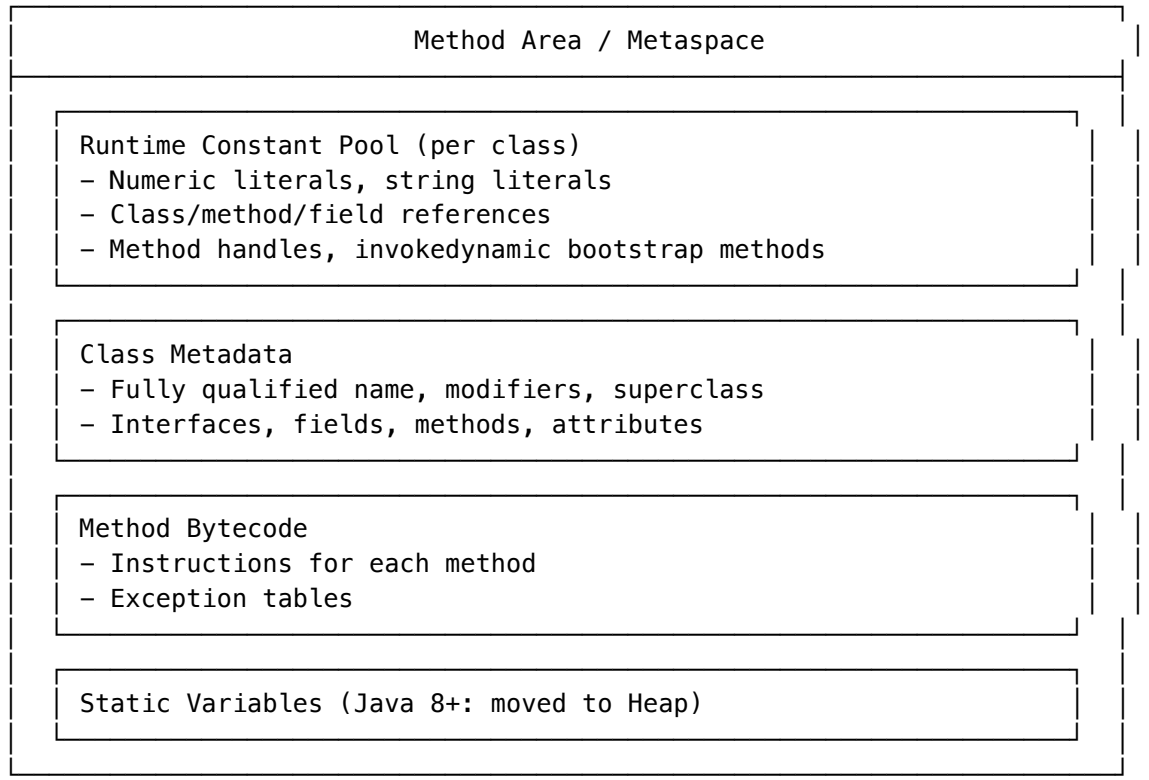
1.2.2.4 Phase 3: Initialization

```java
// Static initializers run in order
class InitOrder {
    static int a = initA();  // 1st
    static { System.out.println("Static block 1"); }  // 2nd
    static int b = initB();  // 3rd
    static { System.out.println("Static block 2"); }  // 4th

    static int initA() { System.out.println("initA"); return 1; }
    static int initB() { System.out.println("initB"); return 2; }
}
// Output: initA, Static block 1, initB, Static block 2

// When does initialization occur?
// 1. new keyword
// 2. Accessing static field (not final compile-time constant)
// 3. Calling static method
// 4. Reflection (Class.forName)
// 5. Initializing a subclass
// 6. Main class of JVM startup
```

1.3 Runtime Data Areas

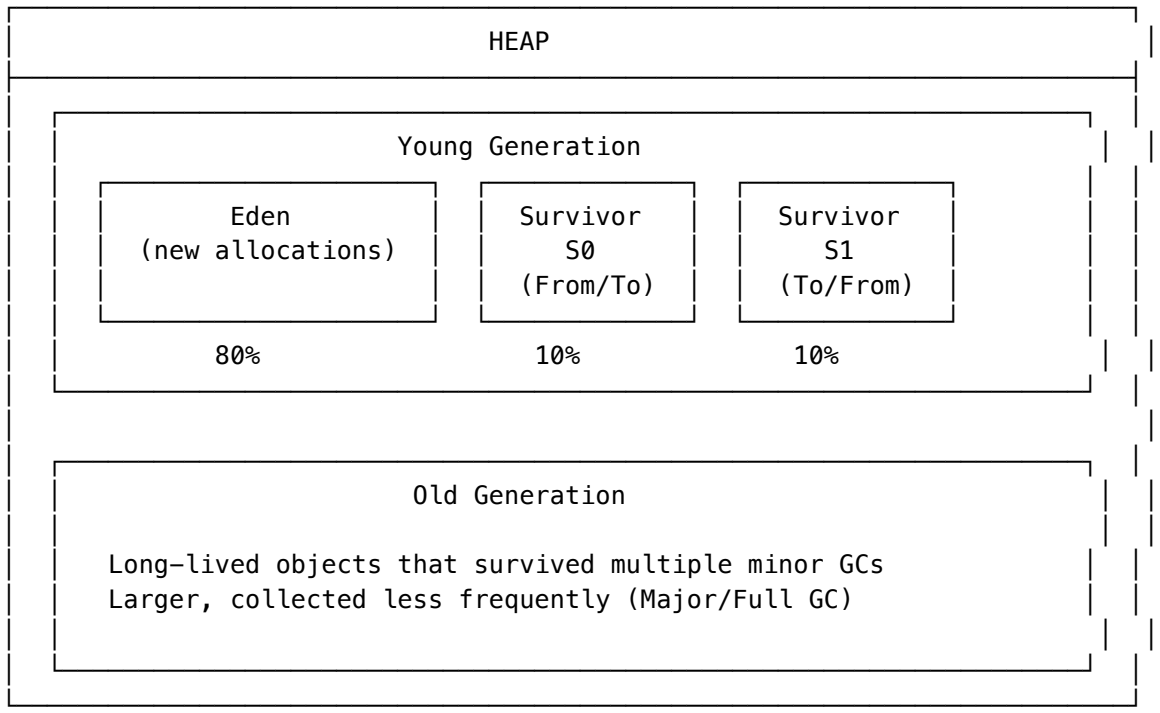
1.3.1 Method Area (Metaspace since Java 8)

Method Area Contents:



1.3.2 Heap Structure

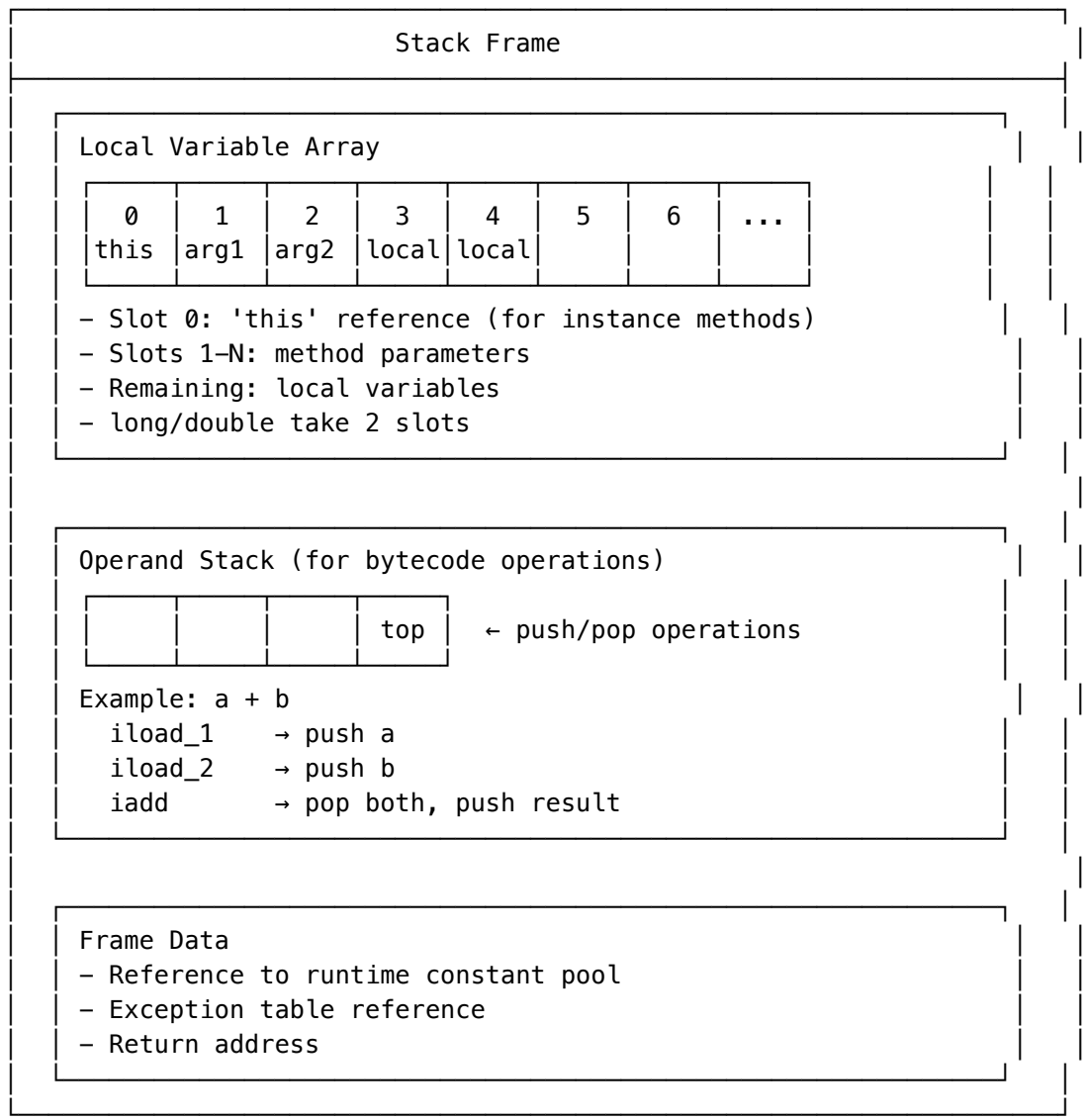
Java Heap Memory Layout:



Object Lifecycle:
1. New object allocated in Eden
2. Eden full → Minor GC → Survivors copied to S0/S1
3. Object survives N GCs → Promoted to Old Generation
4. Old Generation full → Major/Full GC

1.3.3 Java Stack (Thread Stack)

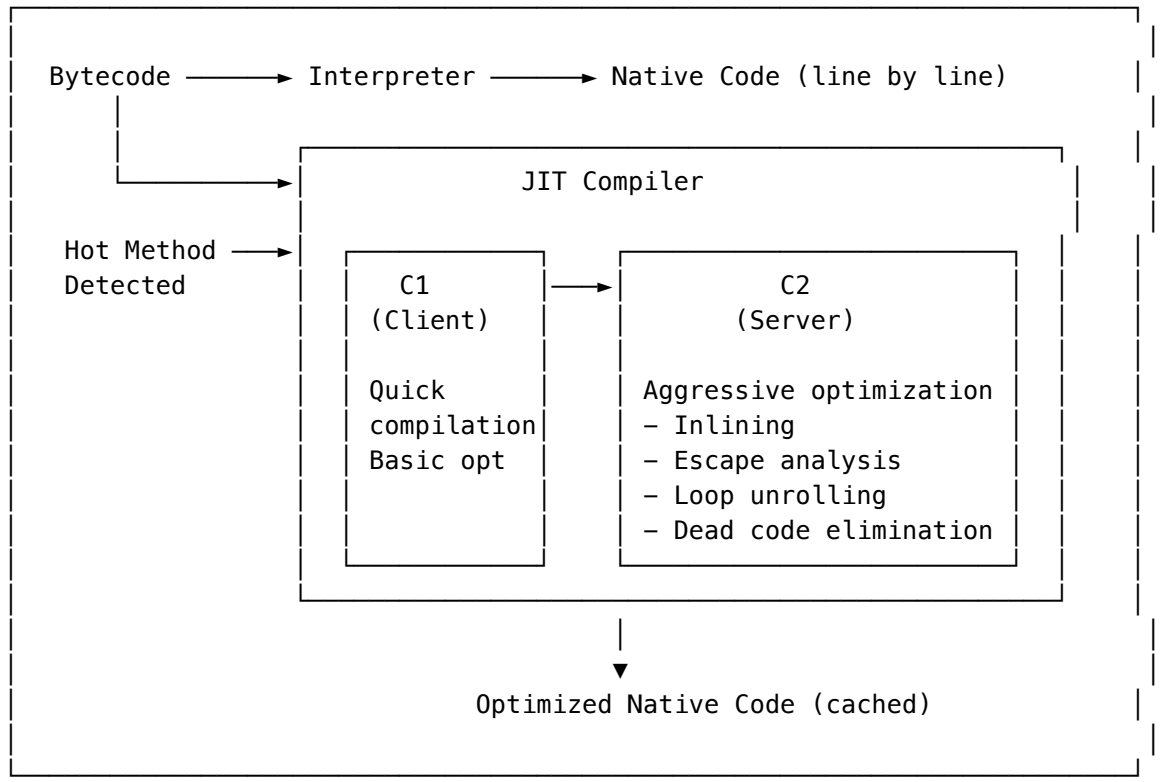
Stack Frame Structure:



1.4 Execution Engine

1.4.1 Interpreter vs JIT Compiler

Execution Phases:



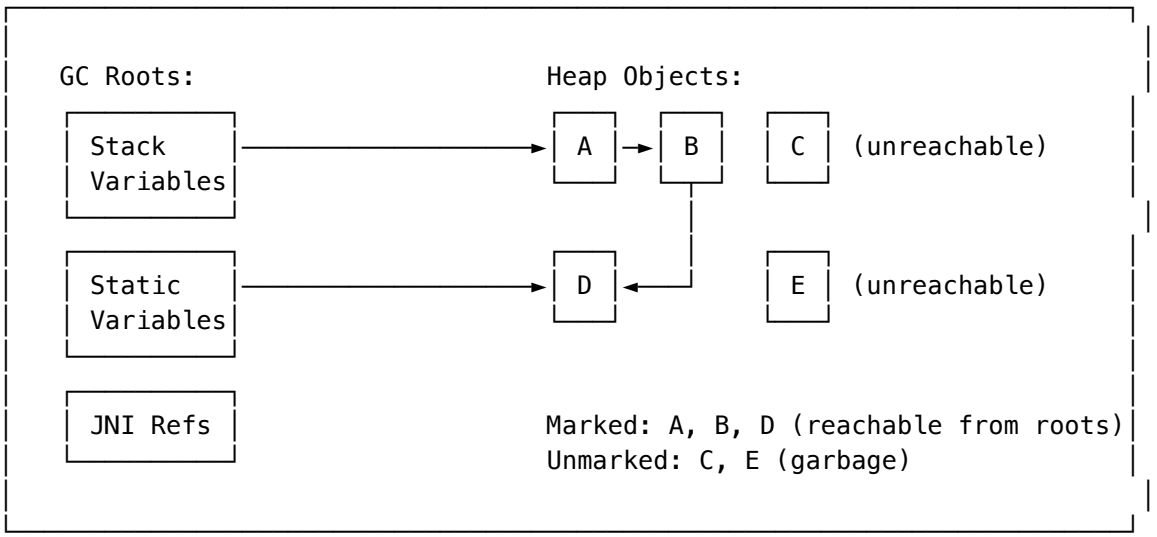
1.4.2 Tiered Compilation (Default in modern JVMs)

```
Level 0: Interpreted
    │
    ▼ (invocation count)
Level 1: C1 with full instrumentation
    │
    ▼ (more invocations)
Level 2: C1 with limited profiling
    │
    ▼ (even more)
Level 3: C1 with full profiling
    │
    ▼ (hot method threshold)
Level 4: C2 fully optimized

// JVM flags:
// -XX:+TieredCompilation (default: on)
// -XX:CompileThreshold=10000 (invocations before compilation)
```

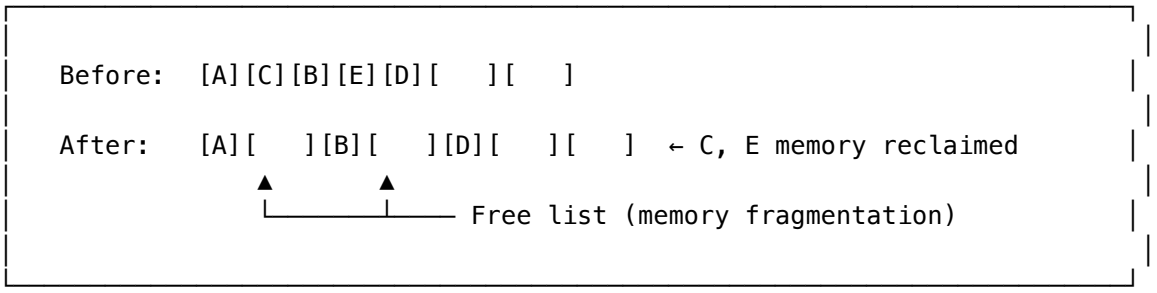
1.5 Garbage Collection Internals

1.5.1 Mark and Sweep Algorithm

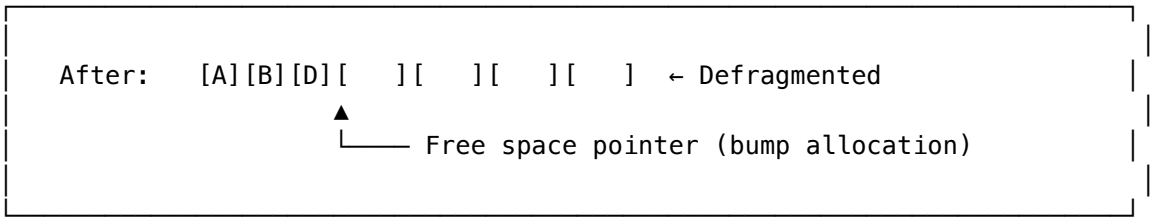
Phase 1: Mark (Stop-the-World)



Phase 2: Sweep

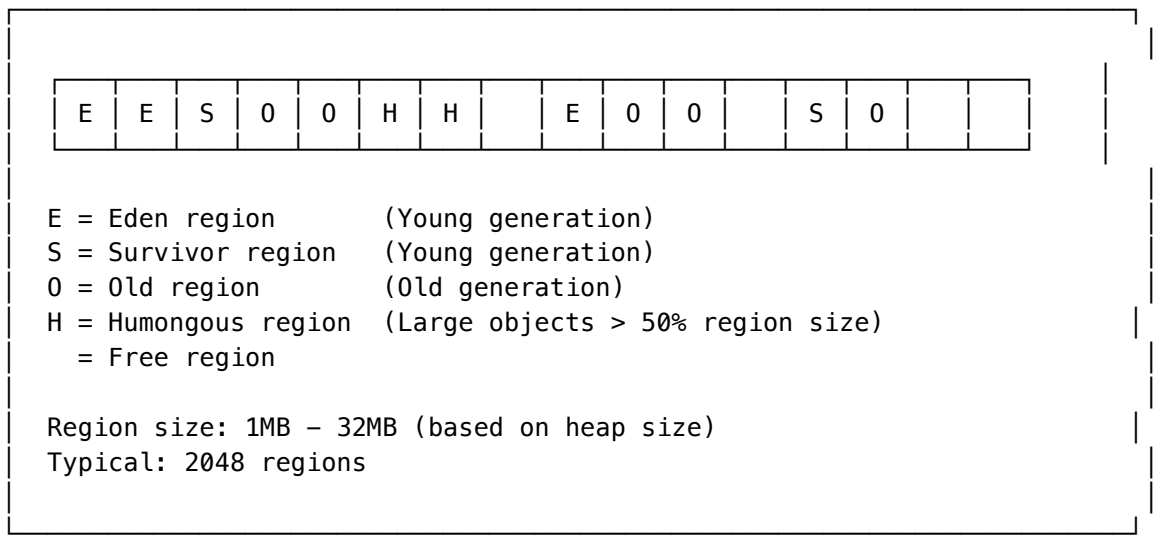


Phase 3: Compact (Optional)



1.5.2 G1 GC Internals (Default GC since Java 9)

G1 Heap Layout (Region-based):



G1 Collection Types:
1. Young GC: Collects all Eden + Survivor regions
2. Mixed GC: Young + selected Old regions (garbage first!)
3. Full GC: Fallback, entire heap (avoid!)

1.6 Java Memory Model (JMM)

1.6.1 Happens-Before Relationships

```java
// JMM defines which memory operations are guaranteed to be visible

// 1. Program Order Rule
// Within a thread, each action happens-before subsequent actions
int a = 1;  // HB
int b = 2;  // This sees a = 1

// 2. Monitor Lock Rule
// Unlock happens-before subsequent lock on same monitor
synchronized (lock) {
    sharedVar = 42;
}  // Unlock HB...
// ...another thread:
synchronized (lock) {  // Lock
    int x = sharedVar;  // Sees 42
}

// 3. Volatile Variable Rule
// Write to volatile happens-before subsequent read
volatile boolean ready = false;
// Thread 1:
data = 42;
ready = true;  // volatile write HB...
// Thread 2:
if (ready) {   // volatile read
    // Guaranteed to see data = 42
}

// 4. Thread Start Rule
// Thread.start() happens-before any action in started thread
data = 42;
thread.start();  // HB
// In thread: sees data = 42

// 5. Thread Join Rule
// All actions in thread happen-before join() returns
// In thread: data = 42
thread.join();  // HB
// After join: sees data = 42
```

1.6.2 Memory Barriers

```
CPU Memory Barriers (enforced by JMM):

┌──────────────────────────────────────────────────────────────────────────┐
│                                                                          │
│  LoadLoad   - Ensures Load1 completes before Load2                       │
│               Load1; LoadLoad; Load2                                     │
│                                                                          │
│  StoreStore - Ensures Store1 visible before Store2                       │
│               Store1; StoreStore; Store2                                 │
│                                                                          │
│  LoadStore  - Ensures Load1 completes before Store2 visible              │
│               Load1; LoadStore; Store2                                   │
│                                                                          │
│  StoreLoad  - Ensures Store1 visible before Load2 (most expensive)       │
│               Store1; StoreLoad; Load2                                   │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘

volatile semantics:
- volatile read:  LoadLoad + LoadStore barrier after
- volatile write: StoreStore barrier before + StoreLoad barrier after
```

1.7 Interview Questions

1.7.1 Q1: What is the difference between stack and heap memory?

Stack	Heap
Per-thread, private	Shared across threads
LIFO order	No particular order
Stores primitives, references	Stores objects
Fast allocation (bump pointer)	Slower (GC managed)
Auto-deallocated on method exit	GC deallocates
Fixed size (-Xss)	Dynamic size (-Xmx)
StackOverflowError if exhausted	OutOfMemoryError

1.7.2 Q2: Explain class loading with code example

```java
// Class is loaded only when first actively used
public class LazyLoading {
    public static void main(String[] args) {
        System.out.println("Main started");

        // Reference to class doesn't cause loading
        Class<?> clazz = MyClass.class;  // Loaded

        // Accessing constant doesn't cause initialization
        int x = MyClass.CONSTANT;  // NOT initialized!

        // This causes full initialization
        MyClass.doSomething();  // Loaded + Initialized
    }
}

class MyClass {
    static final int CONSTANT = 42;  // Compile-time constant
    static int value = initValue();

    static {
        System.out.println("MyClass initialized");
    }

    static int initValue() {
        System.out.println("initValue called");
        return 100;
    }

    static void doSomething() { }
}
```

1.7.3 Q3: How does G1 GC determine which regions to collect?

G1 maintains a priority queue of regions sorted by “garbage first” - regions with most garbage (lowest live data ratio) are collected first, maximizing freed memory with minimal work.

1.7.4 Q4: What causes a Full GC and how to avoid it?

Causes:

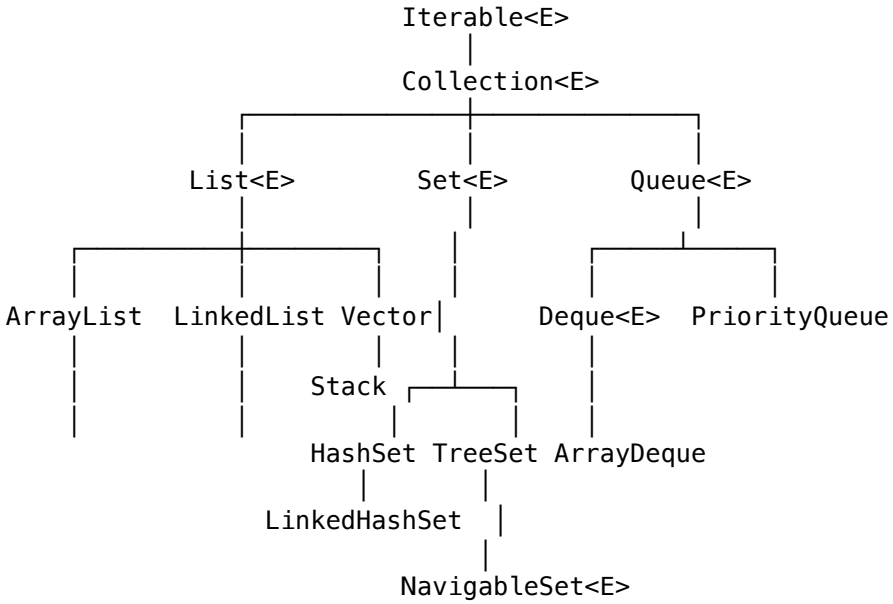
1. Old Generation full
2. Metaspace exhaustion
3. Humongous allocation failure
4. Explicit System.gc() call
5. Promotion failure

Avoidance:

1. Size heap appropriately
2. Tune young generation size
3. Avoid creating large objects
4. Use -XX:+DisableExplicitGC
5. Monitor and tune GC parameters

2 Java Collections Framework - Deep Internals

2.1 Collections Hierarchy



2.2 ArrayList Internals

2.2.1 Internal Structure

ArrayList is backed by a **dynamic array** (Object[]).

```
public class ArrayList<E> extends AbstractList<E>
        implements List<E>, RandomAccess, Cloneable, java.io.Serializable {

    // The array buffer where elements are stored
    transient Object[] elementData;  // non-private for nested class access

    // The size of the ArrayList (number of elements)
    private int size;

    // Default initial capacity
    private static final int DEFAULT_CAPACITY = 10;

    // Shared empty array for empty instances
    private static final Object[] EMPTY_ELEMENTDATA = {};

    // Shared empty array for default sized empty instances
    private static final Object[] DEFAULTCAPACITY_EMPTY_ELEMENTDATA = {};
}
```

2.2.2 Growth Strategy (Amortized O(1) Add)

```java
// When capacity is exceeded, ArrayList grows by 50%
private void grow(int minCapacity) {
    int oldCapacity = elementData.length;

    // New capacity = old capacity + old capacity / 2 (1.5x growth)
    int newCapacity = oldCapacity + (oldCapacity >> 1);

    if (newCapacity - minCapacity < 0)
        newCapacity = minCapacity;

    if (newCapacity - MAX_ARRAY_SIZE > 0)
        newCapacity = hugeCapacity(minCapacity);

    // Copy elements to new array
    elementData = Arrays.copyOf(elementData, newCapacity);
}
```

2.2.3 Why 1.5x Growth Factor?

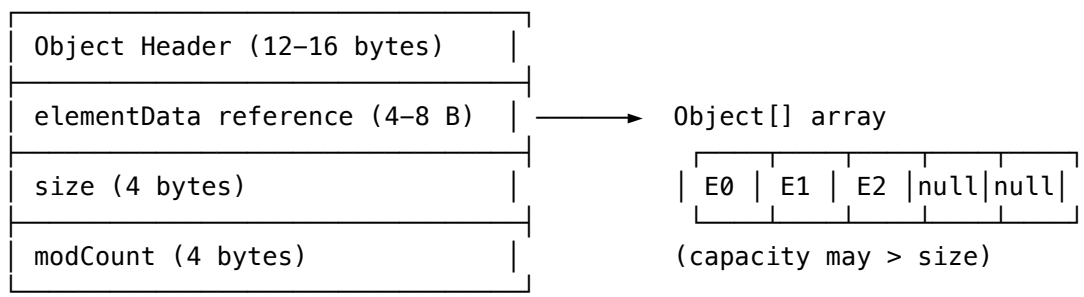
Growth Factor	Memory Waste	Reallocations for N elements
2x (doubling)	Up to 50%	O(log N)
1.5x	Up to 33%	O(log N) but more copies
1.25x	Up to 20%	More frequent copies

Trade-off: 1.5x balances memory efficiency with copy overhead.

2.2.4 Time Complexity Analysis

Operation	Average Case	Worst Case	Notes
get(i)	O(1)	O(1)	Direct array access
add(E)	O(1)*	O(n)	Amortized; worst when resize
add(i, E)	O(n)	O(n)	Shift elements right
remove(i)	O(n)	O(n)	Shift elements left
contains()	O(n)	O(n)	Linear search
indexOf()	O(n)	O(n)	Linear search

2.2.5 Memory Layout



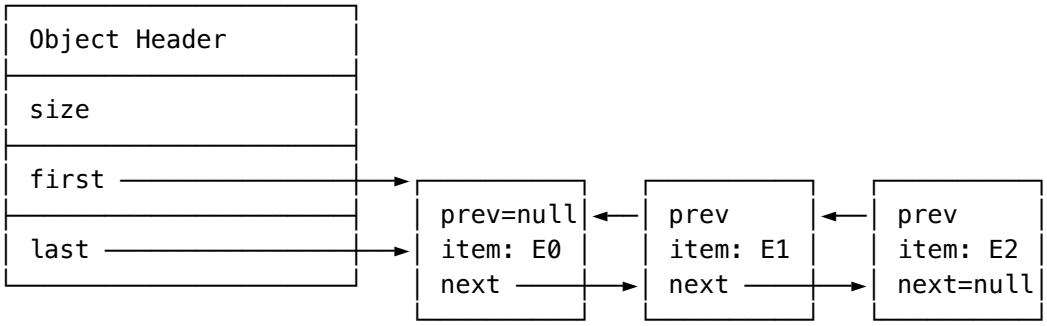
2.3 LinkedList Internals

2.3.1 Internal Structure (Doubly Linked List)

```java
public class LinkedList<E> extends AbstractSequentialList<E>
        implements List<E>, Deque<E>, Cloneable, java.io.Serializable {

    transient int size = 0;
    transient Node<E> first;  // Pointer to first node
    transient Node<E> last;   // Pointer to last node

    // Node inner class
    private static class Node<E> {
        E item;
        Node<E> next;
        Node<E> prev;

        Node(Node<E> prev, E element, Node<E> next) {
            this.item = element;
            this.next = next;
            this.prev = prev;
        }
    }
}
```

2.3.2 Memory Layout

LinkedList object:



2.3.3 Time Complexity Comparison

Operation	ArrayList	LinkedList
get(i)	O(1)	O(n)
add(E) at end	O(1)*	O(1)
add(i, E)	O(n)	O(n)†
remove(i)	O(n)	O(n)†
contains()	O(n)	O(n)
Iterator.remove()	O(n)	O(1)

*Amortized †Traversal + O(1) operation

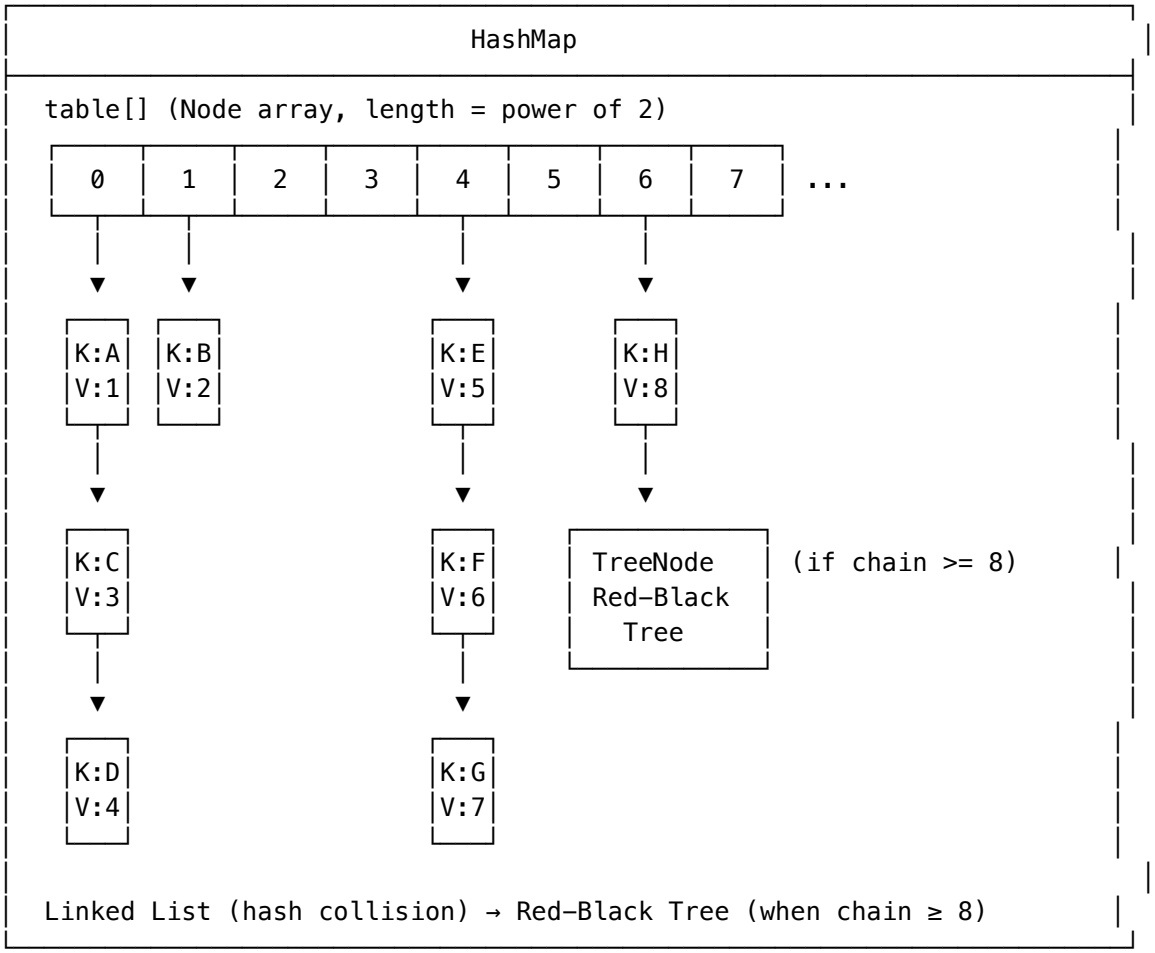
2.4 HashMap Internals

2.4.1 Structure (Java 8+)

```
public class HashMap<K,V> extends AbstractMap<K,V>
        implements Map<K,V>, Cloneable, Serializable {

    // The table, resized as necessary. Length MUST be power of two.
    transient Node<K,V>[] table;

    // Number of key-value mappings
    transient int size;

    // Threshold for resizing (capacity * loadFactor)
    int threshold;

    // Load factor (default 0.75)
    final float loadFactor;

    // Modification count for fail-fast iterators
    transient int modCount;

    // Constants
    static final int DEFAULT_INITIAL_CAPACITY = 16;  // Must be power of 2
    static final float DEFAULT_LOAD_FACTOR = 0.75f;
    static final int TREEIFY_THRESHOLD = 8;   // Convert to tree
    static final int UNTREEIFY_THRESHOLD = 6;  // Convert back to list
    static final int MIN_TREEIFY_CAPACITY = 64;

    // Node structure (linked list node)
    static class Node<K,V> implements Map.Entry<K,V> {
        final int hash;
        final K key;
        V value;
        Node<K,V> next;
    }

    // TreeNode structure (red-black tree node)
    static final class TreeNode<K,V> extends LinkedHashMap.Entry<K,V> {
        TreeNode<K,V> parent;
        TreeNode<K,V> left;
        TreeNode<K,V> right;
        TreeNode<K,V> prev;
        boolean red;
    }
}
```

2.4.2 HashMap Visual Layout (Java 8+)

HashMap Internal Structure:



2.4.3 Hash Function and Index Calculation

```
// How HashMap computes bucket index

static final int hash(Object key) {
    int h;
    // XOR high bits with low bits (spread hash codes)
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}

// Bucket index calculation
// Using bitwise AND (much faster than modulo)
int index = (n - 1) & hash;  // where n = table.length (power of 2)

// Example:
// hash = 0b10110101_11001010_00110011_11110000
// n = 16, n-1 = 15 = 0b00001111
// index = hash & 0b00001111 = 0b00000000 = 0 (uses only last 4 bits)

// Why XOR with h >>> 16?
// Without: only low bits determine bucket (many collisions)
// With XOR: high bits influence low bits (better distribution)
```

2.4.4 Put Operation Internals

```java
// Simplified put() implementation
final V putVal(int hash, K key, V value, boolean onlyIfAbsent, boolean evict) {
    Node<K,V>[] tab; Node<K,V> p; int n, i;

    // 1. Initialize table if empty
    if ((tab = table) == null || (n = tab.length) == 0)
        n = (tab = resize()).length;

    // 2. If bucket is empty, create new node
    if ((p = tab[i = (n - 1) & hash]) == null)
        tab[i] = newNode(hash, key, value, null);
    else {
        Node<K,V> e; K k;

        // 3. Check if first node matches key
        if (p.hash == hash && ((k = p.key) == key || (key != null && key.equals(k))))
            e = p;

        // 4. If tree node, use tree insertion
        else if (p instanceof TreeNode)
            e = ((TreeNode<K,V>)p).putTreeVal(this, tab, hash, key, value);

        // 5. Traverse linked list
        else {
            for (int binCount = 0; ; ++binCount) {
                if ((e = p.next) == null) {
                    p.next = newNode(hash, key, value, null);
                    // Convert to tree if threshold reached
                    if (binCount >= TREEIFY_THRESHOLD - 1)
                        treeifyBin(tab, hash);
                    break;
                }
                if (e.hash == hash && ((k = e.key) == key || (key != null && key.equals(k))))
                    break;
                p = e;
            }
        }

        // 6. Update existing value
        if (e != null) {
            V oldValue = e.value;
            if (!onlyIfAbsent || oldValue == null)
                e.value = value;
            return oldValue;
        }
    }

    // 7. Check if resize needed
    if (++size > threshold)
        resize();
    return null;
}
```

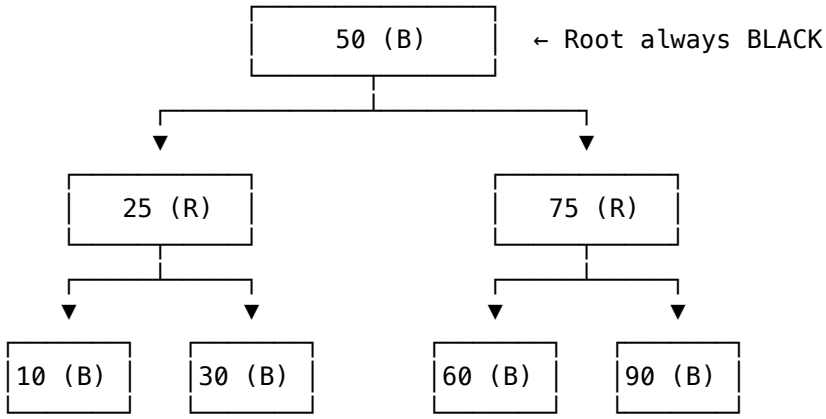
2.4.5 Resize Operation

```java
// HashMap doubles in size when size > capacity * loadFactor

final Node<K,V>[] resize() {
    Node<K,V>[] oldTab = table;
    int oldCap = (oldTab == null) ? 0 : oldTab.length;
    int oldThr = threshold;
    int newCap, newThr = 0;

    // Double capacity
    if (oldCap > 0) {
        newCap = oldCap << 1;  // Double
        newThr = oldThr << 1;
    }

    // Rehash all entries
    Node<K,V>[] newTab = (Node<K,V>[])new Node[newCap];
    table = newTab;

    // Clever rehashing: entry goes to same index OR index + oldCap
    // Based on one bit: (hash & oldCap) == 0 ? same : same + oldCap
    for (int j = 0; j < oldCap; ++j) {
        Node<K,V> e;
        if ((e = oldTab[j]) != null) {
            oldTab[j] = null;
            if (e.next == null)
                newTab[e.hash & (newCap - 1)] = e;
            else {
                // Split bucket into two lists: lo (same index), hi (index + oldCap)
                Node<K,V> loHead = null, loTail = null;
                Node<K,V> hiHead = null, hiTail = null;
                Node<K,V> next;
                do {
                    next = e.next;
                    if ((e.hash & oldCap) == 0) {
                        // Stays in same bucket
                        if (loTail == null) loHead = e;
                        else loTail.next = e;
                        loTail = e;
                    } else {
                        // Moves to bucket + oldCap
                        if (hiTail == null) hiHead = e;
                        else hiTail.next = e;
                        hiTail = e;
                    }
                } while ((e = next) != null);

                if (loTail != null) {
                    loTail.next = null;
                    newTab[j] = loHead;
                }
                if (hiTail != null) {
                    hiTail.next = null;
                    newTab[j + oldCap] = hiHead;
                }
            }
        }
    }
    return newTab;
}
```

2.5 TreeMap Internals

2.5.1 Red-Black Tree Properties

```
Red-Black Tree Rules:
1. Every node is either RED or BLACK
2. Root is always BLACK
3. Every leaf (NIL) is BLACK
4. If a node is RED, both children are BLACK
5. Every path from root to leaves has same number of BLACK nodes

Visual Example:
```




```
Black Height: Every path has exactly 2 black nodes (excluding root)
Height: At most 2*log(n+1) → O(log n) operations guaranteed
```

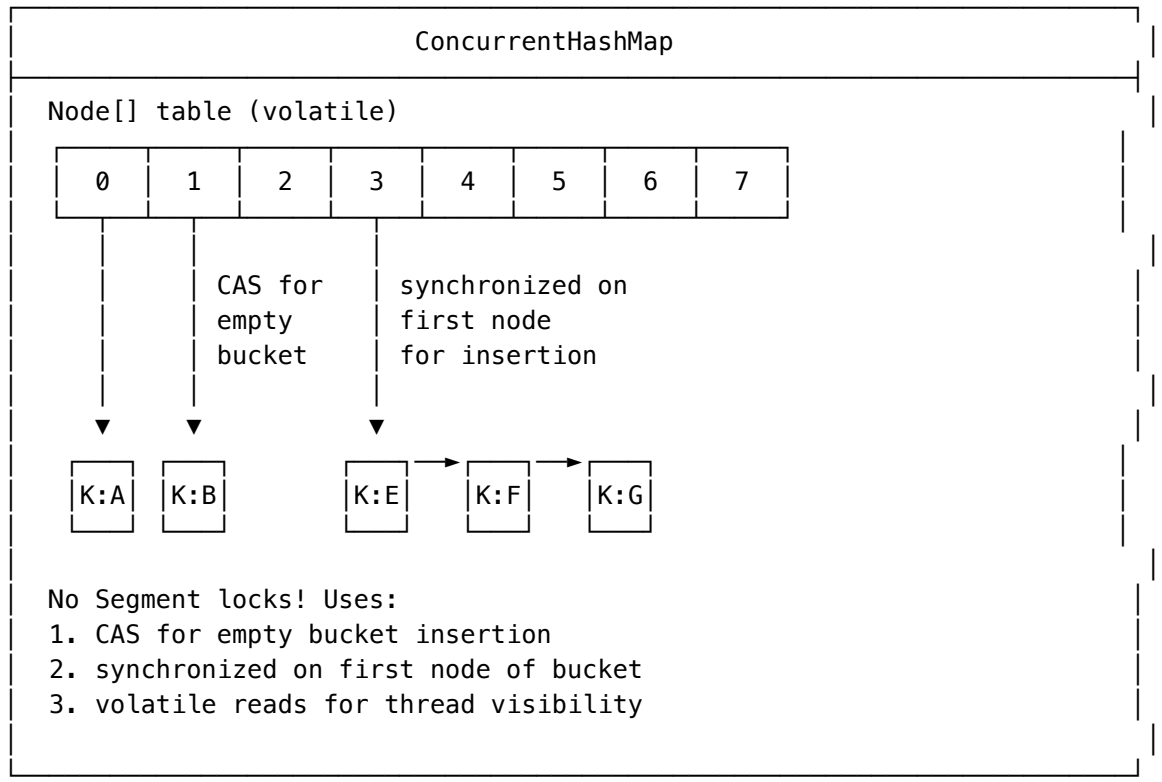
2.5.2 TreeMap Implementation

```java
public class TreeMap<K,V> extends AbstractMap<K,V>
        implements NavigableMap<K,V>, Cloneable, java.io.Serializable {

    private final Comparator<? super K> comparator;
    private transient Entry<K,V> root;
    private transient int size = 0;

    static final class Entry<K,V> implements Map.Entry<K,V> {
        K key;
        V value;
        Entry<K,V> left;
        Entry<K,V> right;
        Entry<K,V> parent;
        boolean color = BLACK;  // New nodes start as BLACK
    }

    // All operations O(log n):
    // get(), put(), remove(), containsKey()
    // firstKey(), lastKey(), higherKey(), lowerKey()
    // subMap(), headMap(), tailMap()
}
```

2.6 ConcurrentHashMap Internals

2.6.1 Java 8+ Implementation (Lock Striping → CAS + synchronized)

ConcurrentHashMap Structure (Java 8+):



2.6.2 Key Operations

```java
// put() - Uses CAS for empty bucket, synchronized for non-empty
final V putVal(K key, V value, boolean onlyIfAbsent) {
    int hash = spread(key.hashCode());
    for (Node<K,V>[] tab = table;;) {
        Node<K,V> f; int n, i, fh;
        if (tab == null || (n = tab.length) == 0)
            tab = initTable();  // CAS-based lazy init
        else if ((f = tabAt(tab, i = (n - 1) & hash)) == null) {
            // CAS to insert into empty bucket
            if (casTabAt(tab, i, null, new Node<K,V>(hash, key, value, null)))
                break;
        }
        else if ((fh = f.hash) == MOVED)
            tab = helpTransfer(tab, f);  // Help with resize
        else {
            // synchronized on first node only
            synchronized (f) {
                // Insert into chain/tree
            }
        }
    }
}

// get() - Lock-free! Uses volatile reads
public V get(Object key) {
    Node<K,V>[] tab; Node<K,V> e, p; int n, eh; K ek;
    int h = spread(key.hashCode());
    if ((tab = table) != null && (n = tab.length) > 0 &&
        (e = tabAt(tab, (n - 1) & h)) != null) {  // volatile read
        // Traverse without locking
    }
    return null;
}
```

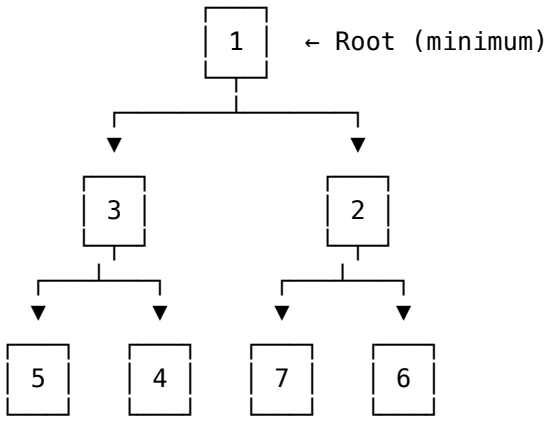
2.7 PriorityQueue and Heaps

2.7.1 Binary Heap Implementation

```java
public class PriorityQueue<E> extends AbstractQueue<E> {
    transient Object[] queue;  // Binary heap stored in array
    private int size = 0;
    private final Comparator<? super E> comparator;

    // Parent/child relationships (0-indexed)
    // Parent of node i: (i - 1) / 2
    // Left child of i:  2 * i + 1
    // Right child of i: 2 * i + 2
}
```

2.7.2 Heap Visual Representation

```
Binary Min-Heap:
```




```
Array representation:
Index:    0    1    2    3    4    5    6
Value:  [ 1 ][ 3 ][ 2 ][ 5 ][ 4 ][ 7 ][ 6 ]

Operations:
- offer(E): O(log n) - Add at end, sift up
- poll():   O(log n) - Remove root, move last to root, sift down
- peek():   O(1)     - Return root
```

2.8 Interview Questions

2.8.1 Q1: Why does HashMap require capacity to be power of 2?

Answer: For fast index calculation using bitwise AND: - `index = hash & (capacity - 1)` is equivalent to `hash % capacity` - Bitwise AND is much faster than modulo division - Only works when capacity is power of 2

2.8.2 Q2: HashMap vs TreeMap vs LinkedHashMap?

Feature	HashMap	TreeMap	LinkedHashMap
Order	None	Sorted by keys	Insertion order
Get/Put	O(1)	O(log n)	O(1)
Null keys	1 allowed	Not allowed	1 allowed
Iteration	Unpredictable	Sorted	Predictable
Implementation	Hash table	Red-Black Tree	Hash + Linked List

2.8.3 Q3: When does ArrayList.add() become O(n)?

Answer: When the internal array needs to be resized (capacity exceeded). This involves creating a new array 1.5x larger and copying all elements. However, this is rare (amortized O(1)).

2.8.4 Q4: How does ConcurrentHashMap achieve thread-safety without blocking reads?

Answer: 1. Volatile reads of table array 2. Volatile writes when updating nodes 3. CAS for empty bucket insertion 4. Fine-grained synchronization only on bucket's first node 5. Safe publication through volatile variables

3 Java Concurrency and Multithreading - Deep Dive

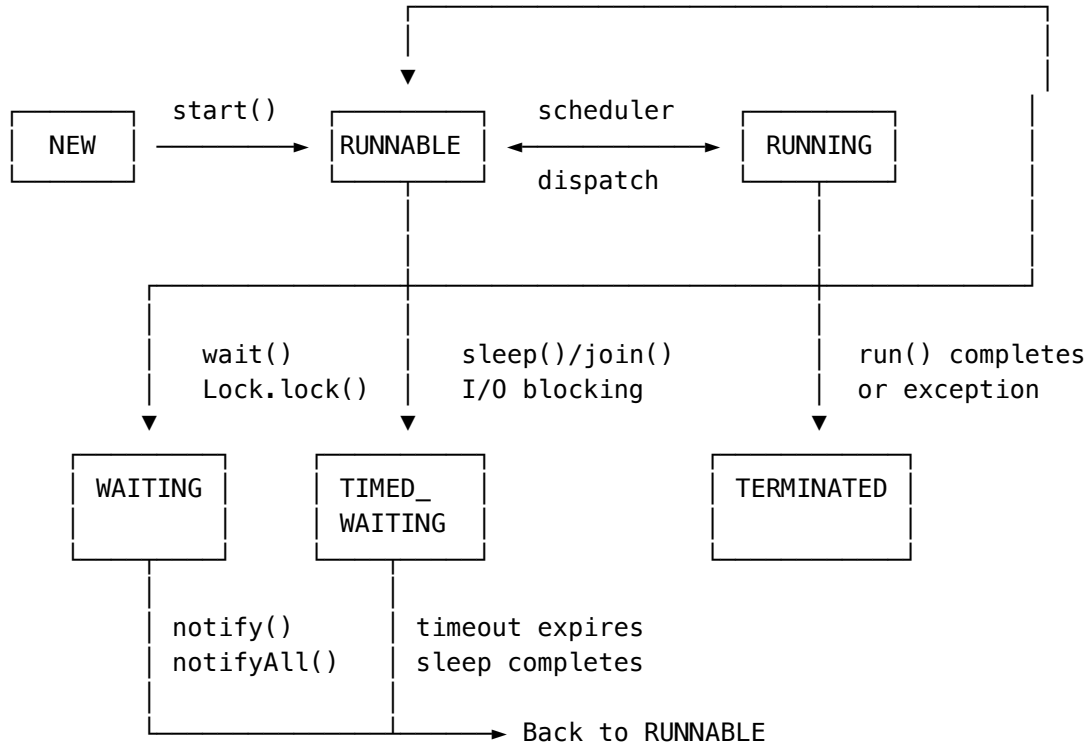
3.1 Thread Fundamentals

3.1.1 Thread Creation Methods

```java
// Method 1: Extending Thread class
class MyThread extends Thread {
    @Override
    public void run() {
        System.out.println("Thread running: " + Thread.currentThread().getName());
    }
}

// Method 2: Implementing Runnable (Preferred)
class MyRunnable implements Runnable {
    @Override
    public void run() {
        System.out.println("Runnable running: " + Thread.currentThread().getName());
    }
}

// Method 3: Using Callable (Returns value, can throw exceptions)
class MyCallable implements Callable<Integer> {
    @Override
    public Integer call() throws Exception {
        return 42;
    }
}

// Method 4: Lambda expressions (Java 8+)
Runnable r = () -> System.out.println("Lambda thread");

// Usage
public class ThreadDemo {
    public static void main(String[] args) throws Exception {
        // Method 1
        Thread t1 = new MyThread();
        t1.start();

        // Method 2
        Thread t2 = new Thread(new MyRunnable());
        t2.start();

        // Method 3
        ExecutorService executor = Executors.newSingleThreadExecutor();
        Future<Integer> future = executor.submit(new MyCallable());
        System.out.println("Callable result: " + future.get());
        executor.shutdown();

        // Method 4
        new Thread(() -> System.out.println("Lambda")).start();
    }
}
```

3.2 Thread Lifecycle and States

3.2.1 Thread State Diagram



3.2.2 Thread States in Code

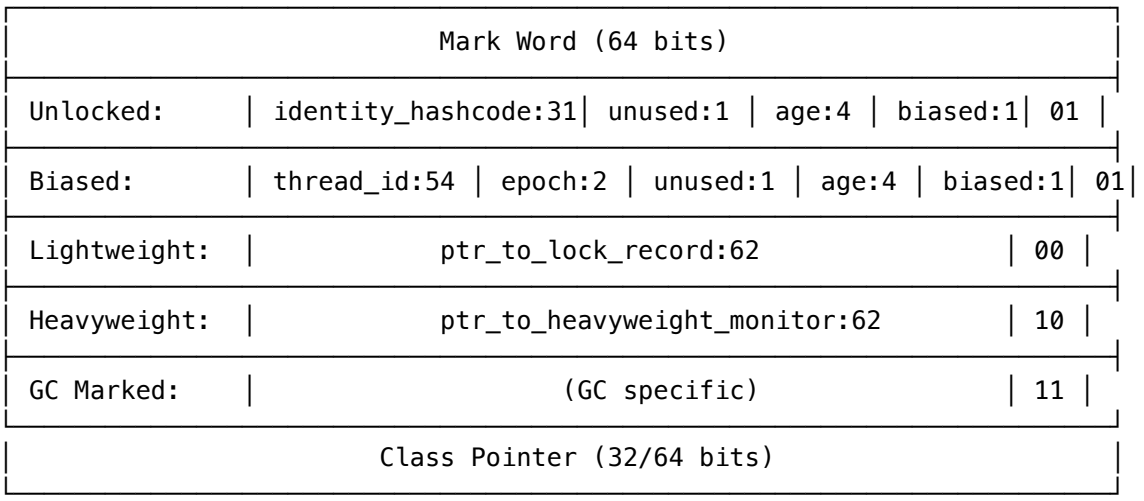
```
public class ThreadStateDemo {
    public static void main(String[] args) throws InterruptedException {
        Object lock = new Object();

        Thread t = new Thread(() -> {
            synchronized (lock) {
                try {
                    lock.wait();  // WAITING state
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            }
        });

        System.out.println("After creation: " + t.getState());  // NEW

        t.start();
        Thread.sleep(100);
        System.out.println("After start (waiting): " + t.getState());  // WAITING

        synchronized (lock) {
            lock.notify();
        }
        t.join();
        System.out.println("After completion: " + t.getState());  // TERMINATED
    }
}
```

3.3 Synchronization Internals

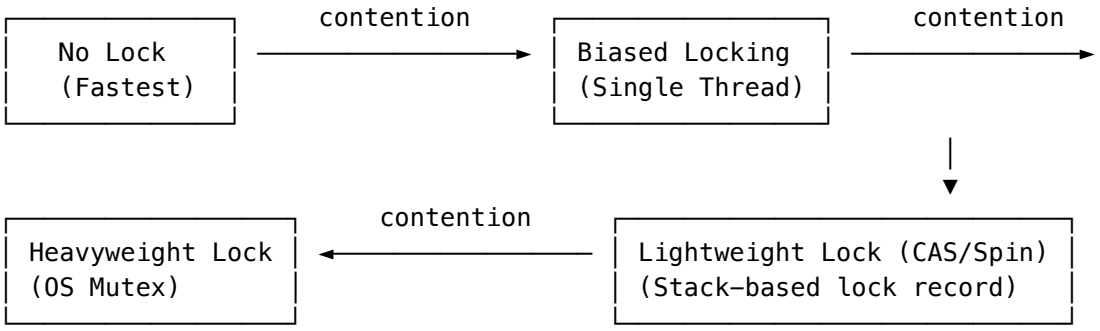
3.3.1 Object Monitor Structure (HotSpot JVM)

Every Java object has an associated monitor. The object header contains mark word that stores lock state.

```
Object Header Layout (64-bit JVM):
```



3.3.2 Lock Escalation Path



3.3.3 synchronized Block Bytecode

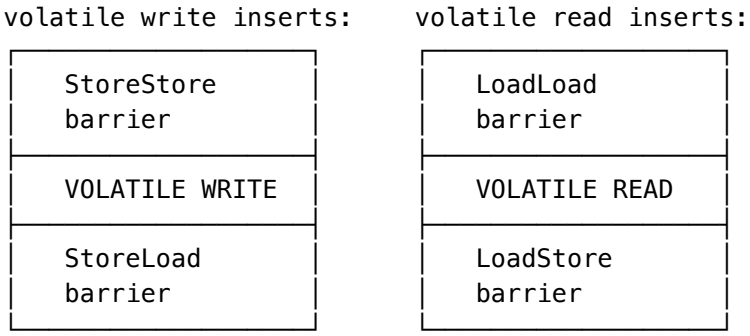
```java
// Source code
public void syncMethod() {
    synchronized (this) {
        // critical section
    }
}

// Bytecode (simplified)
public void syncMethod();
  Code:
    0: aload_0           // Load 'this'
    1: dup               // Duplicate for exception handler
    2: astore_1          // Store lock reference
    3: monitorenter      // ← Acquire monitor
    4: aload_1
    5: monitorexit       // ← Release monitor (normal path)
    6: goto 14
    9: astore_2          // Exception handler
   10: aload_1
   11: monitorexit       // ← Release monitor (exception path)
   12: aload_2
   13: athrow
   14: return
```

3.4 volatile Keyword

3.4.1 What volatile Guarantees

```java
// volatile provides:
// 1. Visibility: Changes visible to all threads immediately
// 2. Ordering: Prevents instruction reordering (memory barrier)

// volatile does NOT provide:
// 1. Atomicity for compound operations (i++, check-then-act)

public class VolatileDemo {
    private volatile boolean running = true;
    private volatile int counter = 0;

    // CORRECT: Simple flag
    public void stop() {
        running = false;  // Immediately visible to other threads
    }

    public void runLoop() {
        while (running) {  // Always reads fresh value
            doWork();
        }
    }

    // INCORRECT: volatile doesn't make i++ atomic!
    public void increment() {
        counter++;  // NOT ATOMIC! Read-Modify-Write
    }

    // CORRECT: Use AtomicInteger instead
    private AtomicInteger atomicCounter = new AtomicInteger(0);
    public void safeIncrement() {
        atomicCounter.incrementAndGet();  // Atomic CAS operation
    }
}
```

3.4.2 Memory Barriers



Barriers prevent reordering across the barrier point.

3.4.3 Double-Checked Locking Pattern

```java
// Singleton with proper double-checked locking
public class Singleton {
    // volatile prevents instruction reordering
    private static volatile Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) {                 // First check (no locking)
            synchronized (Singleton.class) {
                if (instance == null) {         // Second check (with lock)
                    instance = new Singleton(); // Safe with volatile
                }
            }
        }
        return instance;
    }
}

// Why volatile is needed:
// Without volatile, this can happen:
// 1. Memory allocated for Singleton
// 2. instance assigned to memory address  ← Other thread sees non-null!
// 3. Constructor executes                 ← But object not initialized!
// volatile prevents steps 2 and 3 from being reordered
```

3.5 Locks and Monitors

3.5.1 ReentrantLock vs synchronized

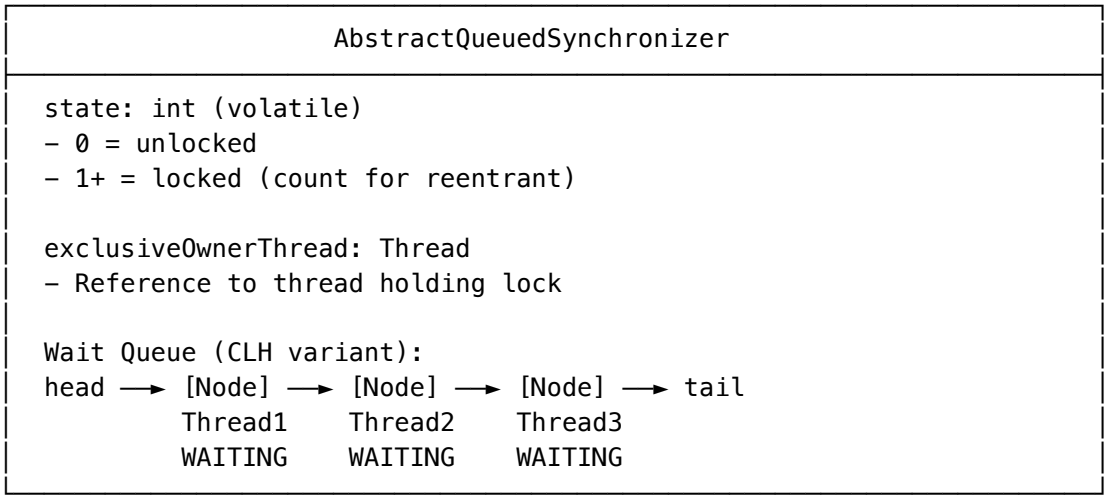
```java
import java.util.concurrent.locks.*;

public class LockComparison {

    // synchronized - implicit lock
    private final Object lock = new Object();

    public void synchronizedMethod() {
        synchronized (lock) {
            // critical section
        }
    }

    // ReentrantLock - explicit lock with more features
    private final ReentrantLock reentrantLock = new ReentrantLock();

    public void reentrantLockMethod() {
        reentrantLock.lock();
        try {
            // critical section
        } finally {
            reentrantLock.unlock();  // ALWAYS in finally!
        }
    }

    // ReentrantLock advanced features
    public void advancedFeatures() throws InterruptedException {
        // 1. Try lock with timeout
        if (reentrantLock.tryLock(1, TimeUnit.SECONDS)) {
            try {
                // Got the lock within 1 second
            } finally {
                reentrantLock.unlock();
            }
        } else {
            // Couldn't get lock, do something else
        }

        // 2. Interruptible lock acquisition
        reentrantLock.lockInterruptibly();  // Can be interrupted while waiting

        // 3. Check if lock is held
        boolean isHeld = reentrantLock.isLocked();
        boolean isHeldByMe = reentrantLock.isHeldByCurrentThread();
        int holdCount = reentrantLock.getHoldCount();  // For reentrant locks
    }
}
```

3.5.2 Comparison Table

Feature	synchronized	ReentrantLock
Automatic release	Yes	No (finally required)
Try with timeout	No	Yes (tryLock)
Interruptible	No	Yes (lockInterruptibly)
Fair locking	No	Yes (optional)
Multiple conditions	No (1 wait set)	Yes (multiple Conditions)
Performance	Similar (JDK 6+)	Similar

3.5.3 ReentrantLock Internals (AbstractQueuedSynchronizer)

ReentrantLock uses AQS (AbstractQueuedSynchronizer):




```
Lock acquisition:
1. CAS to change state from 0 to 1
2. If successful, set exclusiveOwnerThread
3. If failed, enqueue in wait queue, park thread

Unlock:
1. Set state to 0, clear exclusiveOwnerThread
2. Unpark head of wait queue
```

3.5.4 ReadWriteLock

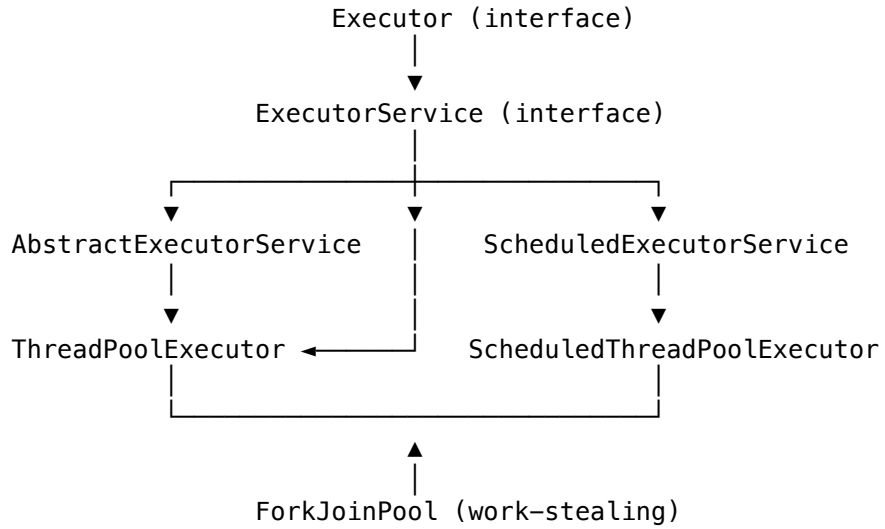
```java
public class ReadWriteLockDemo {
    private final ReadWriteLock rwLock = new ReentrantReadWriteLock();
    private final Lock readLock = rwLock.readLock();
    private final Lock writeLock = rwLock.writeLock();

    private Map<String, Object> cache = new HashMap<>();

    // Multiple readers can access simultaneously
    public Object read(String key) {
        readLock.lock();
        try {
            return cache.get(key);
        } finally {
            readLock.unlock();
        }
    }

    // Writers have exclusive access
    public void write(String key, Object value) {
        writeLock.lock();
        try {
            cache.put(key, value);
        } finally {
            writeLock.unlock();
        }
    }
}
```

3.5.5 StampedLock (Java 8+)

```java
public class StampedLockDemo {
    private final StampedLock sl = new StampedLock();
    private double x, y;

    // Optimistic read (no blocking!)
    public double distanceFromOrigin() {
        // Try optimistic read first
        long stamp = sl.tryOptimisticRead();
        double currentX = x, currentY = y;

        // Validate that no write occurred
        if (!sl.validate(stamp)) {
            // Optimistic read failed, use regular read lock
            stamp = sl.readLock();
            try {
                currentX = x;
                currentY = y;
            } finally {
                sl.unlockRead(stamp);
            }
        }
        return Math.sqrt(currentX * currentX + currentY * currentY);
    }

    // Write lock
    public void move(double deltaX, double deltaY) {
        long stamp = sl.writeLock();
        try {
            x += deltaX;
            y += deltaY;
        } finally {
            sl.unlockWrite(stamp);
        }
    }
}
```

3.6 Thread Pools and Executors

3.6.1 Executor Framework Hierarchy

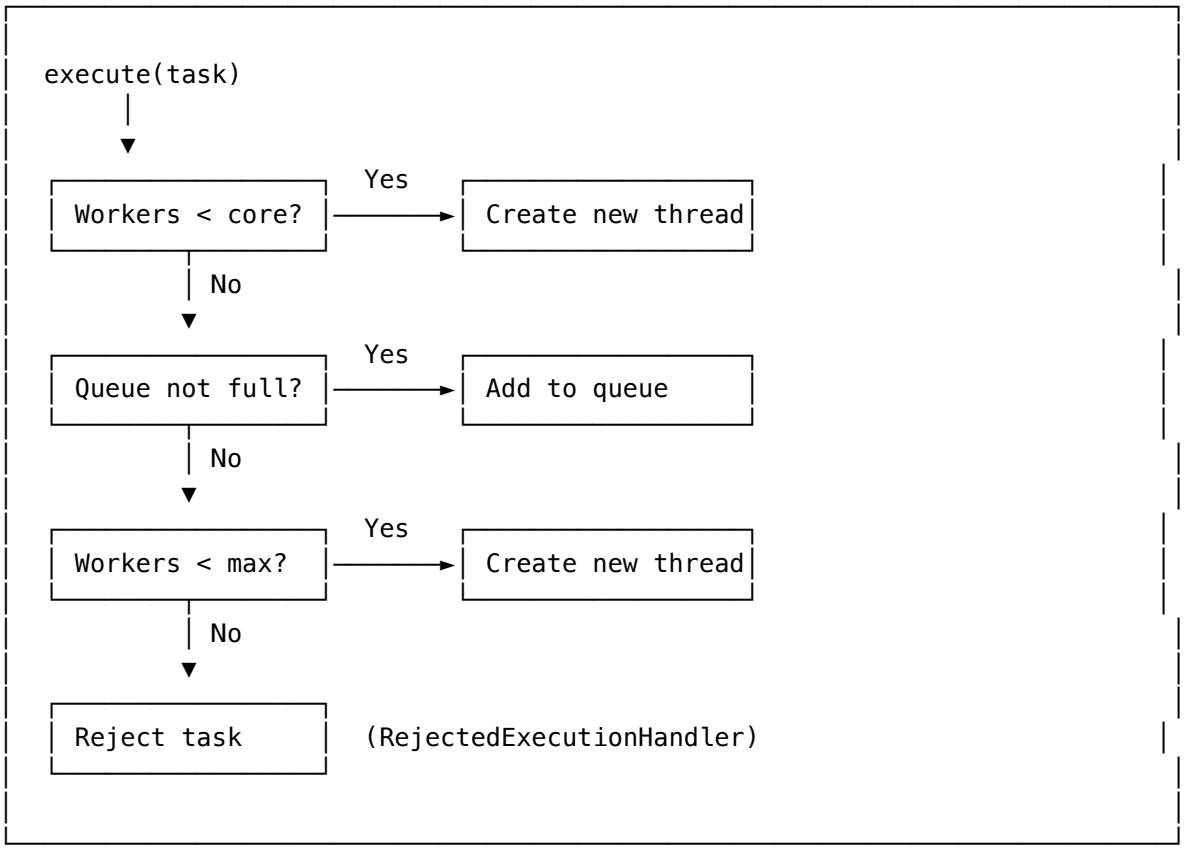


3.6.2 ThreadPoolExecutor Parameters

```java
public ThreadPoolExecutor(
    int corePoolSize,        // Min threads kept alive
    int maximumPoolSize,     // Max threads allowed
    long keepAliveTime,      // Idle thread timeout
    TimeUnit unit,           // Time unit for keepAlive
    BlockingQueue<Runnable> workQueue,  // Task queue
    ThreadFactory threadFactory,        // Creates new threads
    RejectedExecutionHandler handler    // Rejection policy
);
```

3.6.3 Task Execution Flow

Task Submission Flow:



3.6.4 Common Thread Pools

```java
public class ThreadPoolExamples {

    // Fixed thread pool - bounded, fixed size
    ExecutorService fixed = Executors.newFixedThreadPool(4);
    // Uses LinkedBlockingQueue (unbounded) - can cause OOM!

    // Cached thread pool - grows as needed
    ExecutorService cached = Executors.newCachedThreadPool();
    // Uses SynchronousQueue - no capacity, creates new threads
    // Can create unlimited threads!

    // Single thread - sequential execution
    ExecutorService single = Executors.newSingleThreadExecutor();

    // Scheduled - delayed/periodic tasks
    ScheduledExecutorService scheduled = Executors.newScheduledThreadPool(2);
    scheduled.scheduleAtFixedRate(() -> doTask(), 0, 1, TimeUnit.SECONDS);

    // Work-stealing (Java 8+) - ForkJoinPool
    ExecutorService workStealing = Executors.newWorkStealingPool();

    // Custom thread pool (RECOMMENDED for production)
    ExecutorService custom = new ThreadPoolExecutor(
        4,                    // corePoolSize
        16,                   // maximumPoolSize
        60L, TimeUnit.SECONDS, // keepAliveTime
        new ArrayBlockingQueue<>(100),  // Bounded queue!
        new ThreadPoolExecutor.CallerRunsPolicy()  // Rejection policy
    );
}
```

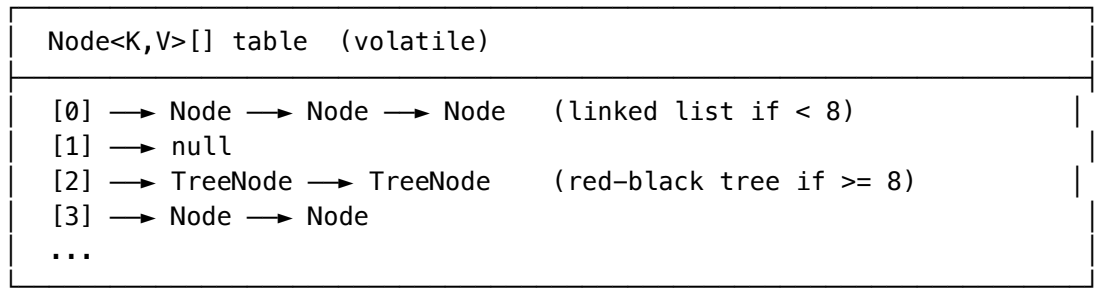
3.6.5 Rejection Policies

```java
// When pool and queue are full:

// 1. AbortPolicy (default) - throws RejectedExecutionException
new ThreadPoolExecutor.AbortPolicy();

// 2. CallerRunsPolicy - caller thread runs the task
new ThreadPoolExecutor.CallerRunsPolicy();

// 3. DiscardPolicy - silently discard task
new ThreadPoolExecutor.DiscardPolicy();

// 4. DiscardOldestPolicy - discard oldest in queue
new ThreadPoolExecutor.DiscardOldestPolicy();

// 5. Custom policy
RejectedExecutionHandler custom = (r, executor) -> {
    // Log, persist, or handle rejection
    logger.warn("Task rejected: " + r.toString());
};
```

3.7 Concurrent Collections

3.7.1 ConcurrentHashMap Internals

ConcurrentHashMap (Java 8+):



Key features:
- No segment locks (unlike Java 7)
- Per-bucket synchronization using CAS + synchronized
- Lock-free reads using volatile
- Treeify threshold: 8 nodes → red-black tree
- Untreeify threshold: 6 nodes → linked list

3.7.2 ConcurrentHashMap Operations

```
public class ConcurrentHashMapDemo {
    private ConcurrentHashMap<String, Integer> map = new ConcurrentHashMap<>();

    // Atomic compound operations
    public void atomicOperations() {
        // putIfAbsent - only puts if key doesn't exist
        map.putIfAbsent("key", 1);

        // compute - atomic read-modify-write
        map.compute("key", (k, v) -> v == null ? 1 : v + 1);

        // computeIfAbsent - lazy initialization
        map.computeIfAbsent("key", k -> expensiveComputation(k));

        // merge - combine old and new values
        map.merge("key", 1, Integer::sum);  // Increment or set to 1

        // replace - conditional replace
        map.replace("key", 1, 2);  // Only if value is 1
    }

    // Bulk operations (Java 8+)
    public void bulkOperations() {
        // forEach with parallelism threshold
        map.forEach(2, (k, v) -> System.out.println(k + "=" + v));

        // search - find first matching entry
        String found = map.search(2, (k, v) -> v > 10 ? k : null);

        // reduce - aggregate values
        int sum = map.reduceValues(2, Integer::sum);
    }
}
```

3.7.3 CopyOnWriteArrayList

```java
// Best for read-heavy, write-rare scenarios
public class CopyOnWriteDemo {
    private CopyOnWriteArrayList<String> list = new CopyOnWriteArrayList<>();

    // Write creates new array copy (expensive!)
    public void add(String item) {
        list.add(item);  // O(n) - copies entire array
    }

    // Reads never block, always see consistent snapshot
    public void iterate() {
        // Iterator uses snapshot - won't see concurrent modifications
        for (String item : list) {
            // Never throws ConcurrentModificationException
            System.out.println(item);
        }
    }
}
```

3.7.4 BlockingQueue Implementations

```java
public class BlockingQueueDemo {

    // ArrayBlockingQueue - bounded, array-backed
    BlockingQueue<String> array = new ArrayBlockingQueue<>(100);

    // LinkedBlockingQueue - optionally bounded, linked nodes
    BlockingQueue<String> linked = new LinkedBlockingQueue<>(100);

    // PriorityBlockingQueue - unbounded, priority-ordered
    BlockingQueue<Task> priority = new PriorityBlockingQueue<>();

    // DelayQueue - elements become available after delay
    BlockingQueue<DelayedTask> delay = new DelayQueue<>();

    // SynchronousQueue - no capacity, direct handoff
    BlockingQueue<String> sync = new SynchronousQueue<>();
}
```

3.8 Common Concurrency Patterns

3.8.1 1. Producer-Consumer

```
public class ProducerConsumer {
    private final BlockingQueue<Task> queue = new ArrayBlockingQueue<>(10);
    private volatile boolean running = true;

    // Producer
    class Producer implements Runnable {
        @Override
        public void run() {
            while (running) {
                Task task = generateTask();
                try {
                    queue.put(task);  // Blocks if full
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    break;
                }
            }
        }
    }

    // Consumer
    class Consumer implements Runnable {
        @Override
        public void run() {
            while (running || !queue.isEmpty()) {
                try {
                    Task task = queue.poll(100, TimeUnit.MILLISECONDS);
                    if (task != null) {
                        process(task);
                    }
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    break;
                }
            }
        }
    }
}
```

3.8.2 2. CountDownLatch (Wait for N events)

```
public class CountDownLatchDemo {
    public void parallelStartup() throws InterruptedException {
        int serviceCount = 5;
        CountDownLatch latch = new CountDownLatch(serviceCount);

        for (int i = 0; i < serviceCount; i++) {
            new Thread(() -> {
                try {
                    initializeService();
                } finally {
                    latch.countDown();  // Decrement count
                }
            }).start();
        }

        latch.await();  // Wait until count reaches 0
        System.out.println("All services initialized!");
    }
}
```

3.8.3 3. CyclicBarrier (Wait for all threads at barrier)

```
public class CyclicBarrierDemo {
    public void parallelPhases() {
        int threadCount = 4;
        CyclicBarrier barrier = new CyclicBarrier(threadCount,
            () -> System.out.println("Phase complete!"));  // Barrier action

        for (int i = 0; i < threadCount; i++) {
            new Thread(() -> {
                try {
                    phase1();
                    barrier.await();  // Wait for all threads

                    phase2();
                    barrier.await();  // Barrier is reusable!

                    phase3();
                } catch (Exception e) {
                    Thread.currentThread().interrupt();
                }
            }).start();
        }
    }
}
```

3.8.4 4. Semaphore (Limit concurrent access)

```
public class SemaphoreDemo {
    private final Semaphore semaphore = new Semaphore(3);  // 3 permits

    public void accessResource() {
        try {
            semaphore.acquire();  // Block until permit available
            try {
                useResource();  // Only 3 threads can be here
            } finally {
                semaphore.release();  // Return permit
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

3.8.5 5. CompletableFuture (Async programming)

```java
public class CompletableFutureDemo {

    public void asyncOperations() {
        // Run async task
        CompletableFuture<String> future = CompletableFuture.supplyAsync(() -> {
            return fetchData();
        });

        // Transform result
        CompletableFuture<Integer> transformed = future
            .thenApply(String::length)
            .thenApply(len -> len * 2);

        // Combine multiple futures
        CompletableFuture<String> combined = future
            .thenCombine(CompletableFuture.supplyAsync(() -> fetchMoreData()),
                (data1, data2) -> data1 + data2);

        // Handle errors
        CompletableFuture<String> withErrorHandling = future
            .exceptionally(ex -> "Default value")
            .thenApply(String::toUpperCase);

        // Wait for all
        CompletableFuture<Void> all = CompletableFuture.allOf(
            CompletableFuture.runAsync(() -> task1()),
            CompletableFuture.runAsync(() -> task2()),
            CompletableFuture.runAsync(() -> task3())
        );

        // Wait for any
        CompletableFuture<Object> any = CompletableFuture.anyOf(
            CompletableFuture.supplyAsync(() -> fastService()),
            CompletableFuture.supplyAsync(() -> slowService())
        );
    }
}
```

3.9 Interview Questions

3.9.1 Q1: What is the difference between wait() and sleep()?

wait()	sleep()
Object method	Thread method
Releases lock	Holds lock
Must be in synchronized	No requirement
Woken by notify/notifyAll	Wakes after time
Used for inter-thread communication	Used for pause

3.9.2 Q2: How to prevent deadlock?

```
// Deadlock conditions (all 4 must be present):
// 1. Mutual Exclusion
// 2. Hold and Wait
// 3. No Preemption
// 4. Circular Wait

// Prevention strategies:
// 1. Lock ordering - always acquire locks in same order
// 2. Lock timeout - tryLock with timeout
// 3. Deadlock detection - use tryLock, back off on failure
// 4. Single lock - use one lock instead of multiple
```

3.9.3 Q3: Explain thread-safety of HashMap vs ConcurrentHashMap

```
// HashMap - NOT thread-safe
// - Concurrent modification can corrupt internal structure
// - Can cause infinite loops during resize

// ConcurrentHashMap - Thread-safe
// - Lock-free reads (volatile)
// - Fine-grained locking for writes (per-bucket)
// - No null keys or values (prevents NPE ambiguity)
// - Weakly consistent iterators
```

3.9.4 Q4: What is the happens-before relationship?

Happens-before guarantees that memory writes by one statement are visible to another statement. Key rules: 1. Program order within thread 2. Monitor unlock → subsequent lock 3. Volatile write → subsequent read 4. Thread start → first statement in thread 5. Thread actions → join returns 6. Transitivity

3.9.5 Q5: Thread Local Usage

```
public class ThreadLocalDemo {
    // Each thread has its own copy
    private static ThreadLocal<SimpleDateFormat> dateFormat =
        ThreadLocal.withInitial(() -> new SimpleDateFormat("yyyy-MM-dd"));

    public String formatDate(Date date) {
        return dateFormat.get().format(date);  // Thread-safe!
    }

    // IMPORTANT: Clean up in thread pools!
    public void cleanup() {
        dateFormat.remove();  // Prevent memory leaks
    }
}
```

3.9.6 Q6: Fork/Join Framework

```java
public class ForkJoinDemo extends RecursiveTask<Long> {
    private final long[] numbers;
    private final int start, end;
    private static final int THRESHOLD = 10_000;

    public ForkJoinDemo(long[] numbers, int start, int end) {
        this.numbers = numbers;
        this.start = start;
        this.end = end;
    }

    @Override
    protected Long compute() {
        if (end - start <= THRESHOLD) {
            // Base case: compute directly
            long sum = 0;
            for (int i = start; i < end; i++) {
                sum += numbers[i];
            }
            return sum;
        } else {
            // Recursive case: split
            int mid = start + (end - start) / 2;
            ForkJoinDemo left = new ForkJoinDemo(numbers, start, mid);
            ForkJoinDemo right = new ForkJoinDemo(numbers, mid, end);

            left.fork();  // Submit to pool
            long rightResult = right.compute();  // Compute right in current thread
            long leftResult = left.join();  // Wait for left

            return leftResult + rightResult;
        }
    }

    public static void main(String[] args) {
        long[] numbers = new long[1_000_000];
        ForkJoinPool pool = ForkJoinPool.commonPool();
        long sum = pool.invoke(new ForkJoinDemo(numbers, 0, numbers.length));
    }
}
```

4 Data Structures and Algorithms in Java

4.1 Time/Space Complexity Analysis

4.1.1 Big-O Complexity Chart

```
Time Complexity Growth Rates:

O(1)       ──────────────────────────────────────────────  Constant
O(log n)   ──────────────────────                          Logarithmic
O(n)       ──────────────────────────────                  Linear
O(n log n) ────────────────────────────────────            Linearithmic
O(n²)      ──────────────────────────────────────────────  Quadratic
O(2^n)     ──────────────────────────────────────────────  Exponential
O(n!)      ──────────────────────────────────────────────  Factorial
```

Common Operations:

Data Structure	Access	Search	Insert/Del
Array	O(1)	O(n)	O(n)
Sorted Array	O(1)	O(log n)	O(n)
Linked List	O(n)	O(n)	O(1)*
Hash Table	N/A	O(1)*	O(1)*
BST (balanced)	O(log n)	O(log n)	O(log n)
Heap	O(1)†	O(n)	O(log n)

* Amortized † For min/max only

4.2 Arrays and Strings

4.2.1 Two Pointer Technique

```java
// Pattern: Two pointers moving towards each other
public boolean isPalindrome(String s) {
    int left = 0, right = s.length() - 1;
    while (left < right) {
        while (left < right && !Character.isLetterOrDigit(s.charAt(left))) left++;
        while (left < right && !Character.isLetterOrDigit(s.charAt(right))) right--;
        if (Character.toLowerCase(s.charAt(left)) !=
            Character.toLowerCase(s.charAt(right))) {
            return false;
        }
        left++;
        right--;
    }
    return true;
}

// Pattern: Two Sum with sorted array
public int[] twoSumSorted(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left < right) {
        int sum = nums[left] + nums[right];
        if (sum == target) return new int[]{left, right};
        else if (sum < target) left++;
        else right--;
    }
    return new int[]{-1, -1};
}
```

4.2.2 Sliding Window Pattern

```java
// Fixed-size window: Maximum sum of k consecutive elements
public int maxSumSubarray(int[] arr, int k) {
    int windowSum = 0, maxSum = Integer.MIN_VALUE;

    for (int i = 0; i < arr.length; i++) {
        windowSum += arr[i];

        if (i >= k - 1) {
            maxSum = Math.max(maxSum, windowSum);
            windowSum -= arr[i - k + 1];  // Remove leftmost element
        }
    }
    return maxSum;
}

// Variable-size window: Longest substring without repeating characters
public int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> lastSeen = new HashMap<>();
    int maxLen = 0, start = 0;

    for (int end = 0; end < s.length(); end++) {
        char c = s.charAt(end);
        if (lastSeen.containsKey(c) && lastSeen.get(c) >= start) {
            start = lastSeen.get(c) + 1;  // Shrink window
        }
        lastSeen.put(c, end);
        maxLen = Math.max(maxLen, end - start + 1);
    }
    return maxLen;
}

// Minimum window substring (Classic sliding window)
public String minWindow(String s, String t) {
    if (t.isEmpty()) return "";

    Map<Character, Integer> need = new HashMap<>();
    Map<Character, Integer> window = new HashMap<>();
    for (char c : t.toCharArray()) need.merge(c, 1, Integer::sum);

    int left = 0, right = 0;
    int valid = 0;
    int start = 0, minLen = Integer.MAX_VALUE;

    while (right < s.length()) {
        char c = s.charAt(right++);
        if (need.containsKey(c)) {
            window.merge(c, 1, Integer::sum);
            if (window.get(c).equals(need.get(c))) valid++;
        }

        while (valid == need.size()) {
            if (right - left < minLen) {
                start = left;
                minLen = right - left;
            }
            char d = s.charAt(left++);
            if (need.containsKey(d)) {
                if (window.get(d).equals(need.get(d))) valid--;
                window.merge(d, -1, Integer::sum);
            }
        }
    }
    return minLen == Integer.MAX_VALUE ? "" : s.substring(start, start + minLen);
}
```

4.2.3 Prefix Sum Pattern

```java
// Subarray sum equals K
public int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> prefixCount = new HashMap<>();
    prefixCount.put(0, 1);  // Empty prefix

    int sum = 0, count = 0;
    for (int num : nums) {
        sum += num;
        // If (sum - k) exists, there's a subarray with sum k
        count += prefixCount.getOrDefault(sum - k, 0);
        prefixCount.merge(sum, 1, Integer::sum);
    }
    return count;
}

// 2D Prefix Sum
public class NumMatrix {
    private int[][] prefix;

    public NumMatrix(int[][] matrix) {
        int m = matrix.length, n = matrix[0].length;
        prefix = new int[m + 1][n + 1];

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                prefix[i][j] = matrix[i-1][j-1]
                             + prefix[i-1][j]
                             + prefix[i][j-1]
                             - prefix[i-1][j-1];
            }
        }
    }

    public int sumRegion(int r1, int c1, int r2, int c2) {
        return prefix[r2+1][c2+1] - prefix[r1][c2+1]
             - prefix[r2+1][c1] + prefix[r1][c1];
    }
}
```

4.3 Linked Lists

4.3.1 Common Techniques

```java
// Fast and Slow Pointer (Floyd's Algorithm)
// Detect cycle, find middle, find nth from end

public class LinkedListTechniques {

    // Find middle of linked list
    public ListNode findMiddle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;  // Middle (or second middle if even length)
    }

    // Detect cycle
    public boolean hasCycle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }

    // Find cycle start (Floyd's algorithm part 2)
    public ListNode detectCycleStart(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) {
                // Reset one pointer to head
                slow = head;
                while (slow != fast) {
                    slow = slow.next;
                    fast = fast.next;
                }
                return slow;  // Cycle start
            }
        }
        return null;
    }

    // Reverse linked list (iterative)
    public ListNode reverse(ListNode head) {
        ListNode prev = null, curr = head;
        while (curr != null) {
            ListNode next = curr.next;
            curr.next = prev;
            prev = curr;
            curr = next;
        }
        return prev;
    }

    // Reverse in groups of k
    public ListNode reverseKGroup(ListNode head, int k) {
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode prevGroup = dummy;

        while (true) {
            ListNode kth = getKth(prevGroup, k);
            if (kth == null) break;
```

4.4 Trees and Graphs

4.4.1 Tree Traversals

```java
public class TreeTraversals {

    // Inorder (Left, Root, Right) - BST gives sorted order
    public List<Integer> inorder(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        inorderHelper(root, result);
        return result;
    }

    private void inorderHelper(TreeNode node, List<Integer> result) {
        if (node == null) return;
        inorderHelper(node.left, result);
        result.add(node.val);
        inorderHelper(node.right, result);
    }

    // Iterative Inorder with Stack
    public List<Integer> inorderIterative(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        Deque<TreeNode> stack = new ArrayDeque<>();
        TreeNode curr = root;

        while (curr != null || !stack.isEmpty()) {
            while (curr != null) {
                stack.push(curr);
                curr = curr.left;
            }
            curr = stack.pop();
            result.add(curr.val);
            curr = curr.right;
        }
        return result;
    }

    // Level Order (BFS)
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();
        if (root == null) return result;

        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root);

        while (!queue.isEmpty()) {
            int size = queue.size();
            List<Integer> level = new ArrayList<>();

            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                level.add(node.val);
                if (node.left != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
            }
            result.add(level);
        }
        return result;
    }

    // Morris Traversal (O(1) space inorder)
    public List<Integer> morrisInorder(TreeNode root) {
        List<Integer> result = new ArrayList<>();
        TreeNode curr = root;

        while (curr != null) {
            if (curr.left == null) {
                result.add(curr.val);
                curr = curr.right;
```

4.4.2 Binary Search Tree Operations

```java
public class BSTOperations {

    // Search O(log n) average, O(n) worst
    public TreeNode search(TreeNode root, int target) {
        if (root == null || root.val == target) return root;
        return target < root.val
            ? search(root.left, target)
            : search(root.right, target);
    }

    // Insert
    public TreeNode insert(TreeNode root, int val) {
        if (root == null) return new TreeNode(val);
        if (val < root.val) root.left = insert(root.left, val);
        else root.right = insert(root.right, val);
        return root;
    }

    // Delete
    public TreeNode delete(TreeNode root, int key) {
        if (root == null) return null;

        if (key < root.val) {
            root.left = delete(root.left, key);
        } else if (key > root.val) {
            root.right = delete(root.right, key);
        } else {
            // Node to delete found
            if (root.left == null) return root.right;
            if (root.right == null) return root.left;

            // Node has two children
            TreeNode successor = findMin(root.right);
            root.val = successor.val;
            root.right = delete(root.right, successor.val);
        }
        return root;
    }

    private TreeNode findMin(TreeNode node) {
        while (node.left != null) node = node.left;
        return node;
    }

    // Validate BST
    public boolean isValidBST(TreeNode root) {
        return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }

    private boolean validate(TreeNode node, long min, long max) {
        if (node == null) return true;
        if (node.val <= min || node.val >= max) return false;
        return validate(node.left, min, node.val) &&
               validate(node.right, node.val, max);
    }

    // Lowest Common Ancestor in BST
    public TreeNode lcaBST(TreeNode root, TreeNode p, TreeNode q) {
        if (p.val < root.val && q.val < root.val)
            return lcaBST(root.left, p, q);
        if (p.val > root.val && q.val > root.val)
            return lcaBST(root.right, p, q);
        return root;
    }

    // LCA in Binary Tree (not BST)
```

4.5 Graph Algorithms

4.5.1 Graph Representations

```java
public class GraphRepresentations {

    // Adjacency List (most common, space-efficient for sparse graphs)
    List<List<Integer>> adjList;

    public void buildAdjList(int n, int[][] edges) {
        adjList = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            adjList.add(new ArrayList<>());
        }
        for (int[] edge : edges) {
            adjList.get(edge[0]).add(edge[1]);
            adjList.get(edge[1]).add(edge[0]);  // For undirected
        }
    }

    // Adjacency Matrix (good for dense graphs, O(1) edge lookup)
    int[][] adjMatrix;

    public void buildAdjMatrix(int n, int[][] edges) {
        adjMatrix = new int[n][n];
        for (int[] edge : edges) {
            adjMatrix[edge[0]][edge[1]] = 1;
            adjMatrix[edge[1]][edge[0]] = 1;
        }
    }

    // Edge List (good for Kruskal's algorithm)
    List<int[]> edgeList;
}
```

4.5.2 BFS and DFS

```java
public class GraphTraversal {

    // BFS - Level-by-level, shortest path in unweighted graph
    public List<Integer> bfs(List<List<Integer>> graph, int start) {
        List<Integer> result = new ArrayList<>();
        boolean[] visited = new boolean[graph.size()];
        Queue<Integer> queue = new LinkedList<>();

        queue.offer(start);
        visited[start] = true;

        while (!queue.isEmpty()) {
            int node = queue.poll();
            result.add(node);

            for (int neighbor : graph.get(node)) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    queue.offer(neighbor);
                }
            }
        }
        return result;
    }

    // DFS - Go deep first
    public List<Integer> dfs(List<List<Integer>> graph, int start) {
        List<Integer> result = new ArrayList<>();
        boolean[] visited = new boolean[graph.size()];
        dfsHelper(graph, start, visited, result);
        return result;
    }

    private void dfsHelper(List<List<Integer>> graph, int node,
                           boolean[] visited, List<Integer> result) {
        visited[node] = true;
        result.add(node);

        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                dfsHelper(graph, neighbor, visited, result);
            }
        }
    }

    // Iterative DFS
    public List<Integer> dfsIterative(List<List<Integer>> graph, int start) {
        List<Integer> result = new ArrayList<>();
        boolean[] visited = new boolean[graph.size()];
        Deque<Integer> stack = new ArrayDeque<>();

        stack.push(start);

        while (!stack.isEmpty()) {
            int node = stack.pop();
            if (visited[node]) continue;

            visited[node] = true;
            result.add(node);

            // Add neighbors in reverse order for consistent ordering
            List<Integer> neighbors = graph.get(node);
            for (int i = neighbors.size() - 1; i >= 0; i--) {
                if (!visited[neighbors.get(i)]) {
                    stack.push(neighbors.get(i));
                }
```

4.5.3 Dijkstra's Algorithm

```java
public class Dijkstra {

    // Single-source shortest path (non-negative weights)
    public int[] dijkstra(int[][] graph, int source) {
        int n = graph.length;
        int[] dist = new int[n];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[source] = 0;

        // Min-heap: [distance, node]
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
        pq.offer(new int[]{0, source});

        while (!pq.isEmpty()) {
            int[] curr = pq.poll();
            int d = curr[0], u = curr[1];

            // Skip if we already found a shorter path
            if (d > dist[u]) continue;

            for (int v = 0; v < n; v++) {
                if (graph[u][v] > 0) {  // Edge exists
                    int newDist = dist[u] + graph[u][v];
                    if (newDist < dist[v]) {
                        dist[v] = newDist;
                        pq.offer(new int[]{newDist, v});
                    }
                }
            }
        }
        return dist;
    }
}
```

4.5.4 Topological Sort

```java
public class TopologicalSort {

    // Kahn's Algorithm (BFS-based)
    public int[] topologicalSort(int n, int[][] edges) {
        List<List<Integer>> graph = new ArrayList<>();
        int[] inDegree = new int[n];

        for (int i = 0; i < n; i++) graph.add(new ArrayList<>());

        for (int[] edge : edges) {
            graph.get(edge[0]).add(edge[1]);
            inDegree[edge[1]]++;
        }

        Queue<Integer> queue = new LinkedList<>();
        for (int i = 0; i < n; i++) {
            if (inDegree[i] == 0) queue.offer(i);
        }

        int[] result = new int[n];
        int index = 0;

        while (!queue.isEmpty()) {
            int node = queue.poll();
            result[index++] = node;

            for (int neighbor : graph.get(node)) {
                if (--inDegree[neighbor] == 0) {
                    queue.offer(neighbor);
                }
            }
        }

        return index == n ? result : new int[0];  // Empty if cycle exists
    }

    // DFS-based Topological Sort
    public List<Integer> topologicalSortDFS(int n, int[][] edges) {
        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < n; i++) graph.add(new ArrayList<>());

        for (int[] edge : edges) {
            graph.get(edge[0]).add(edge[1]);
        }

        int[] state = new int[n];  // 0: unvisited, 1: visiting, 2: visited
        List<Integer> result = new ArrayList<>();

        for (int i = 0; i < n; i++) {
            if (state[i] == 0) {
                if (!dfs(graph, i, state, result)) {
                    return Collections.emptyList();  // Cycle detected
                }
            }
        }

        Collections.reverse(result);
        return result;
    }

    private boolean dfs(List<List<Integer>> graph, int node,
                        int[] state, List<Integer> result) {
        state[node] = 1;  // Visiting

        for (int neighbor : graph.get(node)) {
            if (state[neighbor] == 1) return false;  // Cycle!
```

4.6 Dynamic Programming

4.6.1 DP Patterns

```java
public class DynamicProgramming {

    // Pattern 1: 0/1 Knapsack
    public int knapsack(int[] weights, int[] values, int capacity) {
        int n = weights.length;
        int[][] dp = new int[n + 1][capacity + 1];

        for (int i = 1; i <= n; i++) {
            for (int w = 1; w <= capacity; w++) {
                if (weights[i-1] <= w) {
                    dp[i][w] = Math.max(
                        dp[i-1][w],  // Don't take item
                        dp[i-1][w - weights[i-1]] + values[i-1]  // Take item
                    );
                } else {
                    dp[i][w] = dp[i-1][w];
                }
            }
        }
        return dp[n][capacity];
    }

    // Space-optimized 1D
    public int knapsackOptimized(int[] weights, int[] values, int capacity) {
        int[] dp = new int[capacity + 1];

        for (int i = 0; i < weights.length; i++) {
            // Traverse backwards to avoid using same item twice
            for (int w = capacity; w >= weights[i]; w--) {
                dp[w] = Math.max(dp[w], dp[w - weights[i]] + values[i]);
            }
        }
        return dp[capacity];
    }

    // Pattern 2: Unbounded Knapsack (items can be used multiple times)
    public int unboundedKnapsack(int[] weights, int[] values, int capacity) {
        int[] dp = new int[capacity + 1];

        for (int w = 1; w <= capacity; w++) {
            for (int i = 0; i < weights.length; i++) {
                if (weights[i] <= w) {
                    dp[w] = Math.max(dp[w], dp[w - weights[i]] + values[i]);
                }
            }
        }
        return dp[capacity];
    }

    // Pattern 3: Longest Common Subsequence
    public int longestCommonSubsequence(String text1, String text2) {
        int m = text1.length(), n = text2.length();
        int[][] dp = new int[m + 1][n + 1];

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (text1.charAt(i-1) == text2.charAt(j-1)) {
                    dp[i][j] = dp[i-1][j-1] + 1;
                } else {
                    dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
                }
            }
        }
        return dp[m][n];
    }
```

4.7 Sorting Algorithms

4.7.1 Comparison of Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)	Yes
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)	No
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No
Counting Sort	O(n + k)	O(n + k)	O(n + k)	O(k)	Yes
Radix Sort	O(nk)	O(nk)	O(nk)	O(n + k)	Yes

4.7.2 Quick Sort Implementation

```java
public class QuickSort {

    public void quickSort(int[] arr, int low, int high) {
        if (low < high) {
            int pi = partition(arr, low, high);
            quickSort(arr, low, pi - 1);
            quickSort(arr, pi + 1, high);
        }
    }

    private int partition(int[] arr, int low, int high) {
        // Random pivot to avoid worst case
        int randomIdx = low + (int)(Math.random() * (high - low + 1));
        swap(arr, randomIdx, high);

        int pivot = arr[high];
        int i = low - 1;

        for (int j = low; j < high; j++) {
            if (arr[j] < pivot) {
                i++;
                swap(arr, i, j);
            }
        }
        swap(arr, i + 1, high);
        return i + 1;
    }

    private void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}
```

4.7.3 Merge Sort Implementation

```java
public class MergeSort {

    public void mergeSort(int[] arr, int left, int right) {
        if (left < right) {
            int mid = left + (right - left) / 2;
            mergeSort(arr, left, mid);
            mergeSort(arr, mid + 1, right);
            merge(arr, left, mid, right);
        }
    }

    private void merge(int[] arr, int left, int mid, int right) {
        int[] temp = new int[right - left + 1];
        int i = left, j = mid + 1, k = 0;

        while (i <= mid && j <= right) {
            if (arr[i] <= arr[j]) {
                temp[k++] = arr[i++];
            } else {
                temp[k++] = arr[j++];
            }
        }

        while (i <= mid) temp[k++] = arr[i++];
        while (j <= right) temp[k++] = arr[j++];

        System.arraycopy(temp, 0, arr, left, temp.length);
    }
}
```

5 Object-Oriented Programming and Design Patterns

5.1 OOP Principles Deep Dive

5.1.1 1. Encapsulation - Information Hiding

```java
// BAD: Exposing internal state
public class BankAccountBad {
    public double balance;  // Direct access - dangerous!
}

// GOOD: Encapsulated with validation
public class BankAccount {
    private double balance;
    private final String accountNumber;
    private List<Transaction> transactions = new ArrayList<>();

    public BankAccount(String accountNumber, double initialDeposit) {
        if (initialDeposit < 0) {
            throw new IllegalArgumentException("Initial deposit cannot be negative");
        }
        this.accountNumber = accountNumber;
        this.balance = initialDeposit;
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Deposit amount must be positive");
        }
        balance += amount;
        transactions.add(new Transaction(TransactionType.DEPOSIT, amount));
    }

    public void withdraw(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Withdrawal amount must be positive");
        }
        if (amount > balance) {
            throw new InsufficientFundsException("Insufficient balance");
        }
        balance -= amount;
        transactions.add(new Transaction(TransactionType.WITHDRAWAL, amount));
    }

    public double getBalance() {
        return balance;
    }

    // Return defensive copy to prevent external modification
    public List<Transaction> getTransactions() {
        return Collections.unmodifiableList(transactions);
    }
}
```

5.1.2 2. Inheritance - Type Hierarchy

```java
// Template Method Pattern using inheritance
public abstract class DataProcessor {
    // Template method - defines algorithm skeleton
    public final void process() {
        readData();
        processData();
        writeData();
        if (needsCleanup()) {
            cleanup();
        }
    }

    protected abstract void readData();
    protected abstract void processData();
    protected abstract void writeData();

    // Hook method - optional override
    protected boolean needsCleanup() {
        return false;
    }

    protected void cleanup() {
        // Default implementation
    }
}

public class CSVProcessor extends DataProcessor {
    private List<String[]> data;

    @Override
    protected void readData() {
        // Read CSV file
    }

    @Override
    protected void processData() {
        // Process CSV data
    }

    @Override
    protected void writeData() {
        // Write processed data
    }

    @Override
    protected boolean needsCleanup() {
        return true;  // Override hook
    }
}
```

5.1.3 3. Polymorphism - Multiple Forms

```java
// Compile-time polymorphism (Method Overloading)
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public double add(double a, double b) {
        return a + b;
    }

    public int add(int... numbers) {
        return Arrays.stream(numbers).sum();
    }
}

// Runtime polymorphism (Method Overriding)
interface Shape {
    double area();
    double perimeter();
}

class Circle implements Shape {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public double area() {
        return Math.PI * radius * radius;
    }

    @Override
    public double perimeter() {
        return 2 * Math.PI * radius;
    }
}

class Rectangle implements Shape {
    private double width, height;

    public Rectangle(double width, double height) {
        this.width = width;
        this.height = height;
    }

    @Override
    public double area() {
        return width * height;
    }

    @Override
    public double perimeter() {
        return 2 * (width + height);
    }
}

// Polymorphic usage
public class ShapeCalculator {
    public double totalArea(List<Shape> shapes) {
        return shapes.stream()
                     .mapToDouble(Shape::area)
                     .sum();
    }
}
```

5.1.4 4. Abstraction - Essential Features

```java
// Interface: Pure abstraction
public interface PaymentGateway {
    PaymentResult processPayment(PaymentRequest request);
    RefundResult refund(String transactionId, double amount);
    PaymentStatus getStatus(String transactionId);
}

// Abstract class: Partial abstraction with shared code
public abstract class AbstractPaymentGateway implements PaymentGateway {
    protected final Logger logger = LoggerFactory.getLogger(getClass());
    protected final PaymentValidator validator;

    protected AbstractPaymentGateway(PaymentValidator validator) {
        this.validator = validator;
    }

    @Override
    public final PaymentResult processPayment(PaymentRequest request) {
        // Template method with shared validation logic
        validator.validate(request);
        logger.info("Processing payment: {}", request.getId());

        PaymentResult result = doProcessPayment(request);

        logger.info("Payment result: {}", result.getStatus());
        return result;
    }

    protected abstract PaymentResult doProcessPayment(PaymentRequest request);
}
```

5.2 SOLID Principles

5.2.1 1. Single Responsibility Principle (SRP)

```java
// BAD: One class doing too much
public class UserServiceBad {
    public void createUser(User user) { /* ... */ }
    public void sendEmail(String to, String subject) { /* ... */ }
    public String generateReport(List<User> users) { /* ... */ }
    public void validateEmail(String email) { /* ... */ }
}

// GOOD: Each class has one responsibility
public class UserService {
    private final UserRepository repository;
    private final UserValidator validator;
    private final EmailService emailService;

    public User createUser(CreateUserRequest request) {
        validator.validate(request);
        User user = repository.save(new User(request));
        emailService.sendWelcomeEmail(user);
        return user;
    }
}

public class EmailService {
    public void sendWelcomeEmail(User user) { /* ... */ }
    public void sendPasswordReset(User user, String token) { /* ... */ }
}

public class UserReportGenerator {
    public String generateReport(List<User> users) { /* ... */ }
}
```

5.2.2 2. Open/Closed Principle (OCP)

```java
// BAD: Adding new shapes requires modifying existing code
public class AreaCalculatorBad {
    public double calculateArea(Object shape) {
        if (shape instanceof Rectangle r) {
            return r.width * r.height;
        } else if (shape instanceof Circle c) {
            return Math.PI * c.radius * c.radius;
        }
        // Adding Triangle requires modifying this class!
        throw new IllegalArgumentException("Unknown shape");
    }
}

// GOOD: Open for extension, closed for modification
public interface Shape {
    double calculateArea();
}

public class Rectangle implements Shape {
    private final double width, height;

    @Override
    public double calculateArea() {
        return width * height;
    }
}

public class Circle implements Shape {
    private final double radius;

    @Override
    public double calculateArea() {
        return Math.PI * radius * radius;
    }
}

// New shapes can be added without modifying existing code
public class Triangle implements Shape {
    private final double base, height;

    @Override
    public double calculateArea() {
        return 0.5 * base * height;
    }
}

public class AreaCalculator {
    public double calculateTotalArea(List<Shape> shapes) {
        return shapes.stream()
            .mapToDouble(Shape::calculateArea)
            .sum();
    }
}
```

5.2.3 3. Liskov Substitution Principle (LSP)

```java
// BAD: Square breaks Rectangle's contract
public class RectangleBad {
    protected int width, height;

    public void setWidth(int width) { this.width = width; }
    public void setHeight(int height) { this.height = height; }
    public int getArea() { return width * height; }
}

public class SquareBad extends RectangleBad {
    @Override
    public void setWidth(int width) {
        this.width = width;
        this.height = width;  // Violates LSP!
    }

    @Override
    public void setHeight(int height) {
        this.width = height;
        this.height = height;  // Violates LSP!
    }
}

// Test that fails with Square:
void testRectangle(RectangleBad r) {
    r.setWidth(5);
    r.setHeight(4);
    assert r.getArea() == 20;  // Fails for Square!
}

// GOOD: Use composition or separate hierarchies
public interface Shape {
    int getArea();
}

public final class Rectangle implements Shape {
    private final int width, height;

    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    @Override
    public int getArea() { return width * height; }
}

public final class Square implements Shape {
    private final int side;

    public Square(int side) { this.side = side; }

    @Override
    public int getArea() { return side * side; }
}
```

5.2.4 4. Interface Segregation Principle (ISP)

```java
// BAD: Fat interface forces unnecessary implementations
public interface WorkerBad {
    void work();
    void eat();
    void sleep();
}

public class RobotBad implements WorkerBad {
    public void work() { /* working */ }
    public void eat() { /* Robots don't eat! */ }  // Forced to implement
    public void sleep() { /* Robots don't sleep! */ }  // Forced to implement
}

// GOOD: Segregated interfaces
public interface Workable {
    void work();
}

public interface Eatable {
    void eat();
}

public interface Sleepable {
    void sleep();
}

public class Human implements Workable, Eatable, Sleepable {
    public void work() { /* working */ }
    public void eat() { /* eating */ }
    public void sleep() { /* sleeping */ }
}

public class Robot implements Workable {
    public void work() { /* working */ }
    // No need to implement eat() or sleep()
}
```

5.2.5 5. Dependency Inversion Principle (DIP)

```java
// BAD: High-level module depends on low-level module
public class OrderServiceBad {
    private MySQLDatabase database = new MySQLDatabase();  // Concrete dependency
    private SmtpEmailSender emailSender = new SmtpEmailSender();

    public void createOrder(Order order) {
        database.save(order);  // Coupled to MySQL
        emailSender.send(order.getCustomerEmail(), "Order confirmed");
    }
}

// GOOD: Both depend on abstractions
public interface OrderRepository {
    void save(Order order);
    Optional<Order> findById(String id);
}

public interface NotificationService {
    void notifyOrderCreated(Order order);
}

public class OrderService {
    private final OrderRepository repository;
    private final NotificationService notifications;

    // Dependencies injected
    public OrderService(OrderRepository repository, NotificationService notifications) {
        this.repository = repository;
        this.notifications = notifications;
    }

    public void createOrder(Order order) {
        repository.save(order);
        notifications.notifyOrderCreated(order);
    }
}

// Implementations can be swapped easily
public class MySQLOrderRepository implements OrderRepository { /* ... */ }
public class MongoOrderRepository implements OrderRepository { /* ... */ }
public class EmailNotificationService implements NotificationService { /* ... */ }
public class SmsNotificationService implements NotificationService { /* ... */ }
```

5.3 Creational Patterns

5.3.1 Singleton Pattern

```java
// Thread-safe Singleton implementations

// 1. Eager initialization (simplest)
public class EagerSingleton {
    private static final EagerSingleton INSTANCE = new EagerSingleton();
    private EagerSingleton() {}
    public static EagerSingleton getInstance() { return INSTANCE; }
}

// 2. Double-checked locking (lazy, thread-safe)
public class LazyThreadSafeSingleton {
    private static volatile LazyThreadSafeSingleton instance;
    private LazyThreadSafeSingleton() {}

    public static LazyThreadSafeSingleton getInstance() {
        if (instance == null) {
            synchronized (LazyThreadSafeSingleton.class) {
                if (instance == null) {
                    instance = new LazyThreadSafeSingleton();
                }
            }
        }
        return instance;
    }
}

// 3. Bill Pugh Singleton (recommended)
public class BillPughSingleton {
    private BillPughSingleton() {}

    private static class SingletonHelper {
        private static final BillPughSingleton INSTANCE = new BillPughSingleton();
    }

    public static BillPughSingleton getInstance() {
        return SingletonHelper.INSTANCE;  // Loaded only when accessed
    }
}

// 4. Enum Singleton (best - prevents reflection attacks)
public enum EnumSingleton {
    INSTANCE;

    public void doSomething() { /* ... */ }
}
```

5.3.2 Factory Pattern

```java
// Simple Factory
public class PaymentProcessorFactory {
    public static PaymentProcessor create(PaymentMethod method) {
        return switch (method) {
            case CREDIT_CARD -> new CreditCardProcessor();
            case PAYPAL -> new PayPalProcessor();
            case BANK_TRANSFER -> new BankTransferProcessor();
        };
    }
}

// Abstract Factory
public interface UIFactory {
    Button createButton();
    TextField createTextField();
    Checkbox createCheckbox();
}

public class WindowsUIFactory implements UIFactory {
    public Button createButton() { return new WindowsButton(); }
    public TextField createTextField() { return new WindowsTextField(); }
    public Checkbox createCheckbox() { return new WindowsCheckbox(); }
}

public class MacUIFactory implements UIFactory {
    public Button createButton() { return new MacButton(); }
    public TextField createTextField() { return new MacTextField(); }
    public Checkbox createCheckbox() { return new MacCheckbox(); }
}

// Usage
public class Application {
    private UIFactory uiFactory;

    public Application(UIFactory factory) {
        this.uiFactory = factory;
    }

    public void createUI() {
        Button btn = uiFactory.createButton();
        TextField tf = uiFactory.createTextField();
        // All components are from same family
    }
}
```

5.3.3 Builder Pattern

```java
public class HttpRequest {
    private final String url;
    private final String method;
    private final Map<String, String> headers;
    private final String body;
    private final Duration timeout;

    private HttpRequest(Builder builder) {
        this.url = builder.url;
        this.method = builder.method;
        this.headers = Map.copyOf(builder.headers);
        this.body = builder.body;
        this.timeout = builder.timeout;
    }

    public static class Builder {
        private final String url;  // Required
        private String method = "GET";
        private Map<String, String> headers = new HashMap<>();
        private String body;
        private Duration timeout = Duration.ofSeconds(30);

        public Builder(String url) {
            this.url = Objects.requireNonNull(url);
        }

        public Builder method(String method) {
            this.method = method;
            return this;
        }

        public Builder header(String name, String value) {
            headers.put(name, value);
            return this;
        }

        public Builder body(String body) {
            this.body = body;
            return this;
        }

        public Builder timeout(Duration timeout) {
            this.timeout = timeout;
            return this;
        }

        public HttpRequest build() {
            // Validation
            if (body != null && "GET".equals(method)) {
                throw new IllegalStateException("GET requests cannot have body");
            }
            return new HttpRequest(this);
        }
    }
}

// Usage
HttpRequest request = new HttpRequest.Builder("https://api.example.com")
    .method("POST")
    .header("Content-Type", "application/json")
    .header("Authorization", "Bearer token")
    .body("{\"key\": \"value\"}")
    .timeout(Duration.ofSeconds(10))
    .build();
```

5.4 Structural Patterns

5.4.1 Adapter Pattern

```
// Target interface
public interface MediaPlayer {
    void play(String filename);
}

// Adaptee (incompatible interface)
public class VLCPlayer {
    public void playVLC(String filename) { /* VLC specific */ }
}

public class MP4Player {
    public void playMP4(String filename) { /* MP4 specific */ }
}

// Adapter
public class MediaAdapter implements MediaPlayer {
    private VLCPlayer vlcPlayer;
    private MP4Player mp4Player;

    @Override
    public void play(String filename) {
        if (filename.endsWith(".vlc")) {
            if (vlcPlayer == null) vlcPlayer = new VLCPlayer();
            vlcPlayer.playVLC(filename);
        } else if (filename.endsWith(".mp4")) {
            if (mp4Player == null) mp4Player = new MP4Player();
            mp4Player.playMP4(filename);
        }
    }
}
```

5.4.2 Decorator Pattern

```java
// Component interface
public interface Coffee {
    double getCost();
    String getDescription();
}

// Concrete component
public class SimpleCoffee implements Coffee {
    public double getCost() { return 2.0; }
    public String getDescription() { return "Coffee"; }
}

// Base decorator
public abstract class CoffeeDecorator implements Coffee {
    protected final Coffee decoratedCoffee;

    public CoffeeDecorator(Coffee coffee) {
        this.decoratedCoffee = coffee;
    }

    public double getCost() { return decoratedCoffee.getCost(); }
    public String getDescription() { return decoratedCoffee.getDescription(); }
}

// Concrete decorators
public class MilkDecorator extends CoffeeDecorator {
    public MilkDecorator(Coffee coffee) { super(coffee); }

    @Override
    public double getCost() { return super.getCost() + 0.5; }

    @Override
    public String getDescription() { return super.getDescription() + ", Milk"; }
}

public class SugarDecorator extends CoffeeDecorator {
    public SugarDecorator(Coffee coffee) { super(coffee); }

    @Override
    public double getCost() { return super.getCost() + 0.2; }

    @Override
    public String getDescription() { return super.getDescription() + ", Sugar"; }
}

// Usage
Coffee coffee = new SugarDecorator(new MilkDecorator(new SimpleCoffee()));
System.out.println(coffee.getDescription()); // Coffee, Milk, Sugar
System.out.println(coffee.getCost());        // 2.7
```

5.4.3 Proxy Pattern

```java
// Subject interface
public interface Image {
    void display();
}

// Real subject (expensive to create)
public class RealImage implements Image {
    private final String filename;

    public RealImage(String filename) {
        this.filename = filename;
        loadFromDisk();  // Expensive operation
    }

    private void loadFromDisk() {
        System.out.println("Loading " + filename);
    }

    @Override
    public void display() {
        System.out.println("Displaying " + filename);
    }
}

// Proxy (lazy loading)
public class ProxyImage implements Image {
    private final String filename;
    private RealImage realImage;

    public ProxyImage(String filename) {
        this.filename = filename;
    }

    @Override
    public void display() {
        if (realImage == null) {
            realImage = new RealImage(filename);  // Load only when needed
        }
        realImage.display();
    }
}
```

5.5 Behavioral Patterns

5.5.1 Strategy Pattern

```java
// Strategy interface
@FunctionalInterface
public interface CompressionStrategy {
    byte[] compress(byte[] data);
}

// Concrete strategies
public class ZipCompression implements CompressionStrategy {
    @Override
    public byte[] compress(byte[] data) {
        // ZIP compression logic
        return compressedData;
    }
}

public class GzipCompression implements CompressionStrategy {
    @Override
    public byte[] compress(byte[] data) {
        // GZIP compression logic
        return compressedData;
    }
}

// Context
public class FileCompressor {
    private CompressionStrategy strategy;

    public void setStrategy(CompressionStrategy strategy) {
        this.strategy = strategy;
    }

    public byte[] compressFile(byte[] fileData) {
        return strategy.compress(fileData);
    }
}

// Usage
FileCompressor compressor = new FileCompressor();
compressor.setStrategy(new ZipCompression());
byte[] compressed = compressor.compressFile(data);

// With lambda (functional interface)
compressor.setStrategy(data -> customCompress(data));
```

5.5.2 Observer Pattern

```java
// Subject
public class EventManager {
    private Map<String, List<EventListener>> listeners = new HashMap<>();

    public void subscribe(String eventType, EventListener listener) {
        listeners.computeIfAbsent(eventType, k -> new ArrayList<>()).add(listener);
    }

    public void unsubscribe(String eventType, EventListener listener) {
        listeners.getOrDefault(eventType, Collections.emptyList()).remove(listener);
    }

    public void notify(String eventType, Object data) {
        listeners.getOrDefault(eventType, Collections.emptyList())
            .forEach(listener -> listener.update(eventType, data));
    }
}

// Observer interface
@FunctionalInterface
public interface EventListener {
    void update(String eventType, Object data);
}

// Usage
EventManager events = new EventManager();
events.subscribe("order.created", (type, data) -> sendEmail((Order) data));
events.subscribe("order.created", (type, data) -> updateInventory((Order) data));
events.notify("order.created", newOrder);
```

5.5.3 Template Method Pattern

```java
public abstract class DataProcessor {

    // Template method (final to prevent override)
    public final void process() {
        readData();
        processData();
        writeData();
        if (shouldNotify()) {
            notifyComplete();
        }
    }

    // Abstract methods (must be implemented)
    protected abstract void readData();
    protected abstract void processData();
    protected abstract void writeData();

    // Hook methods (optional override)
    protected boolean shouldNotify() {
        return true;
    }

    protected void notifyComplete() {
        System.out.println("Processing complete");
    }
}

public class CSVProcessor extends DataProcessor {
    @Override
    protected void readData() {
        System.out.println("Reading CSV file");
    }

    @Override
    protected void processData() {
        System.out.println("Processing CSV data");
    }

    @Override
    protected void writeData() {
        System.out.println("Writing processed data");
    }
}
```

5.6 Interview Questions

5.6.1 Q1: Difference between Abstract Class and Interface?

Abstract Class	Interface
Can have state (fields)	Only constants (static final)
Can have constructor	No constructor
Single inheritance	Multiple inheritance
Can have any access modifier	Public by default
Use for IS-A + shared code	Use for CAN-DO capability

5.6.2 Q2: When to use Composition vs Inheritance?

```java
// Prefer COMPOSITION when:
// - "HAS-A" relationship
// - Need flexibility to change behavior at runtime
// - Avoiding fragile base class problem

public class Car {
    private Engine engine;  // HAS-A
    private Transmission transmission;

    public void drive() {
        engine.start();
        transmission.shift(1);
    }
}

// Use INHERITANCE when:
// - True "IS-A" relationship
// - Need to override behavior
// - Liskov Substitution holds

public class ElectricCar extends Car {
    @Override
    public void start() {
        // Electric specific startup
    }
}
```

5.6.3 Q3: Explain Method Overloading vs Overriding

```java
// OVERLOADING: Same method name, different parameters (compile-time)
public class Calculator {
    public int add(int a, int b) { return a + b; }
    public double add(double a, double b) { return a + b; }
    public int add(int a, int b, int c) { return a + b + c; }
}

// OVERRIDING: Same signature, different implementation (runtime)
public class Animal {
    public void speak() { System.out.println("Some sound"); }
}

public class Dog extends Animal {
    @Override
    public void speak() { System.out.println("Bark"); }
}
```

6 Generics, Reflection, and Annotations - Deep Dive

6.1 Generics Fundamentals

6.1.1 Why Generics?

```java
// Before Generics (Java 1.4 and earlier)
List list = new ArrayList();
list.add("Hello");
list.add(123);  // No compile error, but dangerous!
String s = (String) list.get(1);  // ClassCastException at runtime!

// With Generics (Java 5+)
List<String> list = new ArrayList<>();
list.add("Hello");
// list.add(123);  // Compile error! Type safety enforced
String s = list.get(0);  // No cast needed
```

6.1.2 Generic Class Implementation

```java
// Generic class with multiple type parameters
public class Pair<K, V> {
    private final K key;
    private final V value;

    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey() { return key; }
    public V getValue() { return value; }

    // Generic static factory method (type inference)
    public static <K, V> Pair<K, V> of(K key, V value) {
        return new Pair<>(key, value);
    }

    // Swap key and value
    public Pair<V, K> swap() {
        return new Pair<>(value, key);
    }

    @Override
    public String toString() {
        return "(" + key + ", " + value + ")";
    }
}

// Usage
Pair<String, Integer> pair = Pair.of("age", 25);
Pair<Integer, String> swapped = pair.swap();
```

6.1.3 Generic Methods

```java
public class GenericMethods {
    // Generic method - type parameter declared before return type
    public static <T> T getFirst(List<T> list) {
        if (list == null || list.isEmpty()) {
            return null;
        }
        return list.get(0);
    }

    // Multiple type parameters
    public static <T, U> Pair<T, U> makePair(T first, U second) {
        return new Pair<>(first, second);
    }

    // Type parameter with bound
    public static <T extends Comparable<T>> T max(T a, T b) {
        return a.compareTo(b) > 0 ? a : b;
    }

    // Generic method in generic class
    public static <T> void copy(List<? super T> dest, List<? extends T> src) {
        for (T item : src) {
            dest.add(item);
        }
    }
}
```

6.2 Type Erasure

6.2.1 How Type Erasure Works

At compile time, generics are converted to raw types:

```java
// What you write:
public class Box<T> {
    private T value;
    public T getValue() { return value; }
    public void setValue(T value) { this.value = value; }
}

// After type erasure (what JVM sees):
public class Box {
    private Object value;
    public Object getValue() { return value; }
    public void setValue(Object value) { this.value = value; }
}

// With bounded type:
public class Box<T extends Number> {
    private T value;
}

// After erasure (bounded type preserved):
public class Box {
    private Number value;  // Erased to bound
}
```

6.2.2 Type Erasure Implications

```java
// 1. Cannot use instanceof with parameterized types
// if (list instanceof ArrayList<String>) {}  // Compile error!
if (list instanceof ArrayList<?>) {}  // OK, unbounded wildcard

// 2. Cannot create arrays of parameterized types
// List<String>[] array = new ArrayList<String>[10];  // Compile error!
List<?>[] array = new ArrayList<?>[10];  // OK with wildcard
@SuppressWarnings("unchecked")
List<String>[] array = (List<String>[]) new ArrayList<?>[10];  // Workaround

// 3. Cannot create instances of type parameters
public class Factory<T> {
    // public T create() { return new T(); }  // Compile error!

    // Workaround using Class token
    private Class<T> type;

    public Factory(Class<T> type) {
        this.type = type;
    }

    public T create() throws Exception {
        return type.getDeclaredConstructor().newInstance();
    }
}

// 4. Static members cannot use class type parameters
public class Problem<T> {
    // private static T instance;  // Compile error!
    // public static T getInstance() {}  // Compile error!

    // Static methods can declare their own type parameters
    public static <U> U staticMethod(U param) { return param; }  // OK
}
```

6.3 Bounded Type Parameters

6.3.1 Upper Bounds

```java
// Single upper bound
public static <T extends Number> double sum(List<T> numbers) {
    double sum = 0;
    for (T n : numbers) {
        sum += n.doubleValue();  // Can call Number methods
    }
    return sum;
}

// Multiple bounds (class first, then interfaces)
public class SortedBox<T extends Comparable<T> & Serializable> {
    private T value;

    public int compareTo(SortedBox<T> other) {
        return this.value.compareTo(other.value);
    }
}

// Recursive type bound (F-bounded polymorphism)
public abstract class Builder<T extends Builder<T>> {
    protected abstract T self();

    private String name;
    private int id;

    public T withName(String name) {
        this.name = name;
        return self();
    }

    public T withId(int id) {
        this.id = id;
        return self();
    }
}

public class UserBuilder extends Builder<UserBuilder> {
    private String email;

    @Override
    protected UserBuilder self() {
        return this;
    }

    public UserBuilder withEmail(String email) {
        this.email = email;
        return this;
    }

    public User build() {
        return new User(/* ... */);
    }
}

// Usage: Fluent API without casting
User user = new UserBuilder()
    .withName("John")  // Returns UserBuilder, not Builder
    .withId(123)
    .withEmail("john@example.com")
    .build();
```

6.4 Type Erasure Deep Dive

6.4.1 How Type Erasure Works

```
// Source code
public class Box<T> {
    private T value;
    public void set(T value) { this.value = value; }
    public T get() { return value; }
}

// After type erasure (what JVM sees)
public class Box {
    private Object value;
    public void set(Object value) { this.value = value; }
    public Object get() { return value; }
}

// Bounded type parameter
public class NumberBox<T extends Number> {
    private T value;
}

// After erasure (erased to bound)
public class NumberBox {
    private Number value;  // Erased to Number, not Object
}
```

6.4.2 Bridge Methods

```
// Generic parent
public class Parent<T> {
    public void process(T value) { }
}

// Concrete child
public class Child extends Parent<String> {
    @Override
    public void process(String value) { }
}

// Compiler generates bridge method
public class Child extends Parent {
    // User's method
    public void process(String value) { }

    // Bridge method (synthetic)
    public void process(Object value) {
        process((String) value);  // Delegates to typed version
    }
}
```

6.4.3 Type Erasure Gotchas

```java
// 1. Cannot create generic arrays
T[] array = new T[10];  // Compile error!
// Workaround:
@SuppressWarnings("unchecked")
T[] array = (T[]) new Object[10];

// 2. Cannot use instanceof with generics
if (obj instanceof List<String>) { }  // Compile error!
// Can only check raw type:
if (obj instanceof List<?>) { }

// 3. Cannot create instances of type parameters
T instance = new T();  // Compile error!
// Workaround: Class<T> factory
public <T> T create(Class<T> clazz) throws Exception {
    return clazz.getDeclaredConstructor().newInstance();
}

// 4. Static members cannot use type parameter
public class Box<T> {
    private static T value;  // Compile error!
    private static List<T> list;  // Compile error!
}
```

6.5 Wildcards

6.5.1 PECS: Producer Extends, Consumer Super

```java
// ? extends T (upper bound) - Producer
// Read from the collection, don't write
public void printNumbers(List<? extends Number> list) {
    for (Number n : list) {
        System.out.println(n);  // Can read as Number
    }
    // list.add(1);  // Compile error! Don't know actual type
}

// ? super T (lower bound) - Consumer
// Write to the collection, limited reading
public void addIntegers(List<? super Integer> list) {
    list.add(1);    // Can add Integer
    list.add(2);
    // Integer n = list.get(0);  // Compile error! Only Object guaranteed
    Object o = list.get(0);  // OK
}

// Example usage
List<Integer> integers = new ArrayList<>();
List<Number> numbers = new ArrayList<>();
List<Object> objects = new ArrayList<>();

// Producer (extends)
printNumbers(integers);  // OK
printNumbers(numbers);   // OK

// Consumer (super)
addIntegers(integers);  // OK
addIntegers(numbers);   // OK
addIntegers(objects);   // OK
```

6.5.2 Wildcard Capture

```
// Wildcard capture helper pattern
public class WildcardHelper {

    // This won't compile
    public static void swap(List<?> list, int i, int j) {
        // list.set(i, list.get(j));  // Compile error!
    }

    // Solution: Capture helper
    public static void swap(List<?> list, int i, int j) {
        swapHelper(list, i, j);
    }

    private static <T> void swapHelper(List<T> list, int i, int j) {
        T temp = list.get(i);
        list.set(i, list.get(j));
        list.set(j, temp);
    }
}
```

6.6 Reflection API

6.6.1 Class Introspection

```
public class ReflectionDemo {

    public void inspectClass(Class<?> clazz) {
        // Class metadata
        System.out.println("Name: " + clazz.getName());
        System.out.println("Simple name: " + clazz.getSimpleName());
        System.out.println("Package: " + clazz.getPackage());
        System.out.println("Superclass: " + clazz.getSuperclass());
        System.out.println("Interfaces: " + Arrays.toString(clazz.getInterfaces()));
        System.out.println("Modifiers: " + Modifier.toString(clazz.getModifiers()));

        // Fields
        for (Field field : clazz.getDeclaredFields()) {
            System.out.println("Field: " + field.getName() + " : " + field.getType());
        }

        // Methods
        for (Method method : clazz.getDeclaredMethods()) {
            System.out.println("Method: " + method.getName());
        }

        // Constructors
        for (Constructor<?> constructor : clazz.getDeclaredConstructors()) {
            System.out.println("Constructor: " + constructor);
        }
    }
}
```

6.6.2 Dynamic Invocation

```
public class DynamicInvocation {

    public Object invokeMethod(Object target, String methodName, Object... args)
            throws Exception {
        // Find matching method
        Class<?>[] paramTypes = Arrays.stream(args)
            .map(Object::getClass)
            .toArray(Class<?>[]::new);

        Method method = target.getClass().getMethod(methodName, paramTypes);
        method.setAccessible(true);  // Bypass access checks

        return method.invoke(target, args);
    }

    public <T> T createInstance(Class<T> clazz, Object... args) throws Exception {
        Class<?>[] paramTypes = Arrays.stream(args)
            .map(Object::getClass)
            .toArray(Class<?>[]::new);

        Constructor<T> constructor = clazz.getDeclaredConstructor(paramTypes);
        constructor.setAccessible(true);

        return constructor.newInstance(args);
    }

    public void setField(Object target, String fieldName, Object value) throws Exception {
        Field field = target.getClass().getDeclaredField(fieldName);
        field.setAccessible(true);
        field.set(target, value);
    }
}
```

6.6.3 Reflection Performance

```
// Reflection is slower - cache Method/Field objects
public class ReflectionOptimization {
    private static final Map<String, Method> methodCache = new ConcurrentHashMap<>();

    public Object invokeOptimized(Object target, String methodName) throws Exception {
        String key = target.getClass().getName() + "#" + methodName;

        Method method = methodCache.computeIfAbsent(key, k -> {
            try {
                Method m = target.getClass().getMethod(methodName);
                m.setAccessible(true);
                return m;
            } catch (NoSuchMethodException e) {
                throw new RuntimeException(e);
            }
        });

        return method.invoke(target);
    }
}
```

6.7 Annotations

6.7.1 Creating Custom Annotations

```java
// Marker annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface Entity {
}

// Annotation with elements
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Column {
    String name() default "";
    boolean nullable() default true;
    int length() default 255;
}

// Repeatable annotation
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@Repeatable(Schedules.class)
public @interface Schedule {
    String cron();
}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Schedules {
    Schedule[] value();
}

// Usage
@Entity
public class User {
    @Column(name = "user_name", nullable = false, length = 100)
    private String name;

    @Schedule(cron = "0 0 * * *")
    @Schedule(cron = "0 12 * * *")
    public void backup() { }
}
```

6.7.2 Processing Annotations at Runtime

```java
public class AnnotationProcessor {

    public void processEntity(Object entity) {
        Class<?> clazz = entity.getClass();

        if (!clazz.isAnnotationPresent(Entity.class)) {
            throw new IllegalArgumentException("Not an entity");
        }

        for (Field field : clazz.getDeclaredFields()) {
            if (field.isAnnotationPresent(Column.class)) {
                Column column = field.getAnnotation(Column.class);
                String columnName = column.name().isEmpty() ? field.getName() : column.name();
                System.out.println("Column: " + columnName +
                    ", nullable=" + column.nullable() +
                    ", length=" + column.length());
            }
        }
    }
}
```

6.8 Building a Mini Framework

6.8.1 Simple Dependency Injection Container

```java
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Inject { }

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface Component { }

public class DIContainer {
    private Map<Class<?>, Object> instances = new HashMap<>();

    public void register(Class<?> clazz) throws Exception {
        if (!clazz.isAnnotationPresent(Component.class)) {
            throw new IllegalArgumentException("Class must be annotated with @Component");
        }

        Object instance = clazz.getDeclaredConstructor().newInstance();
        instances.put(clazz, instance);
    }

    public void autowire() throws Exception {
        for (Object instance : instances.values()) {
            for (Field field : instance.getClass().getDeclaredFields()) {
                if (field.isAnnotationPresent(Inject.class)) {
                    Object dependency = instances.get(field.getType());
                    if (dependency != null) {
                        field.setAccessible(true);
                        field.set(instance, dependency);
                    }
                }
            }
        }
    }

    @SuppressWarnings("unchecked")
    public <T> T get(Class<T> clazz) {
        return (T) instances.get(clazz);
    }
}

// Usage
@Component
public class UserRepository {
    public User findById(int id) { return new User(); }
}

@Component
public class UserService {
    @Inject
    private UserRepository repository;

    public User getUser(int id) {
        return repository.findById(id);
    }
}

// Bootstrap
DIContainer container = new DIContainer();
container.register(UserRepository.class);
container.register(UserService.class);
container.autowire();

UserService service = container.get(UserService.class);
```

6.9 Interview Questions

6.9.1 Q1: Why can't you create generic arrays?

```
// Arrays are covariant, generics are invariant
// This is legal:
Object[] objects = new String[10];
objects[0] = 123;  // ArrayStoreException at runtime

// If generic arrays were allowed:
List<String>[] lists = new List<String>[10];  // Hypothetically
Object[] objects = lists;  // Covariance
objects[0] = new ArrayList<Integer>();  // No exception!
String s = lists[0].get(0);  // ClassCastException - type safety broken!
```

6.9.2 Q2: What is reification?

Reification means type information is available at runtime. - Arrays ARE reified: `String[]` knows it holds Strings - Generics are NOT reified: `List<String>` becomes just `List` at runtime

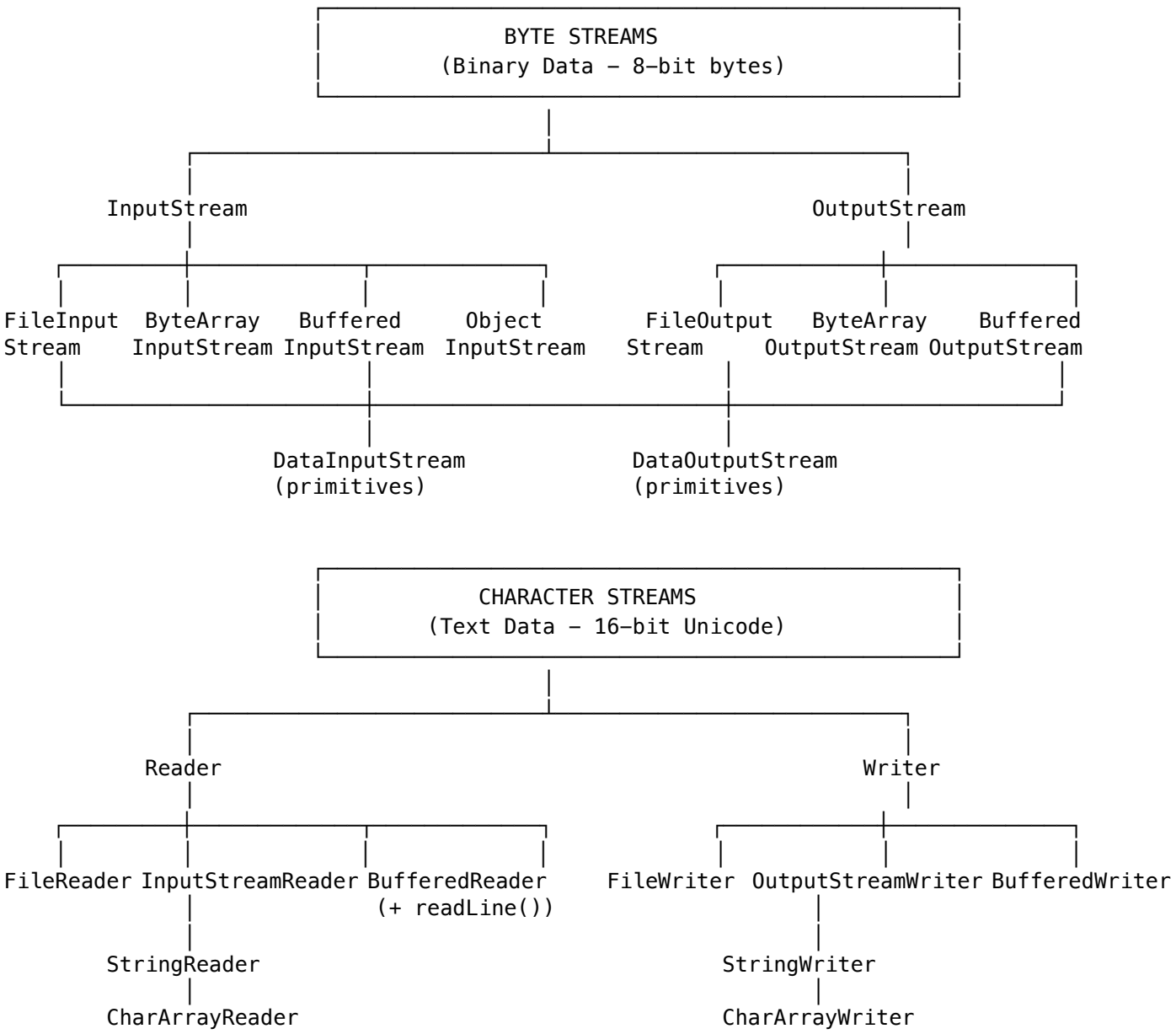
6.9.3 Q3: Difference between getClass() and instanceof?

```
// instanceof checks for IS-A relationship (includes subclasses)
if (obj instanceof Number) { }  // True for Integer, Double, etc.

// getClass() checks exact type
if (obj.getClass() == Integer.class) { }  // Only true for Integer

// With generics
if (obj instanceof List<?>) { }  // OK
// if (obj instanceof List<String>) { }  // Compile error!
```

7 Java I/O, NIO, and Networking - Deep Dive

7.1 I/O Stream Fundamentals

7.1.1 Stream Hierarchy



7.1.2 Decorator Pattern in I/O

```java
// Classic I/O uses Decorator pattern for flexibility
public class IODecoratorDemo {

    // Reading with multiple decorators
    public static void readFile(String path) throws IOException {
        // Layer 1: FileInputStream - reads raw bytes from file
        // Layer 2: BufferedInputStream - adds buffering for performance
        // Layer 3: DataInputStream - adds methods to read primitives

        try (DataInputStream dis = new DataInputStream(
                new BufferedInputStream(
                    new FileInputStream(path)))) {

            int intValue = dis.readInt();
            double doubleValue = dis.readDouble();
            String utfString = dis.readUTF();
        }
    }

    // Writing with decorators
    public static void writeFile(String path) throws IOException {
        try (DataOutputStream dos = new DataOutputStream(
                new BufferedOutputStream(
                    new FileOutputStream(path)))) {

            dos.writeInt(42);
            dos.writeDouble(3.14159);
            dos.writeUTF("Hello, World!");
        }
    }

    // Text file with character streams
    public static List<String> readTextFile(String path) throws IOException {
        List<String> lines = new ArrayList<>();

        try (BufferedReader reader = new BufferedReader(
                new InputStreamReader(
                    new FileInputStream(path), StandardCharsets.UTF_8))) {

            String line;
            while ((line = reader.readLine()) != null) {
                lines.add(line);
            }
        }
        return lines;
    }
}
```

7.2 Classic I/O vs NIO

7.2.1 Comparison Table

Feature	Classic I/O	NIO
Data Unit	Streams (bytes/chars)	Buffers + Channels
I/O Mode	Blocking	Non-blocking + Blocking
Scalability	Thread per connection	One thread, many channels
Data Direction	One-directional	Bi-directional (channels)
Memory	JVM heap only	Direct buffers (off-heap)
File Operations	Basic	Memory-mapped files
Complexity	Simple	More complex
Best For	Simple I/O, text	High-concurrency servers

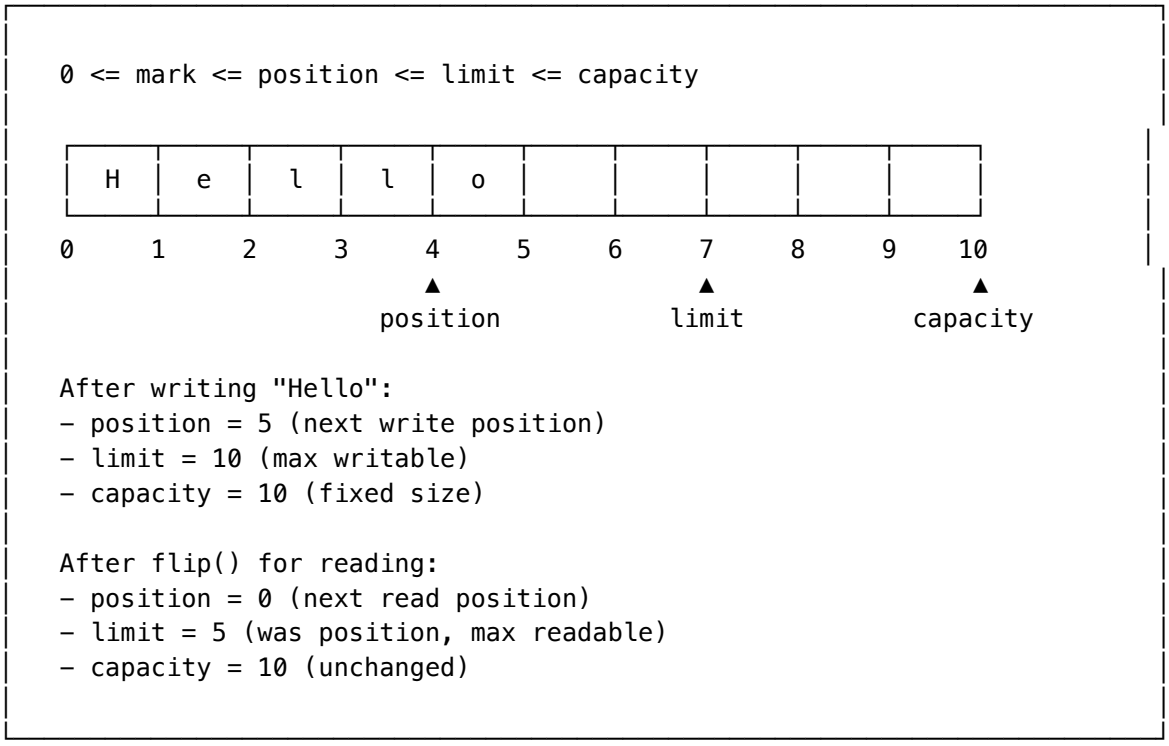
7.2.2 When to Use What

```
// Use Classic I/O when:
// - Simple file reading/writing
// - Text processing with BufferedReader
// - Object serialization
// - Low concurrency requirements

// Classic I/O example - simple and readable
public static String readFileClassic(String path) throws IOException {
    StringBuilder sb = new StringBuilder();
    try (BufferedReader reader = new BufferedReader(new FileReader(path))) {
        String line;
        while ((line = reader.readLine()) != null) {
            sb.append(line).append("\n");
        }
    }
    return sb.toString();
}

// Use NIO when:
// - High-performance file operations
// - Non-blocking network I/O
// - Memory-mapped file access
// - Scatter/Gather I/O
// - Multiplexed I/O (one thread, multiple connections)

// NIO example - more control
public static String readFileNIO(Path path) throws IOException {
    ByteBuffer buffer = ByteBuffer.allocate(1024);
    StringBuilder sb = new StringBuilder();

    try (FileChannel channel = FileChannel.open(path, StandardOpenOption.READ)) {
        while (channel.read(buffer) > 0) {
            buffer.flip();  // Switch to read mode
            sb.append(StandardCharsets.UTF_8.decode(buffer));
            buffer.clear(); // Switch to write mode
        }
    }
    return sb.toString();
}

// NIO.2 (Java 7+) - Best of both worlds
public static String readFileNIO2(Path path) throws IOException {
    return Files.readString(path);  // Simple AND efficient
}
```

7.3 NIO Channels and Buffers

7.3.1 Buffer Internals

Buffer State Variables:



7.3.2 Buffer Operations

```
public class BufferOperations {

    public void demonstrateBuffer() {
        // Allocate buffer
        ByteBuffer buffer = ByteBuffer.allocate(1024);

        // Writing to buffer
        buffer.put("Hello".getBytes());
        System.out.println("After write - position: " + buffer.position()); // 5

        // Prepare for reading
        buffer.flip();
        System.out.println("After flip - position: " + buffer.position()); // 0
        System.out.println("After flip - limit: " + buffer.limit());       // 5

        // Reading from buffer
        byte[] data = new byte[buffer.remaining()];
        buffer.get(data);

        // Prepare for more writing
        buffer.clear();  // position = 0, limit = capacity
        // OR
        buffer.compact(); // Copies unread data to beginning

        // Direct buffer (off-heap, faster for I/O)
        ByteBuffer directBuffer = ByteBuffer.allocateDirect(1024);
    }
}
```

7.3.3 Channel Operations

```
public class ChannelOperations {

    // Copy file using channels (faster than streams for large files)
    public void copyFile(Path source, Path dest) throws IOException {
        try (FileChannel sourceChannel = FileChannel.open(source, StandardOpenOption.READ);
             FileChannel destChannel = FileChannel.open(dest,
                 StandardOpenOption.CREATE, StandardOpenOption.WRITE)) {

            // transferTo/transferFrom use OS-level optimizations
            sourceChannel.transferTo(0, sourceChannel.size(), destChannel);
        }
    }

    // Memory-mapped file (extremely fast for large files)
    public void memoryMappedFile(Path path) throws IOException {
        try (FileChannel channel = FileChannel.open(path,
                StandardOpenOption.READ, StandardOpenOption.WRITE)) {

            MappedByteBuffer buffer = channel.map(
                FileChannel.MapMode.READ_WRITE, 0, channel.size());

            // Direct memory access
            byte firstByte = buffer.get(0);
            buffer.put(0, (byte) 'X');  // Writes directly to file!
        }
    }
}
```

7.4 NIO.2 File API (Java 7+)

7.4.1 Path and Files

```
public class NIO2Demo {

    public void pathOperations() {
        // Creating paths
        Path path = Path.of("/home/user/file.txt");
        Path path2 = Paths.get("/home", "user", "file.txt");

        // Path components
        System.out.println("Filename: " + path.getFileName());
        System.out.println("Parent: " + path.getParent());
        System.out.println("Root: " + path.getRoot());
        System.out.println("Absolute: " + path.toAbsolutePath());

        // Resolving paths
        Path base = Path.of("/home/user");
        Path resolved = base.resolve("documents/file.txt"); // /home/user/documents/file.txt
        Path sibling = path.resolveSibling("other.txt");    // /home/user/other.txt

        // Relativizing
        Path relative = Path.of("/home").relativize(Path.of("/home/user/file.txt")); // user/f
    }

    public void fileOperations() throws IOException {
        Path path = Path.of("test.txt");

        // Read/write entire file
        String content = Files.readString(path);
        Files.writeString(path, "Hello, World!");

        // Read all lines
        List<String> lines = Files.readAllLines(path);

        // Stream lines (lazy, good for large files)
        try (Stream<String> stream = Files.lines(path)) {
            stream.filter(line -> line.contains("error"))
                  .forEach(System.out::println);
        }

        // File attributes
        BasicFileAttributes attrs = Files.readAttributes(path, BasicFileAttributes.class);
        System.out.println("Size: " + attrs.size());
        System.out.println("Created: " + attrs.creationTime());
        System.out.println("Modified: " + attrs.lastModifiedTime());

        // File operations
        Files.copy(path, Path.of("backup.txt"), StandardCopyOption.REPLACE_EXISTING);
        Files.move(path, Path.of("renamed.txt"), StandardCopyOption.ATOMIC_MOVE);
        Files.delete(path);
    }
}
```

7.4.2 Walking Directory Trees

```java
public class DirectoryWalking {

    // Find all Java files
    public List<Path> findJavaFiles(Path root) throws IOException {
        try (Stream<Path> stream = Files.walk(root)) {
            return stream
                .filter(p -> p.toString().endsWith(".java"))
                .collect(Collectors.toList());
        }
    }

    // Using FileVisitor for more control
    public void walkWithVisitor(Path root) throws IOException {
        Files.walkFileTree(root, new SimpleFileVisitor<Path>() {
            @Override
            public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) {
                System.out.println("File: " + file);
                return FileVisitResult.CONTINUE;
            }

            @Override
            public FileVisitResult preVisitDirectory(Path dir, BasicFileAttributes attrs) {
                System.out.println("Directory: " + dir);
                return FileVisitResult.CONTINUE;
            }

            @Override
            public FileVisitResult visitFileFailed(Path file, IOException exc) {
                System.err.println("Failed: " + file);
                return FileVisitResult.CONTINUE;
            }
        });
    }

    // Watch for file changes
    public void watchDirectory(Path dir) throws IOException, InterruptedException {
        WatchService watcher = FileSystems.getDefault().newWatchService();
        dir.register(watcher,
            StandardWatchEventKinds.ENTRY_CREATE,
            StandardWatchEventKinds.ENTRY_DELETE,
            StandardWatchEventKinds.ENTRY_MODIFY);

        while (true) {
            WatchKey key = watcher.take();  // Blocks until event

            for (WatchEvent<?> event : key.pollEvents()) {
                Path changed = (Path) event.context();
                System.out.println(event.kind() + ": " + changed);
            }

            if (!key.reset()) break;
        }
    }
}
```

7.5 Networking

7.5.1 Socket Programming

```java
// Simple TCP Server
public class TcpServer {
    public void start(int port) throws IOException {
        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("Server listening on port " + port);

            while (true) {
                Socket client = serverSocket.accept();
                // Handle in new thread
                new Thread(() -> handleClient(client)).start();
            }
        }
    }

    private void handleClient(Socket client) {
        try (BufferedReader in = new BufferedReader(
                new InputStreamReader(client.getInputStream()));
             PrintWriter out = new PrintWriter(
                client.getOutputStream(), true)) {

            String message;
            while ((message = in.readLine()) != null) {
                System.out.println("Received: " + message);
                out.println("Echo: " + message);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

// Simple TCP Client
public class TcpClient {
    public void connect(String host, int port) throws IOException {
        try (Socket socket = new Socket(host, port);
             BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
             PrintWriter out = new PrintWriter(
                socket.getOutputStream(), true);
             BufferedReader console = new BufferedReader(
                new InputStreamReader(System.in))) {

            String userInput;
            while ((userInput = console.readLine()) != null) {
                out.println(userInput);
                System.out.println("Server: " + in.readLine());
            }
        }
    }
}
```

7.5.2 Non-Blocking I/O with Selectors

```java
public class NioServer {

    public void start(int port) throws IOException {
        Selector selector = Selector.open();
        ServerSocketChannel serverChannel = ServerSocketChannel.open();
        serverChannel.bind(new InetSocketAddress(port));
        serverChannel.configureBlocking(false);
        serverChannel.register(selector, SelectionKey.OP_ACCEPT);

        ByteBuffer buffer = ByteBuffer.allocate(1024);

        while (true) {
            selector.select();  // Blocks until events

            Iterator<SelectionKey> keys = selector.selectedKeys().iterator();
            while (keys.hasNext()) {
                SelectionKey key = keys.next();
                keys.remove();

                if (key.isAcceptable()) {
                    // New connection
                    SocketChannel client = serverChannel.accept();
                    client.configureBlocking(false);
                    client.register(selector, SelectionKey.OP_READ);

                } else if (key.isReadable()) {
                    // Data available to read
                    SocketChannel client = (SocketChannel) key.channel();
                    buffer.clear();
                    int bytesRead = client.read(buffer);

                    if (bytesRead == -1) {
                        client.close();
                    } else {
                        buffer.flip();
                        client.write(buffer);  // Echo back
                    }
                }
            }
        }
    }
}
```

7.5.3 Async I/O (NIO.2)

```java
public class AsyncIODemo {

    public void asyncFileRead(Path path) throws IOException {
        AsynchronousFileChannel channel = AsynchronousFileChannel.open(path, StandardOpenOptio
        ByteBuffer buffer = ByteBuffer.allocate(1024);

        // Callback-based
        channel.read(buffer, 0, buffer, new CompletionHandler<Integer, ByteBuffer>() {
            @Override
            public void completed(Integer bytesRead, ByteBuffer attachment) {
                attachment.flip();
                byte[] data = new byte[attachment.remaining()];
                attachment.get(data);
                System.out.println("Read: " + new String(data));
            }

            @Override
            public void failed(Throwable exc, ByteBuffer attachment) {
                exc.printStackTrace();
            }
        });
    }

    public void asyncSocketClient(String host, int port) throws Exception {
        AsynchronousSocketChannel channel = AsynchronousSocketChannel.open();

        // Connect asynchronously
        Future<Void> future = channel.connect(new InetSocketAddress(host, port));
        future.get();  // Wait for connection

        // Write asynchronously
        ByteBuffer buffer = ByteBuffer.wrap("Hello".getBytes());
        Future<Integer> writeFuture = channel.write(buffer);
        writeFuture.get();

        // Read asynchronously
        buffer.clear();
        Future<Integer> readFuture = channel.read(buffer);
        readFuture.get();

        channel.close();
    }
}
```

7.6 Interview Questions

7.6.1 Q1: Difference between InputStream and Reader?

InputStream	Reader
Byte-oriented (8-bit)	Character-oriented (16-bit Unicode)
For binary data	For text data
read() returns int (0-255)	read() returns int (0-65535)
FileInputStream	FileReader

7.6.2 Q2: What is the Decorator Pattern in Java I/O?

```
// Base stream
InputStream is = new FileInputStream("file.txt");

// Decorated with buffering
is = new BufferedInputStream(is);

// Decorated with data type reading
DataInputStream dis = new DataInputStream(is);

// Each decorator adds functionality without changing the interface
int value = dis.readInt();
```

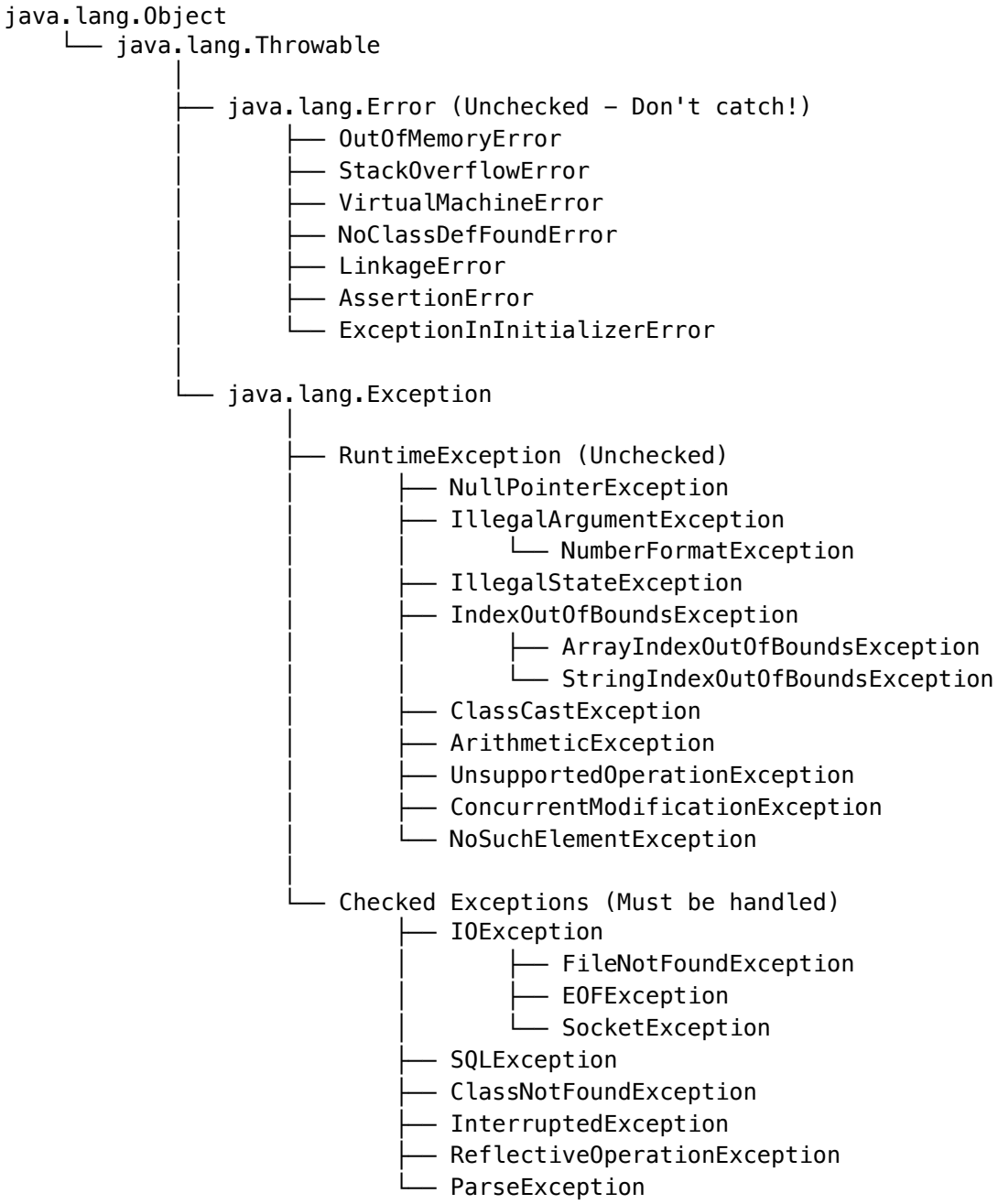
7.6.3 Q3: When to use NIO vs Classic I/O?

- **Classic I/O**: Simple file operations, text processing
- **NIO Channels/Buffers**: Large file transfers, need for memory-mapped files
- **NIO Selectors**: Multiple connections (servers), non-blocking required
- **NIO.2 Async**: High-concurrency servers, callback-based processing

8 Exception Handling and Error Management - Deep Dive

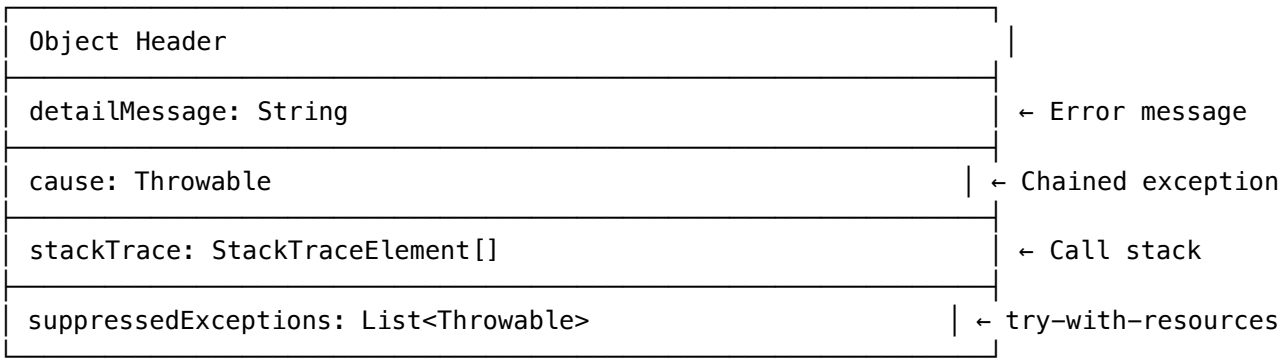
8.1 Exception Hierarchy

8.1.1 Complete Exception Class Hierarchy

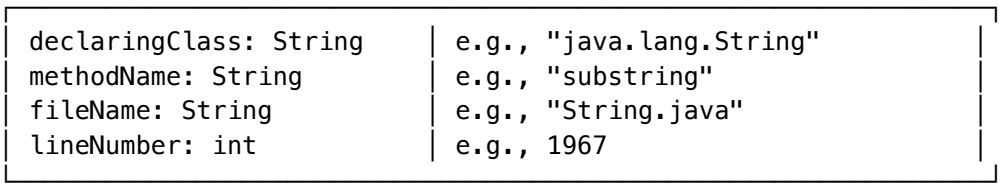


8.1.2 Memory Layout of Exception

Exception Object:

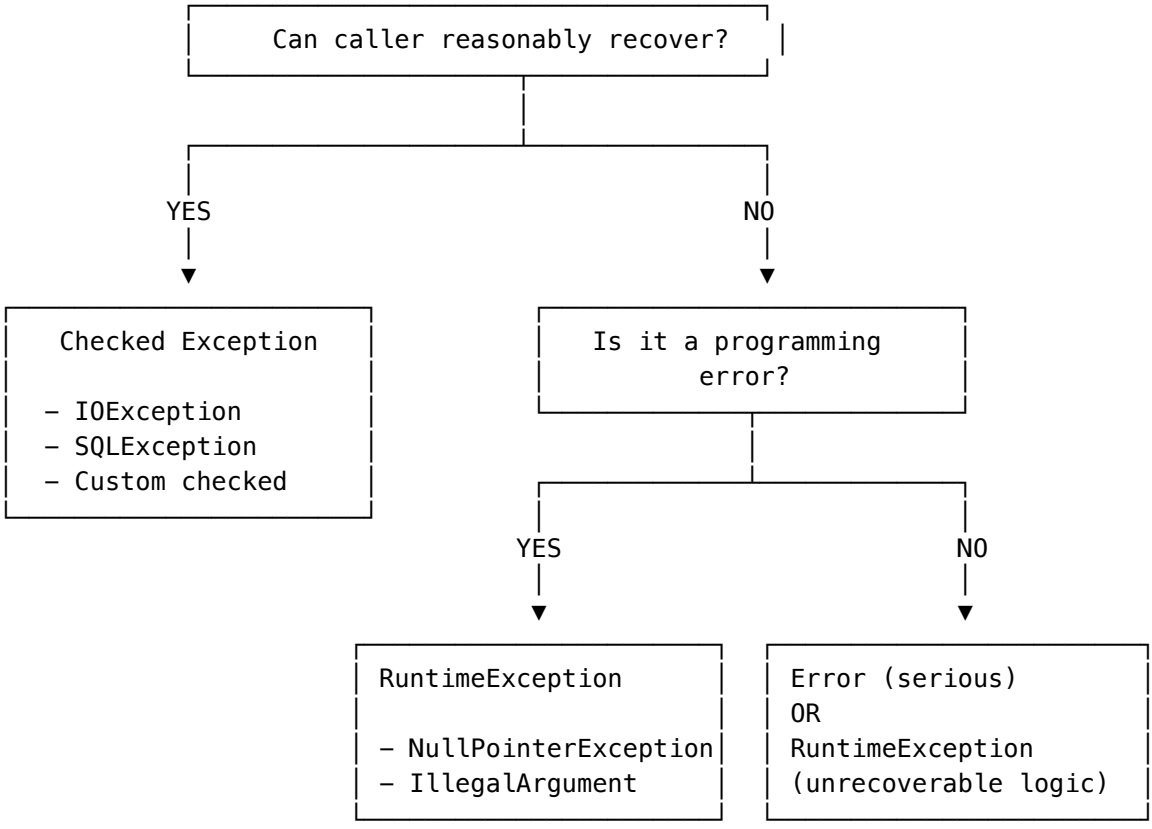


StackTraceElement:



8.2 Checked vs Unchecked Exceptions

8.2.1 Decision Framework



8.2.2 Examples with Rationale

```java
public class ExceptionDesignDemo {

    // CHECKED: Caller can and should handle
    // - File might not exist, but caller can provide alternative
    public String readConfig(String path) throws IOException {
        return Files.readString(Path.of(path));
    }

    // UNCHECKED: Programming error - null should never be passed
    public void process(Object data) {
        if (data == null) {
            throw new IllegalArgumentException("data cannot be null");
        }
        // Process data
    }

    // UNCHECKED: Programming error - invalid state
    public void withdraw(double amount) {
        if (!isAccountActive) {
            throw new IllegalStateException("Account is not active");
        }
        if (amount > balance) {
            // Could be checked if recovery is expected
            throw new InsufficientBalanceException("Insufficient funds");
        }
    }

    // Custom checked exception for business logic recovery
    public class InsufficientBalanceException extends Exception {
        private final double available;
        private final double requested;

        public InsufficientBalanceException(String message, double available, double requested
            super(message);
            this.available = available;
            this.requested = requested;
        }

        public double getShortfall() {
            return requested - available;
        }
    }
}
```

8.3 Exception Handling Internals

8.3.1 Exception Table in Bytecode

When you write a try-catch block, the compiler generates an exception table:

```
// Source code
public void method() {
    try {
        mayThrow();
    } catch (IOException e) {
        handle(e);
    } finally {
        cleanup();
    }
}

// Bytecode exception table (simplified)
Exception table:
   from    to  target type
     0     4     8   Class java/io/IOException  // try block → catch
     0     4    16   any                        // try block → finally
     8    12    16   any                        // catch block → finally
```

8.3.2 Stack Unwinding Process

```
Stack During Exception Propagation:

┌─────────────────────┐
│    method3()        │ ← Exception thrown here
│    line 45          │   throw new IOException()
├─────────────────────┤
│    method2()        │ ← No handler, unwind
│    line 23          │
├─────────────────────┤
│    method1()        │ ← Has catch(IOException), handle here
│    line 12          │
├─────────────────────┤
│    main()           │
└─────────────────────┘

Process:
1. Exception created (expensive - captures stack trace)
2. JVM searches current method's exception table
3. If no handler found, unwind to caller
4. Repeat until handler found or thread terminates
5. Execute finally blocks during unwinding
```

8.4 Best Practices

8.4.1 1. Exception Handling Patterns

```java
// 1. Don't catch Exception or Throwable
// BAD
try {
    doSomething();
} catch (Exception e) {  // Catches everything!
    e.printStackTrace();
}

// GOOD - Catch specific exceptions
try {
    doSomething();
} catch (IOException e) {
    handleIOError(e);
} catch (SQLException e) {
    handleDBError(e);
}

// 2. Don't swallow exceptions
// BAD
try {
    doSomething();
} catch (IOException e) {
    // Empty catch - exception lost!
}

// GOOD - At minimum, log it
try {
    doSomething();
} catch (IOException e) {
    logger.error("Operation failed", e);
    throw new ServiceException("Operation failed", e);
}

// 3. Use exception chaining
// GOOD - Preserve original cause
try {
    readFile();
} catch (IOException e) {
    throw new ConfigurationException("Failed to load config", e);
}
```

8.4.2 2. Resource Management

```java
// BAD - Manual resource management (error-prone)
FileInputStream fis = null;
try {
    fis = new FileInputStream("file.txt");
    // use fis
} catch (IOException e) {
    // handle
} finally {
    if (fis != null) {
        try {
            fis.close();  // Can also throw!
        } catch (IOException e) {
            // Suppress or log
        }
    }
}

// GOOD - Try-with-resources (Java 7+)
try (FileInputStream fis = new FileInputStream("file.txt");
     BufferedInputStream bis = new BufferedInputStream(fis)) {
    // use streams
} catch (IOException e) {
    // handle
}
// Both streams automatically closed in reverse order
```

8.4.3 3. Exception Translation

```java
public class UserService {
    private final UserRepository repository;

    // Translate low-level exceptions to domain exceptions
    public User findUser(int id) throws UserNotFoundException {
        try {
            return repository.findById(id)
                .orElseThrow(() -> new UserNotFoundException("User not found: " + id));
        } catch (SQLException e) {
            throw new DataAccessException("Database error while finding user", e);
        }
    }
}
```

8.5 Custom Exceptions

8.5.1 Designing Custom Exceptions

```java
// Base domain exception
public class DomainException extends RuntimeException {
    private final ErrorCode errorCode;

    public DomainException(ErrorCode errorCode, String message) {
        super(message);
        this.errorCode = errorCode;
    }

    public DomainException(ErrorCode errorCode, String message, Throwable cause) {
        super(message, cause);
        this.errorCode = errorCode;
    }

    public ErrorCode getErrorCode() {
        return errorCode;
    }
}

public enum ErrorCode {
    USER_NOT_FOUND("USR001"),
    INVALID_CREDENTIALS("USR002"),
    INSUFFICIENT_BALANCE("PAY001"),
    PAYMENT_FAILED("PAY002");

    private final String code;

    ErrorCode(String code) { this.code = code; }
    public String getCode() { return code; }
}

// Specific exceptions
public class UserNotFoundException extends DomainException {
    public UserNotFoundException(String userId) {
        super(ErrorCode.USER_NOT_FOUND, "User not found: " + userId);
    }
}

public class InsufficientBalanceException extends DomainException {
    private final BigDecimal available;
    private final BigDecimal required;

    public InsufficientBalanceException(BigDecimal available, BigDecimal required) {
        super(ErrorCode.INSUFFICIENT_BALANCE,
            String.format("Insufficient balance. Available: %s, Required: %s", available, requ
        this.available = available;
        this.required = required;
    }

    public BigDecimal getShortfall() {
        return required.subtract(available);
    }
}
```

8.6 Try-with-Resources Deep Dive

8.6.1 AutoCloseable Interface

```java
public interface AutoCloseable {
    void close() throws Exception;
}

// Custom resource
public class DatabaseConnection implements AutoCloseable {
    private Connection connection;

    public DatabaseConnection(String url) throws SQLException {
        this.connection = DriverManager.getConnection(url);
    }

    @Override
    public void close() throws SQLException {
        if (connection != null && !connection.isClosed()) {
            connection.close();
        }
    }
}

// Usage
try (DatabaseConnection db = new DatabaseConnection(url)) {
    // use db
}  // Automatically closed
```

8.6.2 Suppressed Exceptions

```java
public class SuppressedExceptionDemo {

    public void demonstrateSuppression() {
        try (ProblematicResource resource = new ProblematicResource()) {
            resource.doWork();  // Throws WorkException
        } catch (Exception e) {
            System.out.println("Primary: " + e.getMessage());

            // Access suppressed exceptions
            for (Throwable suppressed : e.getSuppressed()) {
                System.out.println("Suppressed: " + suppressed.getMessage());
            }
        }
    }
}

class ProblematicResource implements AutoCloseable {
    public void doWork() throws WorkException {
        throw new WorkException("Work failed");
    }

    @Override
    public void close() throws CloseException {
        throw new CloseException("Close failed");  // This is suppressed
    }
}
```

8.6.3 Effectively Final in Try-with-Resources (Java 9+)

```java
// Java 7/8 - Variable must be declared in try
try (BufferedReader br = new BufferedReader(new FileReader(path))) {
    // use br
}

// Java 9+ - Effectively final variables work
BufferedReader br = new BufferedReader(new FileReader(path));
try (br) {  // br is effectively final
    // use br
}
```

8.7 Exception Performance

8.7.1 Cost of Exceptions

```java
public class ExceptionPerformance {

    // Exceptions are expensive due to:
    // 1. Stack trace capture (fillInStackTrace)
    // 2. Stack unwinding
    // 3. Object creation

    // If stack trace not needed, can skip it
    public class FastException extends RuntimeException {
        public FastException(String message) {
            super(message, null, true, false);  // Don't capture stack trace
        }
    }

    // Pre-created exception (for very hot paths)
    private static final IndexOutOfBoundsException INDEX_EXCEPTION =
        new IndexOutOfBoundsException();

    // Don't use exceptions for flow control!
    // BAD
    public int findIndexBad(int[] arr, int value) {
        try {
            for (int i = 0; ; i++) {
                if (arr[i] == value) return i;
            }
        } catch (ArrayIndexOutOfBoundsException e) {
            return -1;
        }
    }

    // GOOD
    public int findIndexGood(int[] arr, int value) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == value) return i;
        }
        return -1;
    }
}
```

8.8 Interview Questions

8.8.1 Q1: What happens if both try and finally throw exceptions?

```java
try {
    throw new RuntimeException("Try exception");
} finally {
    throw new RuntimeException("Finally exception");
}
// Result: "Finally exception" is thrown, "Try exception" is lost!
// Use try-with-resources to avoid this (try exception suppresses close exception)
```

8.8.2 Q2: Can you return from finally block?

```java
// Yes, but DON'T DO IT!
public int badMethod() {
    try {
        return 1;
    } finally {
        return 2;  // This overwrites the try return!
    }
}
// Returns 2, not 1!
```

8.8.3 Q3: What is the difference between throw and throws?

```java
// throw - Actually throws an exception
public void validate(String input) {
    if (input == null) {
        throw new IllegalArgumentException("Input cannot be null");
    }
}

// throws - Declares that method might throw exception
public void readFile(String path) throws IOException {
    Files.readString(Path.of(path));
}
```

8.8.4 Q4: When does finally NOT execute?

```java
// 1. System.exit() called
try {
    System.exit(0);
} finally {
    System.out.println("Never printed");
}

// 2. JVM crashes
// 3. Infinite loop or thread killed before finally
// 4. Power outage (obviously)
```

9 Java Performance Tuning and Optimization - Deep Dive

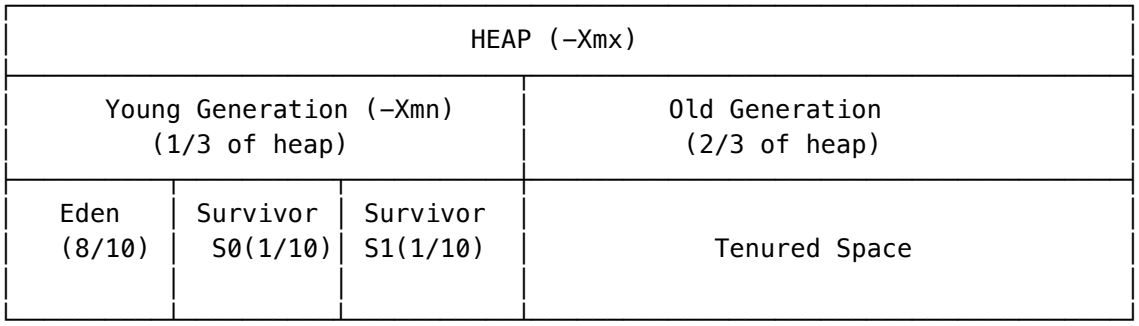
9.1 JVM Tuning Parameters

9.1.1 Memory Configuration

```
# Heap Size Configuration
-Xms<size>      # Initial heap size (e.g., -Xms2g)
-Xmx<size>      # Maximum heap size (e.g., -Xmx8g)
-Xmn<size>      # Young generation size

# Metaspace (Java 8+)
-XX:MetaspaceSize=256m       # Initial metaspace size
-XX:MaxMetaspaceSize=512m    # Maximum metaspace size

# Stack Size
-Xss<size>      # Thread stack size (e.g., -Xss512k)

# Direct Memory
-XX:MaxDirectMemorySize=1g   # For NIO direct buffers
```

9.1.2 Generation Sizing Strategy

Heap Layout:




```
-XX:NewRatio=2           # Old/Young ratio (2 means Old is 2x Young)
-XX:SurvivorRatio=8      # Eden/Survivor ratio
-XX:MaxTenuringThreshold=15  # Age before promotion to Old
```

9.1.3 GC Algorithm Selection

```
# Serial GC (Single-threaded, small heaps)
-XX:+UseSerialGC

# Parallel GC (Throughput-focused)
-XX:+UseParallelGC
-XX:ParallelGCThreads=4

# G1 GC (Default in Java 9+, balanced)
-XX:+UseG1GC
-XX:MaxGCPauseMillis=200    # Target pause time
-XX:G1HeapRegionSize=16m    # Region size (1-32MB)

# ZGC (Java 11+, ultra-low latency)
-XX:+UseZGC
-XX:ZCollectionInterval=5   # Minimum time between collections

# Shenandoah (Java 12+, low latency)
-XX:+UseShenandoahGC
```

9.2 Memory Optimization

9.2.1 Object Memory Layout

```
// Object size calculation (64-bit JVM with compressed oops)

// Empty object: 16 bytes
class Empty {}
// Header: 12 bytes + Padding: 4 bytes = 16 bytes

// With one int: 16 bytes
class WithInt { int x; }
// Header: 12 bytes + int: 4 bytes = 16 bytes

// With one long: 24 bytes
class WithLong { long x; }
// Header: 12 bytes + Padding: 4 bytes + long: 8 bytes = 24 bytes

// Memory-efficient field ordering
class Inefficient {
    boolean a;  // 1 byte + 7 padding
    long b;     // 8 bytes
    boolean c;  // 1 byte + 7 padding
    long d;     // 8 bytes
}  // Total: 32 bytes (wasted: 14 bytes)

class Efficient {
    long b;     // 8 bytes
    long d;     // 8 bytes
    boolean a;  // 1 byte
    boolean c;  // 1 byte + 6 padding
}  // Total: 24 bytes (wasted: 6 bytes)
```

9.2.2 Escape Analysis and Scalar Replacement

```
// JIT can eliminate allocations through escape analysis

public class EscapeAnalysisDemo {

    // Object ESCAPES - must be heap allocated
    public Point getPoint() {
        return new Point(1, 2);  // Escapes method
    }

    // Object does NOT escape - can be stack allocated or eliminated
    public int sumCoordinates() {
        Point p = new Point(1, 2);  // Never escapes
        return p.x + p.y;  // JIT can replace with: return 1 + 2;
    }

    // Scalar replacement: fields inlined as local variables
    public int calculate() {
        Point p = new Point(10, 20);
        // JIT optimization: no Point object created!
        // Becomes: int p_x = 10; int p_y = 20;
        return p.x * p.y;
    }
}

// Enable/disable escape analysis
// -XX:+DoEscapeAnalysis     (default: enabled)
// -XX:-DoEscapeAnalysis     (disable)
```

9.2.3 Object Pooling Patterns

```java
// Object pool for expensive objects
public class ObjectPool<T> {
    private final ConcurrentLinkedQueue<T> pool;
    private final Supplier<T> factory;
    private final Consumer<T> reset;
    private final int maxSize;
    private final AtomicInteger size = new AtomicInteger(0);

    public ObjectPool(Supplier<T> factory, Consumer<T> reset, int maxSize) {
        this.pool = new ConcurrentLinkedQueue<>();
        this.factory = factory;
        this.reset = reset;
        this.maxSize = maxSize;
    }

    public T borrow() {
        T obj = pool.poll();
        if (obj == null) {
            size.incrementAndGet();
            return factory.get();
        }
        return obj;
    }

    public void release(T obj) {
        if (size.get() <= maxSize) {
            reset.accept(obj);
            pool.offer(obj);
        }
    }
}

// Usage
ObjectPool<StringBuilder> sbPool = new ObjectPool<>(
    () -> new StringBuilder(256),
    sb -> sb.setLength(0),
    100
);

StringBuilder sb = sbPool.borrow();
try {
    sb.append("Hello").append(" World");
    return sb.toString();
} finally {
    sbPool.release(sb);
}
```

9.3 Garbage Collection Tuning

9.3.1 GC Log Analysis

```
# Enable GC logging (Java 9+)
-Xlog:gc*:file=gc.log:time,uptime,level,tags:filecount=5,filesize=10M

# Java 8 style
-XX:+PrintGCDetails
-XX:+PrintGCTimeStamps
-XX:+PrintGCDateStamps
-Xloggc:gc.log
```

9.3.2 GC Log Interpretation

```
# G1 GC Log Entry:
[2024-01-15T10:30:45.123+0000][1.234s][info][gc] GC(0) Pause Young (Normal)
    (G1 Evacuation Pause) 24M->8M(256M) 12.345ms

# Breakdown:
# GC(0)           - GC event number
# Pause Young     - Young generation collection
# (Normal)        - Normal collection (not concurrent)
# 24M->8M(256M)   - Heap: 24MB before → 8MB after (256MB total)
# 12.345ms        - Pause time
```

9.3.3 G1 GC Tuning

```
// G1 is region-based:
// - Heap divided into equal-sized regions (1-32MB)
// - Regions can be Eden, Survivor, Old, or Humongous
// - Humongous: Objects > 50% of region size

// Key G1 tuning parameters:
// -XX:MaxGCPauseMillis=200       Target pause time (default: 200ms)
// -XX:G1HeapRegionSize=16m       Region size (default: auto)
// -XX:G1NewSizePercent=5         Min young gen (default: 5%)
// -XX:G1MaxNewSizePercent=60     Max young gen (default: 60%)
// -XX:InitiatingHeapOccupancyPercent=45  Start mixed GC (default: 45%)
// -XX:G1MixedGCLiveThresholdPercent=85   Skip regions with high liveness
```

9.4 String Optimization

9.4.1 String Concatenation Performance

```java
public class StringOptimization {

    // BAD: String concatenation in loop
    public String badConcat(List<String> items) {
        String result = "";
        for (String item : items) {
            result += item + ", ";  // Creates new String each time!
        }
        return result;
    }

    // GOOD: StringBuilder for mutable operations
    public String goodConcat(List<String> items) {
        StringBuilder sb = new StringBuilder(items.size() * 20);  // Pre-size!
        for (String item : items) {
            sb.append(item).append(", ");
        }
        return sb.toString();
    }

    // BETTER: String.join for simple cases
    public String betterConcat(List<String> items) {
        return String.join(", ", items);
    }

    // BEST: Stream collectors for complex cases
    public String streamConcat(List<String> items) {
        return items.stream()
            .collect(Collectors.joining(", ", "[", "]"));
    }
}
```

9.4.2 String Deduplication

```java
// G1 can deduplicate strings automatically
// -XX:+UseStringDeduplication  (G1 only)

// Manual deduplication using intern()
public class StringDedup {
    private Map<String, WeakReference<String>> cache = new WeakHashMap<>();

    public String deduplicate(String s) {
        // For high-repetition strings
        return s.intern();  // Returns canonical reference

        // Custom deduplication (more control)
        // WeakReference<String> ref = cache.get(s);
        // if (ref != null && ref.get() != null) {
        //     return ref.get();
        // }
        // cache.put(s, new WeakReference<>(s));
        // return s;
    }
}
```

9.4.3 Compact Strings (Java 9+)

```
Java 9+ String representation:
- LATIN1: 1 byte per character (for ASCII-only strings)
- UTF16: 2 bytes per character (when needed)

Benefits:
- ~50% memory reduction for ASCII strings
- Automatic, no code changes needed
- -XX:-CompactStrings to disable (rarely needed)
```

9.5 Collection Performance

9.5.1 Choosing the Right Collection

Operation Time Complexity:

Operation	ArrayList	LinkedList	HashSet	TreeSet
get(index)	O(1)	O(n)	N/A	N/A
add(end)	O(1)*	O(1)	O(1)*	O(log n)
add(index)	O(n)	O(n)	N/A	N/A
remove(index)	O(n)	O(n)	N/A	N/A
contains()	O(n)	O(n)	O(1)*	O(log n)
iterator.remove()	O(n)	O(1)	O(1)*	O(log n)

* amortized

Map Performance:

Operation	HashMap	TreeMap	LinkedHashMap
get()	O(1)*	O(log n)	O(1)*
put()	O(1)*	O(log n)	O(1)*
remove()	O(1)*	O(log n)	O(1)*
Iteration order	Random	Sorted	Insertion order

9.5.2 Collection Sizing

```java
public class CollectionSizing {

    // Pre-size collections when size is known
    public void properSizing(int expectedSize) {
        // ArrayList: Avoid reallocations
        List<String> list = new ArrayList<>(expectedSize);

        // HashMap: Account for load factor (0.75)
        // Formula: expectedSize / 0.75 + 1
        Map<String, Object> map = new HashMap<>(expectedSize * 4 / 3 + 1);

        // Or simpler: just add 33%
        Map<String, Object> map2 = new HashMap<>((int) (expectedSize * 1.34));
    }

    // Avoid excessive reallocations
    public void avoidReallocations() {
        // BAD: Unknown size, multiple reallocations
        List<Integer> list = new ArrayList<>();
        for (int i = 0; i < 1_000_000; i++) {
            list.add(i);  // Multiple internal array copies
        }

        // GOOD: Pre-sized
        List<Integer> list2 = new ArrayList<>(1_000_000);
        for (int i = 0; i < 1_000_000; i++) {
            list2.add(i);  // No reallocations
        }
    }
}
```

9.5.3 Primitive Collections

```java
// Wrapper objects have overhead:
// - Object header: 12 bytes
// - int value: 4 bytes
// - Padding: 0 bytes
// Total: 16 bytes per Integer (vs 4 bytes for int)

// For large primitive collections, consider:
// 1. Eclipse Collections
// 2. Trove
// 3. Koloboke
// 4. FastUtil

// Example with primitive arrays
public class PrimitivePerformance {

    // ~4x memory difference
    int[] primitiveArray = new int[1_000_000];        // ~4 MB
    Integer[] wrapperArray = new Integer[1_000_000];  // ~16 MB + references

    // Use primitive streams
    public int sumPrimitive(int[] arr) {
        return IntStream.of(arr).sum();  // No boxing
    }
}
```

9.6 Concurrency Optimization

9.6.1 Lock-Free Data Structures

```
public class LockFreeCounter {
    private final AtomicLong counter = new AtomicLong(0);

    // Non-blocking increment
    public long increment() {
        return counter.incrementAndGet();
    }

    // CAS loop for complex operations
    public long incrementWithLimit(long limit) {
        long current, next;
        do {
            current = counter.get();
            if (current >= limit) return current;
            next = current + 1;
        } while (!counter.compareAndSet(current, next));
        return next;
    }
}

// LongAdder for high contention (better than AtomicLong)
public class HighContentionCounter {
    private final LongAdder adder = new LongAdder();

    public void increment() {
        adder.increment();  // Uses striped cells
    }

    public long sum() {
        return adder.sum();  // May not be exact under contention
    }
}
```

9.6.2 False Sharing Prevention

```java
// Cache lines are typically 64 bytes
// False sharing occurs when unrelated data shares a cache line

// BAD: Counters on same cache line
public class FalseSharing {
    volatile long counter1;  // Same cache line!
    volatile long counter2;  // Causes invalidation traffic
}

// GOOD: Padded to separate cache lines
public class NoPadding {
    volatile long counter1;
    long p1, p2, p3, p4, p5, p6, p7;  // Padding
    volatile long counter2;
}

// Java 8+: @Contended annotation
@sun.misc.Contended
public class ContentionFree {
    volatile long counter1;
    volatile long counter2;
}
// Requires: -XX:-RestrictContended
```

9.6.3 Thread Pool Sizing

```java
public class ThreadPoolSizing {

    // CPU-bound tasks: N threads (N = number of cores)
    ExecutorService cpuBound = Executors.newFixedThreadPool(
        Runtime.getRuntime().availableProcessors()
    );

    // I/O-bound tasks: More threads
    // Formula: N * (1 + W/C) where W = wait time, C = compute time
    // If 80% waiting: N * (1 + 0.8/0.2) = N * 5
    ExecutorService ioBound = Executors.newFixedThreadPool(
        Runtime.getRuntime().availableProcessors() * 5
    );

    // Mixed workload: Use work-stealing
    ExecutorService mixed = Executors.newWorkStealingPool();
}
```

9.7 Profiling and Benchmarking

9.7.1 JMH (Java Microbenchmark Harness)

```java
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
@State(Scope.Benchmark)
@Fork(value = 2, warmups = 1)
@Warmup(iterations = 5, time = 1)
@Measurement(iterations = 5, time = 1)
public class StringBenchmark {

    private List<String> items;

    @Setup
    public void setup() {
        items = new ArrayList<>();
        for (int i = 0; i < 100; i++) {
            items.add("item" + i);
        }
    }

    @Benchmark
    public String concatWithPlus() {
        String result = "";
        for (String item : items) {
            result += item;
        }
        return result;
    }

    @Benchmark
    public String concatWithBuilder() {
        StringBuilder sb = new StringBuilder();
        for (String item : items) {
            sb.append(item);
        }
        return sb.toString();
    }

    @Benchmark
    public String concatWithJoin() {
        return String.join("", items);
    }
}

// Run: mvn clean install && java -jar target/benchmarks.jar
```

9.7.2 Profiling Tools

```
Common profiling tools:
1. JVisualVM (bundled with JDK)
2. JProfiler (commercial)
3. YourKit (commercial)
4. async-profiler (free, low overhead)
5. Java Flight Recorder (JFR)

# Enable JFR (Java 11+)
java -XX:StartFlightRecording=duration=60s,filename=recording.jfr MyApp

# Analyze with JDK Mission Control (JMC)
jmc recording.jfr
```

9.8 Common Performance Anti-patterns

9.8.1 1. Premature Optimization

```
// DON'T optimize without measuring!
// Profile first, then optimize hot paths

// Often these micro-optimizations don't matter:
int x = value / 2;      // vs
int x = value >> 1;     // Compiler optimizes anyway

// Focus on:
// 1. Algorithm complexity (O(n²) → O(n log n))
// 2. I/O operations
// 3. Memory allocations in hot loops
// 4. Lock contention
```

9.8.2 2. Excessive Object Creation

```
// BAD: Creating objects in hot loop
public void processEvents(List<Event> events) {
    for (Event event : events) {
        DateFormat df = new SimpleDateFormat("yyyy-MM-dd");  // Created each time!
        String date = df.format(event.getDate());
    }
}

// GOOD: Reuse or use ThreadLocal
private static final ThreadLocal<DateFormat> DATE_FORMAT =
    ThreadLocal.withInitial(() -> new SimpleDateFormat("yyyy-MM-dd"));

public void processEventsBetter(List<Event> events) {
    DateFormat df = DATE_FORMAT.get();
    for (Event event : events) {
        String date = df.format(event.getDate());
    }
}
```

9.8.3 3. Inefficient Iteration

```java
// BAD: LinkedList with index access
LinkedList<String> list = getLinkedList();
for (int i = 0; i < list.size(); i++) {
    String s = list.get(i);  // O(n) each time! O(n²) total
}

// GOOD: Use iterator
for (String s : list) {
    // O(1) per element, O(n) total
}
```

9.9 Interview Questions

9.9.1 Q1: How would you identify a memory leak?

```java
// Signs of memory leak:
// 1. OutOfMemoryError
// 2. Increasing heap usage over time
// 3. GC taking longer and longer

// Investigation steps:
// 1. Take heap dump: jmap -dump:format=b,file=heap.hprof <pid>
// 2. Analyze with MAT (Memory Analyzer Tool)
// 3. Look for dominator tree, largest objects
// 4. Check for references that should have been cleared

// Common causes:
// - Collections growing without bounds
// - Static fields holding references
// - Listeners not unregistered
// - ThreadLocal not cleaned in thread pools
```

9.9.2 Q2: Explain JIT compilation levels

```
JIT Compilation Levels:
Level 0: Interpreter
Level 1: Simple C1 (quick compilation, few optimizations)
Level 2: Limited C1 (with profiling)
Level 3: Full C1 (with profiling)
Level 4: C2 (aggressive optimizations)

Tiered Compilation (default):
Interpreter → C1 (quick startup) → C2 (peak performance)

Key C2 optimizations:
- Inlining
- Escape analysis
- Loop unrolling
- Dead code elimination
- Lock elision
```

9.9.3 Q3: What is the difference between -Xms and -Xmx?

```
-Xms: Initial (minimum) heap size
-Xmx: Maximum heap size

Best practice: Set them equal to avoid heap resizing overhead
java -Xms4g -Xmx4g MyApp

Why resize is expensive:
1. JVM must allocate new memory
2. Copy objects to new space
3. Update all references
4. May trigger GC
```

10 Java Interview Questions and Competitive Programming Patterns

10.1 Core Java Interview Questions

10.1.1 Q1: Explain the difference between == and equals()

```java
public class EqualsDemo {
    public static void main(String[] args) {
        // == compares references (memory addresses)
        // equals() compares content (if properly overridden)

        String s1 = new String("hello");
        String s2 = new String("hello");
        String s3 = "hello";
        String s4 = "hello";

        System.out.println(s1 == s2);        // false (different objects)
        System.out.println(s1.equals(s2));   // true (same content)
        System.out.println(s3 == s4);        // true (String pool)
        System.out.println(s3.equals(s4));   // true

        // Integer caching (-128 to 127)
        Integer i1 = 100;
        Integer i2 = 100;
        Integer i3 = 200;
        Integer i4 = 200;

        System.out.println(i1 == i2);  // true (cached)
        System.out.println(i3 == i4);  // false (not cached)
    }
}
```

10.1.2 Q2: What is the contract between hashCode() and equals()?

```java
public class HashCodeEqualsContract {
    /*
     * CONTRACT:
     * 1. If equals() returns true, hashCode() MUST return same value
     * 2. If hashCode() returns different values, equals() MUST return false
     * 3. If hashCode() returns same value, equals() MAY return true or false
     * 4. hashCode() should be consistent for same object during execution
     */

    private int id;
    private String name;

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        HashCodeEqualsContract that = (HashCodeEqualsContract) o;
        return id == that.id && Objects.equals(name, that.name);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, name);  // Use same fields as equals()
    }
}

// What happens if contract is violated?
class BadHashCode {
    private int id;

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof BadHashCode)) return false;
        return this.id == ((BadHashCode) o).id;
    }

    // Missing hashCode() override!
    // HashMap/HashSet will NOT work correctly

    public static void main(String[] args) {
        Set<BadHashCode> set = new HashSet<>();
        BadHashCode b1 = new BadHashCode(1);
        BadHashCode b2 = new BadHashCode(1);

        set.add(b1);
        System.out.println(set.contains(b2));  // Might be false!
        // Because b1 and b2 have different hashCodes (Object.hashCode)
    }
}
```

10.1.3 Q3: Explain String immutability and String Pool

```java
public class StringInternals {
    public static void main(String[] args) {
        // String Pool (in Metaspace since Java 8)
        String s1 = "hello";           // Goes to pool
        String s2 = "hello";           // Same reference from pool
        String s3 = new String("hello"); // New object in heap
        String s4 = s3.intern();       // Returns pooled reference

        System.out.println(s1 == s2);  // true
        System.out.println(s1 == s3);  // false
        System.out.println(s1 == s4);  // true

        // Why immutable?
        // 1. Security: Strings used in class loading, network, files
        // 2. Thread Safety: No synchronization needed
        // 3. Caching: hashCode can be cached
        // 4. String Pool: Possible only with immutability

        // String concatenation optimization
        String result = "a" + "b" + "c";  // Compiled to "abc"

        // But in loops, use StringBuilder
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < 1000; i++) {
            sb.append(i);
        }
    }
}
```

10.1.4 Q4: Explain final, finally, and finalize

```java
public class FinalKeywords {

    // final variable - constant
    private final int MAX_SIZE = 100;

    // final reference - can modify object, not reference
    private final List<String> list = new ArrayList<>();

    // final method - cannot be overridden
    public final void cannotOverride() { }

    // final class - cannot be extended
    // public final class String { }

    // finally - always executes (almost)
    public void finallyDemo() {
        try {
            riskyOperation();
        } catch (Exception e) {
            handleError(e);
        } finally {
            cleanup();  // Always runs
        }
    }

    // When finally DOESN'T execute:
    // 1. System.exit() called
    // 2. JVM crashes
    // 3. Infinite loop in try/catch
    // 4. Thread killed

    // finalize() - DEPRECATED in Java 9
    @Override
    protected void finalize() throws Throwable {
        // DON'T USE! Unpredictable, slow, not guaranteed
        // Use try-with-resources and Cleaner API instead
    }
}
```

10.2 Competitive Programming Templates

10.2.1 Fast I/O Template

```java
import java.io.*;
import java.util.*;

public class FastIO {
    static BufferedReader br;
    static StringTokenizer st;
    static PrintWriter out;

    public static void main(String[] args) throws IOException {
        br = new BufferedReader(new InputStreamReader(System.in));
        out = new PrintWriter(new BufferedOutputStream(System.out));

        int t = nextInt();
        while (t-- > 0) {
            solve();
        }

        out.flush();
        out.close();
    }

    static void solve() throws IOException {
        int n = nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = nextInt();
        }
        // Your solution here
        out.println("Answer");
    }

    static String next() throws IOException {
        while (st == null || !st.hasMoreTokens())
            st = new StringTokenizer(br.readLine());
        return st.nextToken();
    }

    static int nextInt() throws IOException { return Integer.parseInt(next()); }
    static long nextLong() throws IOException { return Long.parseLong(next()); }
    static double nextDouble() throws IOException { return Double.parseDouble(next()); }
    static String nextLine() throws IOException { return br.readLine(); }
}
```

10.2.2 Common Data Structures for CP

```java
// 1. Priority Queue (Min/Max Heap)
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
PriorityQueue<int[]> customHeap = new PriorityQueue<>((a, b) -> a[0] - b[0]);

// 2. TreeMap/TreeSet (Balanced BST)
TreeMap<Integer, Integer> map = new TreeMap<>();
map.floorKey(k);    // Greatest key <= k
map.ceilingKey(k);  // Smallest key >= k
map.lowerKey(k);    // Greatest key < k
map.higherKey(k);   // Smallest key > k

// 3. Deque for sliding window
Deque<Integer> deque = new ArrayDeque<>();
deque.offerFirst(x);   // Add to front
deque.offerLast(x);    // Add to back
deque.pollFirst();     // Remove from front
deque.pollLast();      // Remove from back

// 4. BitSet for space-efficient boolean array
BitSet bitset = new BitSet(1_000_000);
bitset.set(i);          // Set bit i
bitset.get(i);          // Get bit i
bitset.nextSetBit(i);   // Next set bit >= i
```

10.3 Common Algorithmic Patterns

10.3.1 Binary Search Variations

```java
public class BinarySearchPatterns {

    // Find first occurrence
    public int findFirst(int[] arr, int target) {
        int left = 0, right = arr.length - 1;
        int result = -1;

        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (arr[mid] == target) {
                result = mid;
                right = mid - 1;  // Continue searching left
            } else if (arr[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return result;
    }

    // Find last occurrence
    public int findLast(int[] arr, int target) {
        int left = 0, right = arr.length - 1;
        int result = -1;

        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (arr[mid] == target) {
                result = mid;
                left = mid + 1;  // Continue searching right
            } else if (arr[mid] < target) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }
        return result;
    }

    // Binary search on answer (minimize/maximize)
    public int minCapacity(int[] weights, int days) {
        int left = Arrays.stream(weights).max().getAsInt();
        int right = Arrays.stream(weights).sum();

        while (left < right) {
            int mid = left + (right - left) / 2;
            if (canShip(weights, days, mid)) {
                right = mid;  // Try smaller capacity
            } else {
                left = mid + 1;
            }
        }
        return left;
    }

    private boolean canShip(int[] weights, int days, int capacity) {
        int daysNeeded = 1, currentLoad = 0;
        for (int w : weights) {
            if (currentLoad + w > capacity) {
                daysNeeded++;
                currentLoad = 0;
            }
            currentLoad += w;
        }
        return daysNeeded <= days;
```

10.3.2 Union-Find (Disjoint Set)

```java
public class UnionFind {
    private int[] parent;
    private int[] rank;
    private int count;  // Number of components

    public UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        count = n;
        for (int i = 0; i < n; i++) {
            parent[i] = i;
        }
    }

    // Find with path compression
    public int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]);  // Path compression
        }
        return parent[x];
    }

    // Union by rank
    public boolean union(int x, int y) {
        int rootX = find(x);
        int rootY = find(y);

        if (rootX == rootY) return false;

        if (rank[rootX] < rank[rootY]) {
            parent[rootX] = rootY;
        } else if (rank[rootX] > rank[rootY]) {
            parent[rootY] = rootX;
        } else {
            parent[rootY] = rootX;
            rank[rootX]++;
        }
        count--;
        return true;
    }

    public boolean connected(int x, int y) {
        return find(x) == find(y);
    }

    public int getCount() {
        return count;
    }
}
```

10.3.3 Segment Tree

```java
public class SegmentTree {
    private int[] tree;
    private int n;

    public SegmentTree(int[] arr) {
        n = arr.length;
        tree = new int[4 * n];
        build(arr, 0, 0, n - 1);
    }

    private void build(int[] arr, int node, int start, int end) {
        if (start == end) {
            tree[node] = arr[start];
        } else {
            int mid = (start + end) / 2;
            build(arr, 2 * node + 1, start, mid);
            build(arr, 2 * node + 2, mid + 1, end);
            tree[node] = tree[2 * node + 1] + tree[2 * node + 2];
        }
    }

    public void update(int idx, int val) {
        update(0, 0, n - 1, idx, val);
    }

    private void update(int node, int start, int end, int idx, int val) {
        if (start == end) {
            tree[node] = val;
        } else {
            int mid = (start + end) / 2;
            if (idx <= mid) {
                update(2 * node + 1, start, mid, idx, val);
            } else {
                update(2 * node + 2, mid + 1, end, idx, val);
            }
            tree[node] = tree[2 * node + 1] + tree[2 * node + 2];
        }
    }

    public int query(int left, int right) {
        return query(0, 0, n - 1, left, right);
    }

    private int query(int node, int start, int end, int left, int right) {
        if (right < start || left > end) {
            return 0;  // Out of range
        }
        if (left <= start && end <= right) {
            return tree[node];  // Fully in range
        }
        int mid = (start + end) / 2;
        return query(2 * node + 1, start, mid, left, right) +
               query(2 * node + 2, mid + 1, end, left, right);
    }
}
```

10.3.4 Trie (Prefix Tree)

```java
public class Trie {
    private TrieNode root;

    public Trie() {
        root = new TrieNode();
    }

    public void insert(String word) {
        TrieNode node = root;
        for (char c : word.toCharArray()) {
            if (!node.children.containsKey(c)) {
                node.children.put(c, new TrieNode());
            }
            node = node.children.get(c);
        }
        node.isEndOfWord = true;
    }

    public boolean search(String word) {
        TrieNode node = searchPrefix(word);
        return node != null && node.isEndOfWord;
    }

    public boolean startsWith(String prefix) {
        return searchPrefix(prefix) != null;
    }

    private TrieNode searchPrefix(String prefix) {
        TrieNode node = root;
        for (char c : prefix.toCharArray()) {
            if (!node.children.containsKey(c)) {
                return null;
            }
            node = node.children.get(c);
        }
        return node;
    }

    class TrieNode {
        Map<Character, TrieNode> children = new HashMap<>();
        boolean isEndOfWord = false;
    }
}
```

10.4 OOP and Design Questions

10.4.1 Q5: Explain Immutable Class Design

```java
public final class ImmutablePerson {  // 1. Class is final
    private final String name;         // 2. All fields are final
    private final int age;
    private final List<String> hobbies;

    public ImmutablePerson(String name, int age, List<String> hobbies) {
        this.name = name;
        this.age = age;
        this.hobbies = List.copyOf(hobbies);  // 3. Defensive copy
    }

    public String getName() { return name; }
    public int getAge() { return age; }

    public List<String> getHobbies() {
        return hobbies;  // Already immutable from List.copyOf
        // Or: return Collections.unmodifiableList(new ArrayList<>(hobbies));
    }

    // 4. No setters
}
```

10.4.2 Q6: When to use Interface vs Abstract Class?

```java
// Use INTERFACE when:
// - Defining a contract/capability
// - Multiple inheritance needed
// - No shared state required

interface Flyable {
    void fly();
    default void land() {
        System.out.println("Landing...");
    }
}

interface Swimmable {
    void swim();
}

class Duck implements Flyable, Swimmable {
    public void fly() { }
    public void swim() { }
}

// Use ABSTRACT CLASS when:
// - Shared code/state among subclasses
// - Template method pattern
// - Controlled extension (protected members)

abstract class Animal {
    protected String name;  // Shared state

    public Animal(String name) {
        this.name = name;
    }

    // Template method
    public final void eat() {
        find();
        consume();
        digest();
    }

    protected abstract void find();
    protected abstract void consume();

    protected void digest() {
        System.out.println("Digesting...");
    }
}
```

10.5 Multithreading Interview Questions

10.5.1 Q7: Implement a Thread-Safe Singleton

```
// Best approach: Enum singleton
public enum Singleton {
    INSTANCE;

    public void doSomething() { }
}

// Why enum is best:
// - Thread-safe by JVM
// - Prevents reflection attacks
// - Handles serialization
// - Lazy initialization
```

10.5.2 Q8: Difference between synchronized and Lock

```
public class LockVsSynchronized {
    private final Object lock = new Object();
    private final ReentrantLock reentrantLock = new ReentrantLock();

    // synchronized: simpler but less flexible
    public synchronized void method1() { }

    public void method2() {
        synchronized (lock) {
            // critical section
        }
    }

    // Lock: more features
    public void method3() {
        reentrantLock.lock();
        try {
            // critical section
        } finally {
            reentrantLock.unlock();  // Must be in finally!
        }
    }

    // Lock advantages:
    // - tryLock() with timeout
    // - lockInterruptibly()
    // - Multiple conditions
    // - Fair locking option
}
```

10.5.3 Q9: How to avoid deadlock?

```java
public class DeadlockPrevention {

    private final Object lockA = new Object();
    private final Object lockB = new Object();

    // DEADLOCK-PRONE
    public void method1() {
        synchronized (lockA) {
            synchronized (lockB) {
                // Thread 1: lockA -> lockB
            }
        }
    }

    public void method2() {
        synchronized (lockB) {
            synchronized (lockA) {
                // Thread 2: lockB -> lockA  -- DEADLOCK!
            }
        }
    }

    // SOLUTION 1: Lock ordering
    public void safeMethod1() {
        synchronized (lockA) {  // Always lock A first
            synchronized (lockB) {
            }
        }
    }

    public void safeMethod2() {
        synchronized (lockA) {  // Always lock A first
            synchronized (lockB) {
            }
        }
    }

    // SOLUTION 2: Lock timeout
    private final Lock lock1 = new ReentrantLock();
    private final Lock lock2 = new ReentrantLock();

    public void tryLockApproach() {
        while (true) {
            if (lock1.tryLock()) {
                try {
                    if (lock2.tryLock()) {
                        try {
                            // Critical section
                            return;
                        } finally {
                            lock2.unlock();
                        }
                    }
                } finally {
                    lock1.unlock();
                }
            }
            // Back off and retry
            Thread.sleep(10);
        }
    }
}
```

10.6 Tricky Questions and Gotchas

10.6.1 Q10: What will this code print?

```
public class TrickyQuestions {

    // Question 1: Integer caching
    public static void question1() {
        Integer a = 127;
        Integer b = 127;
        Integer c = 128;
        Integer d = 128;

        System.out.println(a == b);  // true (cached)
        System.out.println(c == d);  // false (not cached)
    }

    // Question 2: String operations
    public static void question2() {
        String s1 = "hello";
        String s2 = "hel" + "lo";
        String s3 = "hel";
        String s4 = s3 + "lo";

        System.out.println(s1 == s2);  // true (compile-time constant)
        System.out.println(s1 == s4);  // false (runtime concatenation)
        System.out.println(s1 == s4.intern());  // true
    }

    // Question 3: finally return
    public static int question3() {
        try {
            return 1;
        } finally {
            return 2;  // This overwrites the return!
        }
    }
    // Returns 2!

    // Question 4: Short-circuit evaluation
    public static void question4() {
        int i = 0;
        boolean result = true || (++i > 0);
        System.out.println(i);  // 0 (second part not evaluated)
    }

    // Question 5: Array covariance
    public static void question5() {
        Object[] objects = new String[3];
        objects[0] = "Hello";
        objects[1] = 123;  // ArrayStoreException at runtime!
    }
}
```

10.6.2 Q11: Memory Leaks in Java

```java
public class MemoryLeakExamples {

    // Leak 1: Static collection growing forever
    private static List<Object> cache = new ArrayList<>();

    public void addToCache(Object obj) {
        cache.add(obj);  // Never removed = memory leak
    }

    // Leak 2: Listener not unregistered
    public class EventListener {
        public void register() {
            EventManager.addListener(this);  // Strong reference
        }
        // Missing: unregister() in lifecycle methods
    }

    // Leak 3: ThreadLocal in thread pool
    private static final ThreadLocal<byte[]> BUFFER =
        ThreadLocal.withInitial(() -> new byte[1024 * 1024]);

    public void processWithLeak() {
        byte[] buffer = BUFFER.get();
        // ... use buffer
        // Thread returns to pool, but ThreadLocal remains!
    }

    public void processCorrectly() {
        try {
            byte[] buffer = BUFFER.get();
            // ... use buffer
        } finally {
            BUFFER.remove();  // Clean up!
        }
    }

    // Leak 4: Inner class holding reference to outer
    public class Outer {
        private byte[] data = new byte[10_000_000];

        public Runnable createTask() {
            return new Runnable() {  // Holds reference to Outer
                @Override
                public void run() {
                    // Even if we don't use 'data', Outer can't be GC'd
                }
            };
        }

        // Fix: Use static inner class or lambda
        public Runnable createTaskFixed() {
            return () -> { };  // No implicit reference
        }
    }
}
```

10.7 System Design Considerations

10.7.1 Choosing the Right Data Structure

```
Use Case → Data Structure:
- Fast lookup by key → HashMap (O(1))
- Sorted order needed → TreeMap (O(log n))
- Insertion order → LinkedHashMap
- Thread-safe map → ConcurrentHashMap
- High read, low write → CopyOnWriteArrayList
- Producer-consumer → BlockingQueue
- Priority processing → PriorityQueue
- Unique elements → HashSet/TreeSet
- Range queries → TreeMap (subMap, headMap, tailMap)
- LRU cache → LinkedHashMap with removeEldestEntry()
```

10.7.2 LRU Cache Implementation

```java
public class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;

    public LRUCache(int capacity) {
        super(capacity, 0.75f, true);  // accessOrder = true
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;
    }
}

// Thread-safe version
public class ConcurrentLRUCache<K, V> {
    private final Map<K, V> cache;
    private final ReadWriteLock lock = new ReentrantReadWriteLock();

    public ConcurrentLRUCache(int capacity) {
        this.cache = new LinkedHashMap<K, V>(capacity, 0.75f, true) {
            @Override
            protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
                return size() > capacity;
            }
        };
    }

    public V get(K key) {
        lock.readLock().lock();
        try {
            return cache.get(key);
        } finally {
            lock.readLock().unlock();
        }
    }

    public void put(K key, V value) {
        lock.writeLock().lock();
        try {
            cache.put(key, value);
        } finally {
            lock.writeLock().unlock();
        }
    }
}
```

11 Blocking vs Non-Blocking I/O - Deep Dive

11.1 I/O Models Overview

11.1.1 The Five I/O Models (Unix/Linux)

```
┌─────────────────────────────────────────────────────────────────────────┐
│                        I/O OPERATION PHASES                             │
├─────────────────────────────────────────────────────────────────────────┤
│  Phase 1: Wait for data to be ready (network → kernel buffer)           │
│  Phase 2: Copy data from kernel to user space (kernel → application)    │
└─────────────────────────────────────────────────────────────────────────┘
```

Model Comparison:

Model	Phase 1	Phase 2	Thread Behavior
Blocking I/O	BLOCKS	BLOCKS	Waits entirely
Non-blocking I/O	Returns EAGAIN	BLOCKS	Polls repeatedly
I/O Multiplexing	BLOCKS on select	BLOCKS	Monitors many
Signal-driven I/O	Returns (signal)	BLOCKS	Notified async
Asynchronous I/O	Returns	Returns	Fully async

11.1.2 Java I/O Evolution

Java I/O History:

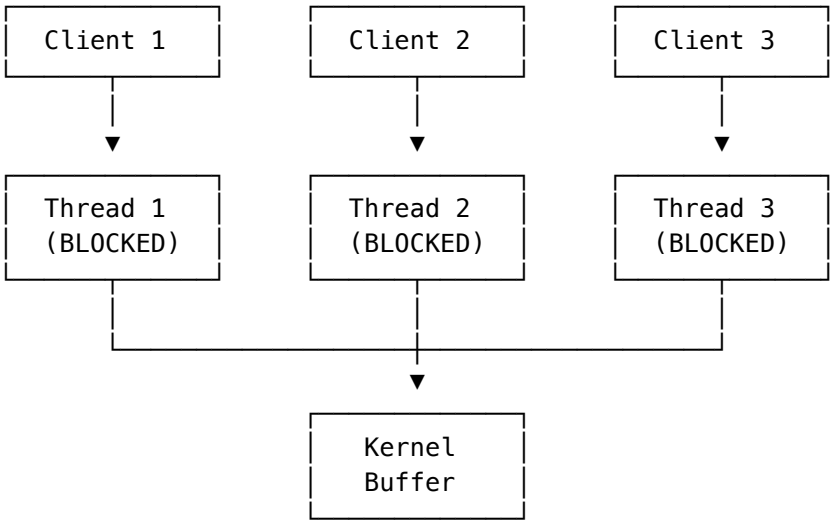
Java 1.0	java.io	Blocking, stream-oriented
Java 1.4	java.nio	Non-blocking, buffer-oriented, selectors
Java 7	java.nio.file	NIO.2, async file I/O, Path API
Java 7	AsynchronousChannel	True async I/O with callbacks/Future

---

11.2 Blocking I/O (BIO)

11.2.1 How Blocking I/O Works

Blocking I/O Thread Model:



Problem: One thread per connection = 10,000 connections = 10,000 threads!

11.2.2 Blocking I/O Example

```
public class BlockingIOServer {

    public static void main(String[] args) throws IOException {
        ServerSocket serverSocket = new ServerSocket(8080);
        System.out.println("Server started on port 8080");

        while (true) {
            // BLOCKS until client connects
            Socket clientSocket = serverSocket.accept();

            // One thread per client (traditional approach)
            new Thread(() -> handleClient(clientSocket)).start();
        }
    }

    private static void handleClient(Socket socket) {
        try (BufferedReader reader = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
             PrintWriter writer = new PrintWriter(
                socket.getOutputStream(), true)) {

            String line;
            // BLOCKS until data available or connection closed
            while ((line = reader.readLine()) != null) {
                writer.println("Echo: " + line);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

11.2.3 Thread Pool Improvement

```
public class ThreadPoolBlockingServer {
    private static final ExecutorService pool =
        Executors.newFixedThreadPool(200);  // Limited threads

    public static void main(String[] args) throws IOException {
        ServerSocket serverSocket = new ServerSocket(8080);

        while (true) {
            Socket clientSocket = serverSocket.accept();
            pool.submit(() -> handleClient(clientSocket));
        }
    }

    // Still blocking, but with bounded threads
    // Problem: 200 threads = max 200 concurrent connections
}
```

11.2.4 Blocking I/O Internals

System Call Flow (read):

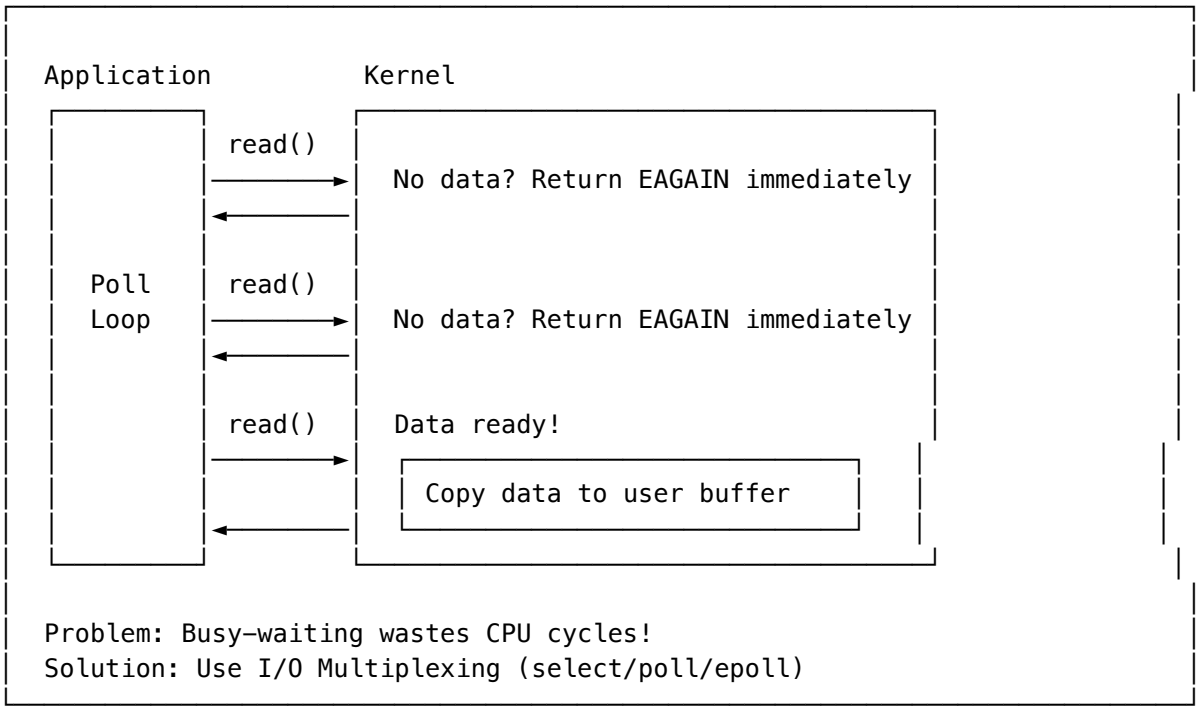
```
┌──────────────────────────────────────────────────────────────────────────┐
│ 1. Application calls read()                                              │
│ 2. Thread transitions to KERNEL MODE                                     │
│ 3. Kernel checks if data available in socket buffer                      │
│    - If NO data: Thread put in WAIT QUEUE, context switch                │
│    - If data available: Copy to user buffer                              │
│ 4. When data arrives (interrupt), thread moved to READY QUEUE            │
│ 5. Scheduler eventually runs thread, returns to USER MODE                │
│ 6. read() returns with data                                              │
└──────────────────────────────────────────────────────────────────────────┘
```

Thread States During Blocking I/O:
RUNNABLE → read() → BLOCKED (waiting for I/O) → RUNNABLE (data ready)

11.3 Non-Blocking I/O (NIO)

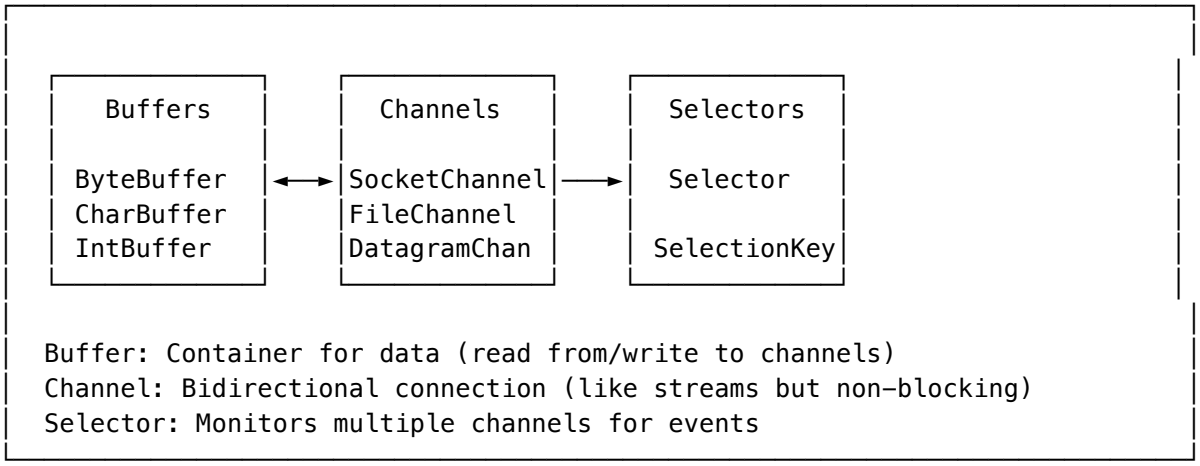
11.3.1 How Non-Blocking I/O Works

Non-Blocking I/O Model:



11.3.2 Java NIO Core Components

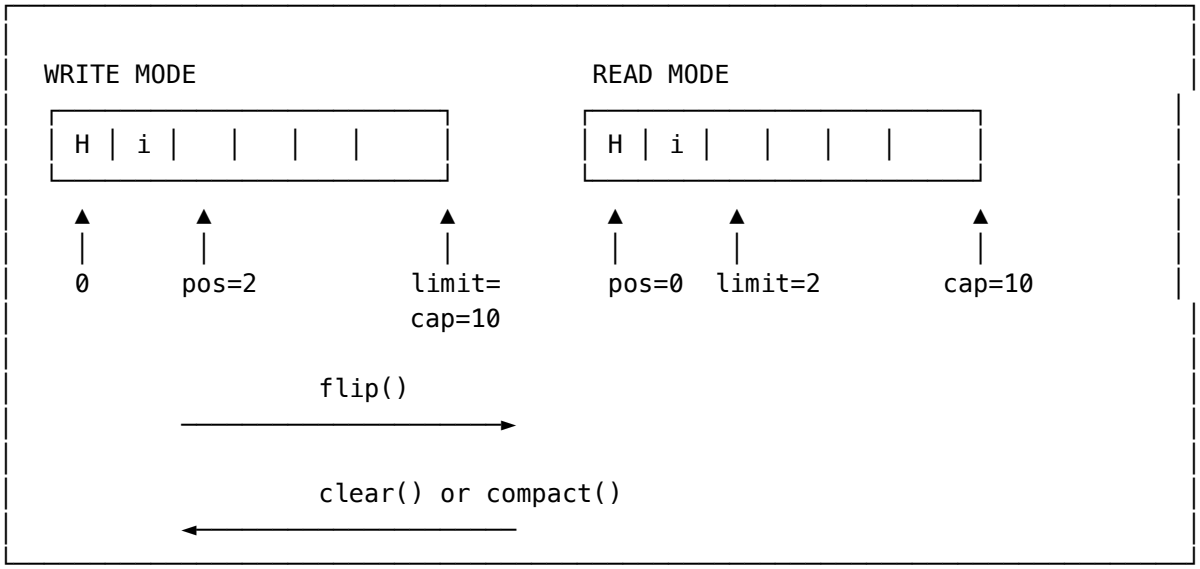
NIO Architecture:



11.3.3 Buffer Internals

```java
public class BufferInternals {

    public static void main(String[] args) {
        // Buffer has 4 key properties:
        // capacity: Maximum elements it can hold (fixed)
        // limit: First element that should not be read/written
        // position: Next element to be read/written
        // mark: Remembered position (optional)

        // Invariant: 0 <= mark <= position <= limit <= capacity

        ByteBuffer buffer = ByteBuffer.allocate(10);
        // State: position=0, limit=10, capacity=10

        buffer.put((byte) 'H');
        buffer.put((byte) 'i');
        // State: position=2, limit=10, capacity=10

        buffer.flip();  // Prepare for reading
        // State: position=0, limit=2, capacity=10

        byte b = buffer.get();  // Returns 'H'
        // State: position=1, limit=2, capacity=10

        buffer.clear();  // Prepare for writing (doesn't erase data)
        // State: position=0, limit=10, capacity=10

        buffer.compact();  // Keep unread data, prepare for writing
        // Moves unread data to beginning
    }
}
```

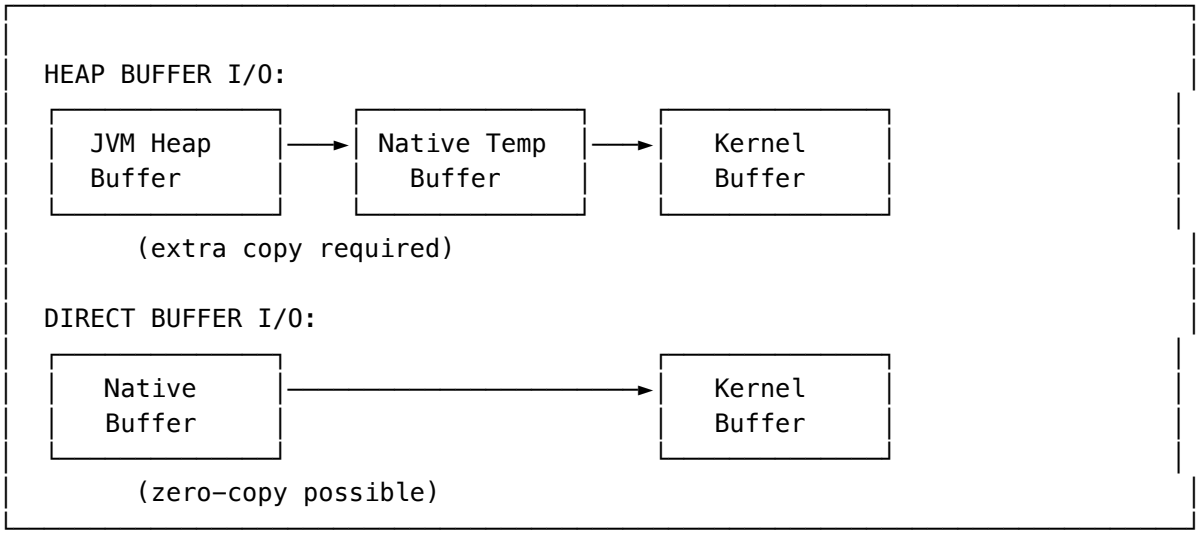
Buffer State Transitions:



11.3.4 Direct vs Heap Buffers

```java
public class DirectVsHeapBuffer {

    public static void compare() {
        // Heap Buffer: Allocated in JVM heap
        ByteBuffer heapBuffer = ByteBuffer.allocate(1024);
        // - Subject to GC
        // - May require extra copy for I/O (JVM heap → native memory)
        // - Faster allocation

        // Direct Buffer: Allocated in native memory
        ByteBuffer directBuffer = ByteBuffer.allocateDirect(1024);
        // - Not subject to GC (but wrapper object is)
        // - Zero-copy I/O possible
        // - Slower allocation
        // - Use for long-lived, I/O-heavy buffers
    }
}
```

Memory Layout:



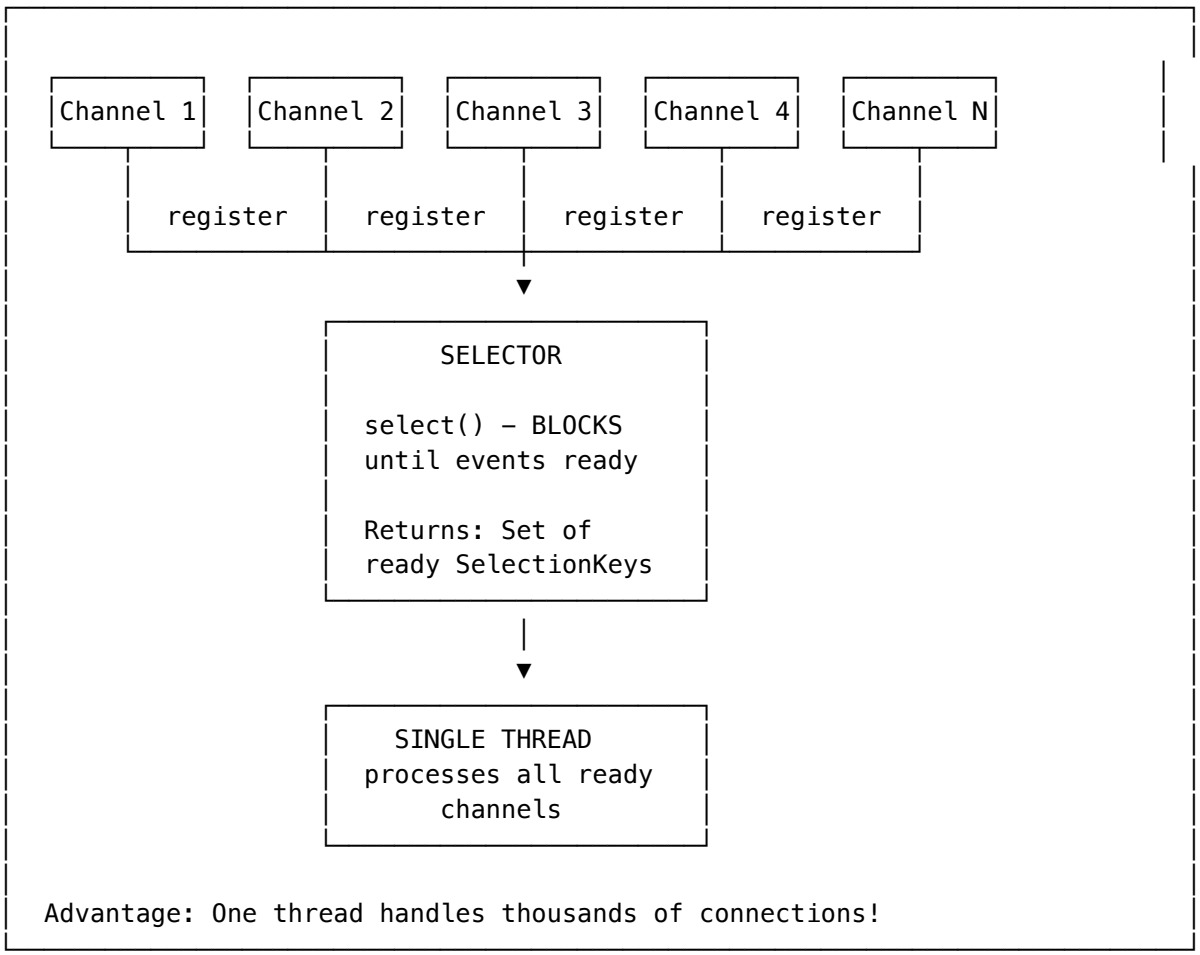
11.3.5 Channel Operations

```
public class ChannelExample {

    public static void nonBlockingChannel() throws IOException {
        // Create non-blocking socket channel
        SocketChannel channel = SocketChannel.open();
        channel.configureBlocking(false);  // KEY: Non-blocking mode

        channel.connect(new InetSocketAddress("localhost", 8080));

        // Non-blocking connect - may not complete immediately
        while (!channel.finishConnect()) {
            // Do other work while connecting
            System.out.println("Still connecting...");
        }

        ByteBuffer buffer = ByteBuffer.allocate(1024);

        // Non-blocking read - returns immediately
        int bytesRead = channel.read(buffer);
        // bytesRead = -1: End of stream
        // bytesRead = 0: No data available (non-blocking)
        // bytesRead > 0: Data read

        if (bytesRead == 0) {
            // No data available, do something else
        }
    }
}
```

11.4 Multiplexing with Selectors

11.4.1 Selector Architecture

Selector Model (I/O Multiplexing):



11.4.2 SelectionKey Operations

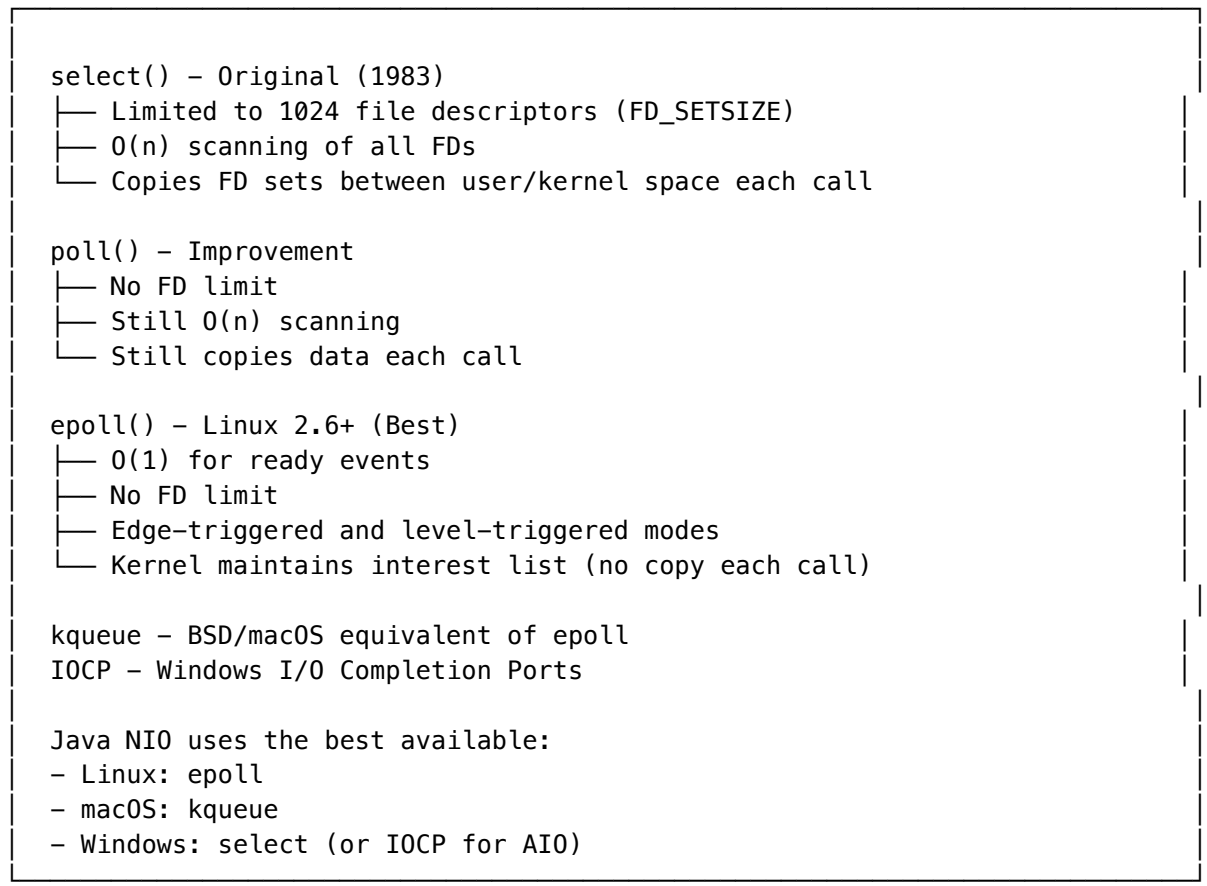
```
public class SelectionKeyOperations {

    // Interest operations (what we want to monitor)
    public static final int OP_READ = SelectionKey.OP_READ;      // 1
    public static final int OP_WRITE = SelectionKey.OP_WRITE;    // 4
    public static final int OP_CONNECT = SelectionKey.OP_CONNECT; // 8
    public static final int OP_ACCEPT = SelectionKey.OP_ACCEPT;  // 16

    public void registerChannel(Selector selector,
                                SocketChannel channel) throws IOException {
        channel.configureBlocking(false);

        // Register channel with selector for READ events
        SelectionKey key = channel.register(selector, OP_READ);

        // Attach custom object to key
        key.attach(new ConnectionContext());

        // Change interest ops later
        key.interestOps(OP_READ | OP_WRITE);

        // Check ready operations
        if (key.isReadable()) { /* handle read */ }
        if (key.isWritable()) { /* handle write */ }
        if (key.isConnectable()) { /* finish connect */ }
        if (key.isAcceptable()) { /* accept connection */ }
    }
}
```

11.4.3 Complete NIO Server Example

```
public class NIOServer {
    private Selector selector;
    private ServerSocketChannel serverChannel;
    private ByteBuffer buffer = ByteBuffer.allocate(1024);

    public void start(int port) throws IOException {
        // Open selector
        selector = Selector.open();

        // Open server channel
        serverChannel = ServerSocketChannel.open();
        serverChannel.bind(new InetSocketAddress(port));
        serverChannel.configureBlocking(false);

        // Register for ACCEPT events
        serverChannel.register(selector, SelectionKey.OP_ACCEPT);

        System.out.println("NIO Server started on port " + port);

        // Event loop
        while (true) {
            // Block until at least one channel is ready
            int readyChannels = selector.select();

            if (readyChannels == 0) continue;

            // Get ready keys
            Set<SelectionKey> selectedKeys = selector.selectedKeys();
            Iterator<SelectionKey> keyIterator = selectedKeys.iterator();

            while (keyIterator.hasNext()) {
                SelectionKey key = keyIterator.next();

                if (key.isAcceptable()) {
                    handleAccept(key);
                } else if (key.isReadable()) {
                    handleRead(key);
                } else if (key.isWritable()) {
                    handleWrite(key);
                }

                // IMPORTANT: Remove processed key
                keyIterator.remove();
            }
        }
    }

    private void handleAccept(SelectionKey key) throws IOException {
        ServerSocketChannel server = (ServerSocketChannel) key.channel();
        SocketChannel client = server.accept();
        client.configureBlocking(false);

        // Register new client for READ events
        client.register(selector, SelectionKey.OP_READ);
        System.out.println("Client connected: " + client.getRemoteAddress());
    }

    private void handleRead(SelectionKey key) throws IOException {
        SocketChannel client = (SocketChannel) key.channel();
        buffer.clear();

        int bytesRead = client.read(buffer);

        if (bytesRead == -1) {
            // Client disconnected
            client.close();
```

11.4.4 OS-Level Multiplexing

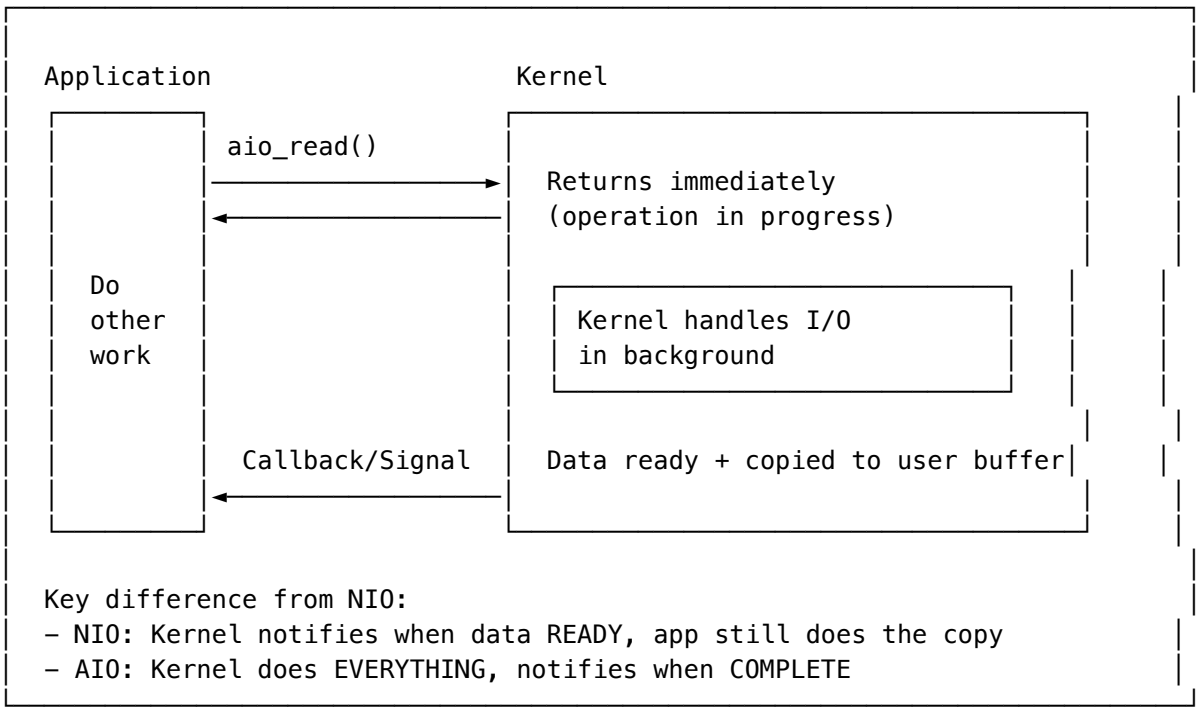
Linux I/O Multiplexing Evolution:



11.5 Asynchronous I/O (AIO)

11.5.1 True Asynchronous I/O

Async I/O Model:



11.5.2 Java AIO (NIO.2) Example

```
public class AsyncIOExample {

    // Method 1: Using Future
    public void readWithFuture() throws Exception {
        AsynchronousFileChannel channel = AsynchronousFileChannel.open(
            Paths.get("data.txt"), StandardOpenOption.READ);

        ByteBuffer buffer = ByteBuffer.allocate(1024);

        // Returns immediately with Future
        Future<Integer> future = channel.read(buffer, 0);

        // Do other work while I/O in progress
        doOtherWork();

        // Block until complete (or check isDone())
        Integer bytesRead = future.get();

        buffer.flip();
        System.out.println("Read " + bytesRead + " bytes");
    }

    // Method 2: Using CompletionHandler (callback)
    public void readWithCallback() throws Exception {
        AsynchronousFileChannel channel = AsynchronousFileChannel.open(
            Paths.get("data.txt"), StandardOpenOption.READ);

        ByteBuffer buffer = ByteBuffer.allocate(1024);

        channel.read(buffer, 0, buffer, new CompletionHandler<Integer, ByteBuffer>() {
            @Override
            public void completed(Integer bytesRead, ByteBuffer attachment) {
                attachment.flip();
                byte[] data = new byte[attachment.remaining()];
                attachment.get(data);
                System.out.println("Read: " + new String(data));
            }

            @Override
            public void failed(Throwable exc, ByteBuffer attachment) {
                System.err.println("Read failed: " + exc.getMessage());
            }
        });

        // Method returns immediately, callback invoked when complete
        System.out.println("Read initiated, doing other work...");
    }
}
```

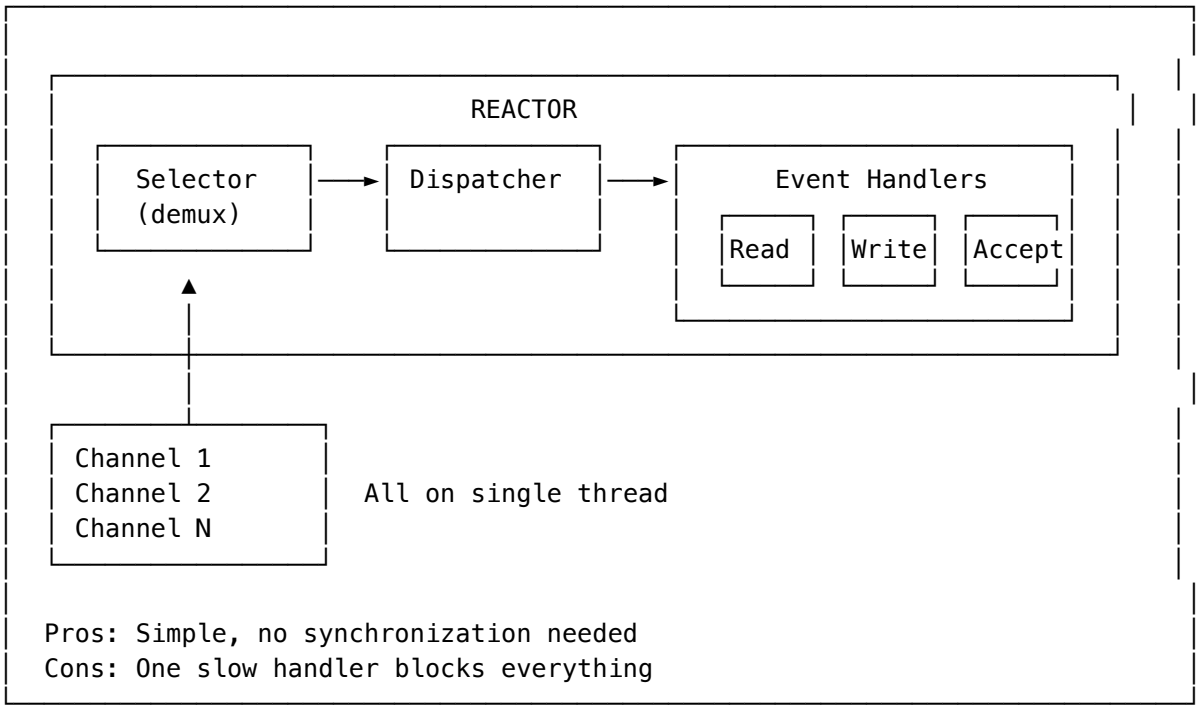
11.5.3 Async Socket Server

```
public class AsyncSocketServer {

    public void start(int port) throws Exception {
        AsynchronousServerSocketChannel server =
            AsynchronousServerSocketChannel.open();
        server.bind(new InetSocketAddress(port));

        System.out.println("Async server started on port " + port);

        // Accept connections asynchronously
        server.accept(null, new CompletionHandler<AsynchronousSocketChannel, Void>() {
            @Override
            public void completed(AsynchronousSocketChannel client, Void attachment) {
                // Accept next connection
                server.accept(null, this);

                // Handle this client
                handleClient(client);
            }

            @Override
            public void failed(Throwable exc, Void attachment) {
                System.err.println("Accept failed: " + exc.getMessage());
            }
        });

        // Keep main thread alive
        Thread.currentThread().join();
    }

    private void handleClient(AsynchronousSocketChannel client) {
        ByteBuffer buffer = ByteBuffer.allocate(1024);

        client.read(buffer, buffer, new CompletionHandler<Integer, ByteBuffer>() {
            @Override
            public void completed(Integer bytesRead, ByteBuffer buf) {
                if (bytesRead == -1) {
                    try { client.close(); } catch (IOException e) {}
                    return;
                }

                buf.flip();
                // Echo back
                client.write(buf, buf, new CompletionHandler<Integer, ByteBuffer>() {
                    @Override
                    public void completed(Integer bytesWritten, ByteBuffer buf) {
                        buf.clear();
                        // Read next message
                        client.read(buf, buf,
                            AsyncSocketServer.this.createReadHandler(client));
                    }

                    @Override
                    public void failed(Throwable exc, ByteBuffer buf) {
                        try { client.close(); } catch (IOException e) {}
                    }
                });
            }

            @Override
            public void failed(Throwable exc, ByteBuffer buf) {
                try { client.close(); } catch (IOException e) {}
            }
        });
    }
```

11.6 Reactor Pattern

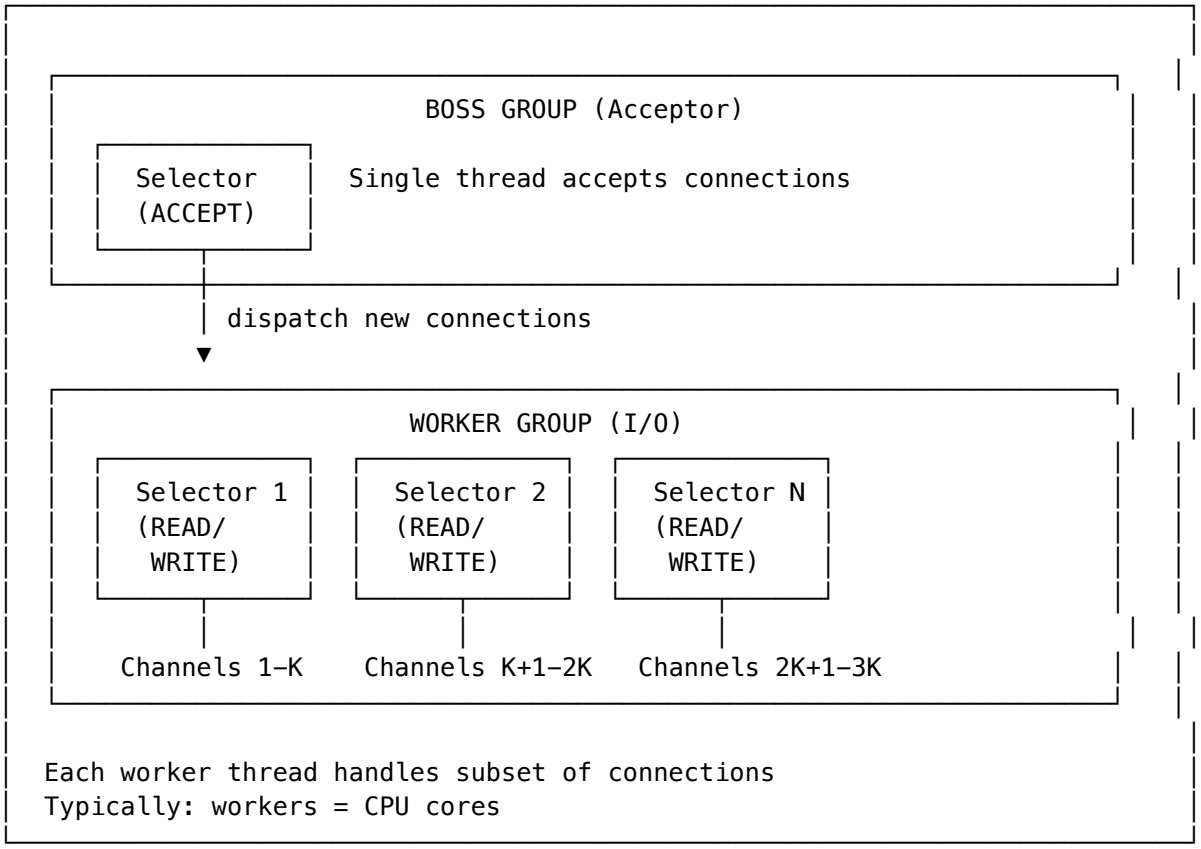
11.6.1 Single-Threaded Reactor

Single-Threaded Reactor:



11.6.2 Multi-Threaded Reactor

Multi-Threaded Reactor (Netty-style):

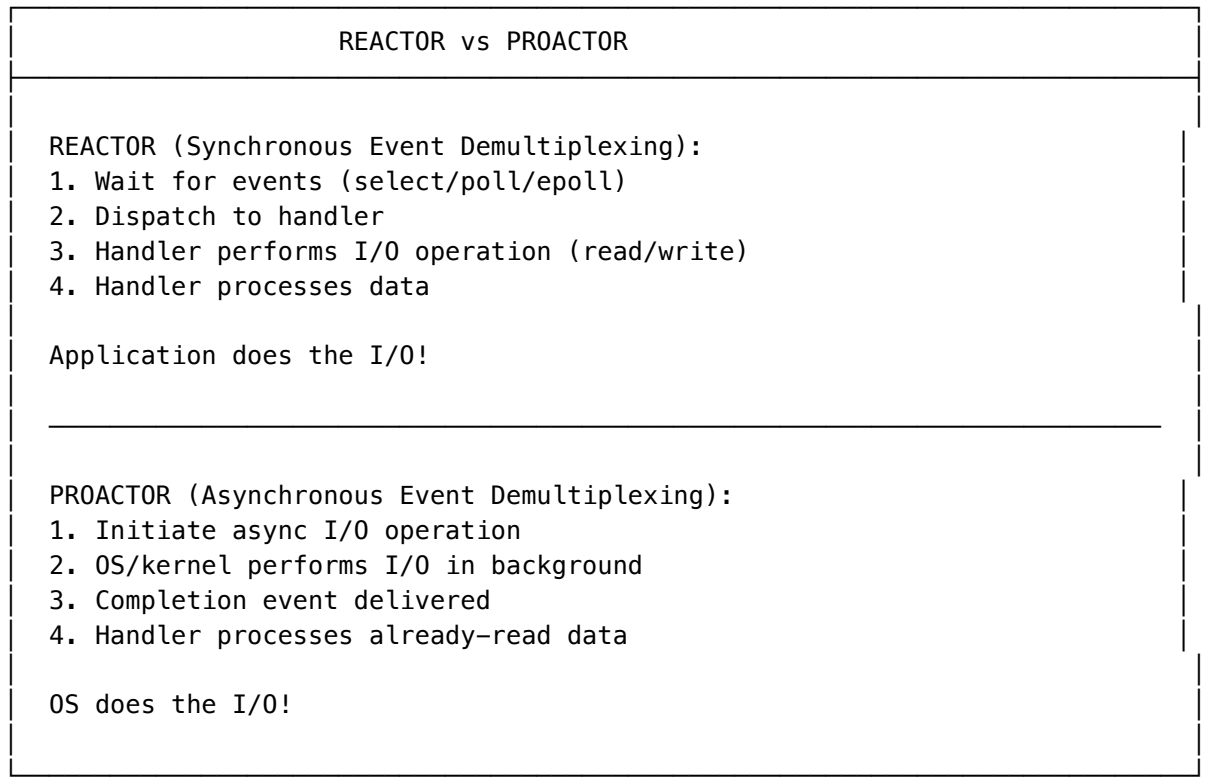


11.6.3 Reactor Implementation

```
public class MultiReactor {

    private final Selector bossSelector;
    private final Selector[] workerSelectors;
    private final int workerCount;
    private int nextWorker = 0;

    public MultiReactor(int workers) throws IOException {
        this.workerCount = workers;
        this.bossSelector = Selector.open();
        this.workerSelectors = new Selector[workers];

        for (int i = 0; i < workers; i++) {
            workerSelectors[i] = Selector.open();
        }
    }

    public void start(int port) throws IOException {
        // Start worker threads
        for (int i = 0; i < workerCount; i++) {
            final int workerId = i;
            new Thread(() -> runWorker(workerId), "worker-" + i).start();
        }

        // Setup server channel
        ServerSocketChannel serverChannel = ServerSocketChannel.open();
        serverChannel.bind(new InetSocketAddress(port));
        serverChannel.configureBlocking(false);
        serverChannel.register(bossSelector, SelectionKey.OP_ACCEPT);

        // Boss loop - accept connections
        while (true) {
            bossSelector.select();

            Iterator<SelectionKey> keys = bossSelector.selectedKeys().iterator();
            while (keys.hasNext()) {
                SelectionKey key = keys.next();
                keys.remove();

                if (key.isAcceptable()) {
                    acceptConnection((ServerSocketChannel) key.channel());
                }
            }
        }
    }

    private void acceptConnection(ServerSocketChannel server) throws IOException {
        SocketChannel client = server.accept();
        client.configureBlocking(false);

        // Round-robin to workers
        Selector workerSelector = workerSelectors[nextWorker];
        nextWorker = (nextWorker + 1) % workerCount;

        // Wake up worker and register channel
        workerSelector.wakeup();
        client.register(workerSelector, SelectionKey.OP_READ);
    }

    private void runWorker(int workerId) {
        Selector selector = workerSelectors[workerId];

        while (true) {
            try {
                selector.select();
```

11.7 Proactor Pattern

11.7.1 Proactor vs Reactor




```
When to use which:
- Reactor: Linux (epoll is very efficient), most Java servers
- Proactor: Windows (IOCP), when true async needed, Java AIO
```

11.8 Performance Comparison

11.8.1 Benchmarks

Connection Scalability (C10K Problem):

```
┌──────────────────────────────────────────────────────────────────────────┐
│                                                                          │
│  Connections    BIO (threads)    NIO (selector)    AIO                   │
│  ──────────────────────────────────────────────────────────────────      │
│      100            100              1-4             1-4                 │
│    1,000          1,000              1-4             1-4                 │
│   10,000         10,000*             1-8             1-8                 │
│  100,000        IMPOSSIBLE          8-16            8-16                 │
│                                                                          │
│  * Thread creation/context switching becomes bottleneck                  │
│                                                                          │
│  Memory Usage (per connection):                                          │
│  - BIO: ~1MB (thread stack) + buffers                                    │
│  - NIO: ~few KB (SelectionKey + buffers)                                 │
│  - AIO: ~few KB (similar to NIO)                                         │
│                                                                          │
│  Latency (single connection):                                            │
│  - BIO: Lowest (dedicated thread, no multiplexing overhead)              │
│  - NIO: Slightly higher (selector overhead)                              │
│  - AIO: Slightly higher (async overhead)                                 │
└──────────────────────────────────────────────────────────────────────────┘
```

11.8.2 When to Use What

Decision Matrix:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                                                                          │
│  Use BLOCKING I/O when:                                                  │
│  ✓ Few concurrent connections (< 100)                                    │
│  ✓ Simple request-response pattern                                       │
│  ✓ Long-running operations per request                                   │
│  ✓ Simplicity is priority                                                │
│                                                                          │
│  Use NIO (Non-blocking) when:                                            │
│  ✓ Many concurrent connections (1000+)                                   │
│  ✓ Short-lived connections                                               │
│  ✓ Need to handle C10K+ problem                                          │
│  ✓ Building servers, proxies, gateways                                   │
│                                                                          │
│  Use AIO (Async) when:                                                   │
│  ✓ File I/O with large files                                             │
│  ✓ Windows platform (IOCP)                                               │
│  ✓ True async semantics needed                                           │
│  ✓ Callback-based programming preferred                                  │
│                                                                          │
│  In Practice:                                                            │
│  - Most Java servers use NIO (Netty, Tomcat NIO connector)               │
│  - AIO adoption is limited (Linux AIO not as mature as epoll)            │
│  - Frameworks abstract the complexity (use Netty!)                       │
└──────────────────────────────────────────────────────────────────────────┘
```

11.9 Interview Questions

11.9.1 Q1: Explain the difference between Blocking and Non-Blocking I/O

```
Answer:

BLOCKING I/O:
- Thread waits (blocks) until I/O operation completes
- Simple programming model
- One thread per connection
- Doesn't scale well (thread overhead)

NON-BLOCKING I/O:
- I/O operations return immediately
- Returns EAGAIN/EWOULDBLOCK if not ready
- Single thread can handle multiple connections
- Requires polling or event notification (select/epoll)
- More complex but scales to thousands of connections

Key insight: The difference is in WHEN the thread waits:
- Blocking: Waits during the I/O operation
- Non-blocking: Waits at the selector (for any channel to be ready)
```

11.9.2 Q2: What is the C10K problem?

```
Answer:

The C10K problem refers to handling 10,000+ concurrent connections.

With blocking I/O:
- 10,000 connections = 10,000 threads
- Each thread: ~1MB stack = 10GB RAM just for stacks
- Context switching overhead becomes prohibitive
- Thread scheduling becomes bottleneck

Solution: Non-blocking I/O with event multiplexing
- Single thread monitors all connections
- Only active connections consume CPU
- Memory usage: O(connections) not O(threads)

Modern servers handle C100K or C1M using:
- epoll (Linux) / kqueue (BSD) / IOCP (Windows)
- Event-driven architecture (Reactor pattern)
- Frameworks like Netty, Node.js
```

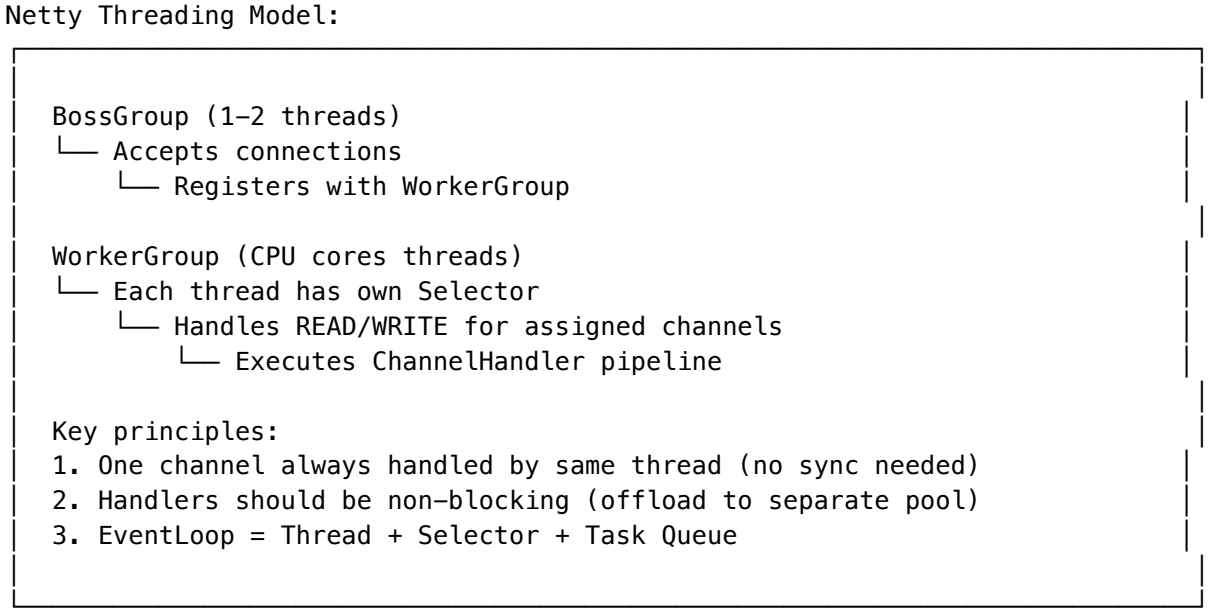
11.9.3 Q3: Explain Selector in Java NIO

```java
// Selector allows single thread to monitor multiple channels

// Key concepts:
// 1. Register channels with selector for specific events
// 2. Call select() to block until events ready
// 3. Process ready channels
// 4. Repeat

Selector selector = Selector.open();

// Register channels
channel1.register(selector, SelectionKey.OP_READ);
channel2.register(selector, SelectionKey.OP_READ);

while (true) {
    // Blocks until at least one channel ready
    int ready = selector.select();

    // Process ready channels
    Set<SelectionKey> keys = selector.selectedKeys();
    for (SelectionKey key : keys) {
        if (key.isReadable()) {
            // Handle read
        }
    }
    keys.clear();  // Must clear!
}

// Under the hood:
// - Linux: Uses epoll
// - macOS: Uses kqueue
// - Windows: Uses select (less efficient)
```

11.9.4 Q4: What is zero-copy and how does Java support it?

```java
// Zero-copy: Transfer data without copying through user space

// Traditional copy (4 copies):
// Disk → Kernel Buffer → User Buffer → Socket Buffer → NIC

// Zero-copy (2 copies with sendfile):
// Disk → Kernel Buffer → NIC (via DMA)

// Java zero-copy with FileChannel.transferTo()
public void zeroCopyTransfer(String file, SocketChannel socket)
        throws IOException {
    FileChannel fileChannel = FileChannel.open(Paths.get(file));

    // Directly transfers from file to socket
    // Uses sendfile() system call on Linux
    long transferred = fileChannel.transferTo(
        0, fileChannel.size(), socket);
}

// Also: MappedByteBuffer for memory-mapped files
FileChannel channel = FileChannel.open(path, READ);
MappedByteBuffer buffer = channel.map(
    FileChannel.MapMode.READ_ONLY, 0, channel.size());
// Buffer directly maps to file, no copying
```

11.9.5 Q5: Compare Netty's threading model




```
// Netty example
EventLoopGroup bossGroup = new NioEventLoopGroup(1);
EventLoopGroup workerGroup = new NioEventLoopGroup();  // Default: CPU cores

ServerBootstrap b = new ServerBootstrap();
b.group(bossGroup, workerGroup)
 .channel(NioServerSocketChannel.class)
 .childHandler(new ChannelInitializer<SocketChannel>() {
     @Override
     protected void initChannel(SocketChannel ch) {
         ch.pipeline().addLast(new MyHandler());
     }
 });
```

11.9.6 Q6: What happens when you call read() on a blocking socket?

```
System call flow:

1. Application calls read(socket, buffer, size)
2. Transition to kernel mode (syscall)
3. Kernel checks socket receive buffer:

   If buffer EMPTY:
   ├── Add thread to socket's wait queue
   ├── Set thread state to INTERRUPTIBLE
   ├── Context switch to another thread
   ├── ... (thread sleeping) ...
   ├── Data arrives (network interrupt)
   ├── Kernel copies data to socket buffer
   ├── Wake up waiting thread
   └── Thread scheduled to run

   If buffer has DATA:
   └── Copy data to user buffer immediately

4. Return to user mode with bytes read

Time breakdown (typical):
- Syscall overhead: ~100ns
- Context switch: ~1-10µs
- Network wait: 1ms - 100ms (depends on network)
- Data copy: ~1µs per KB
```

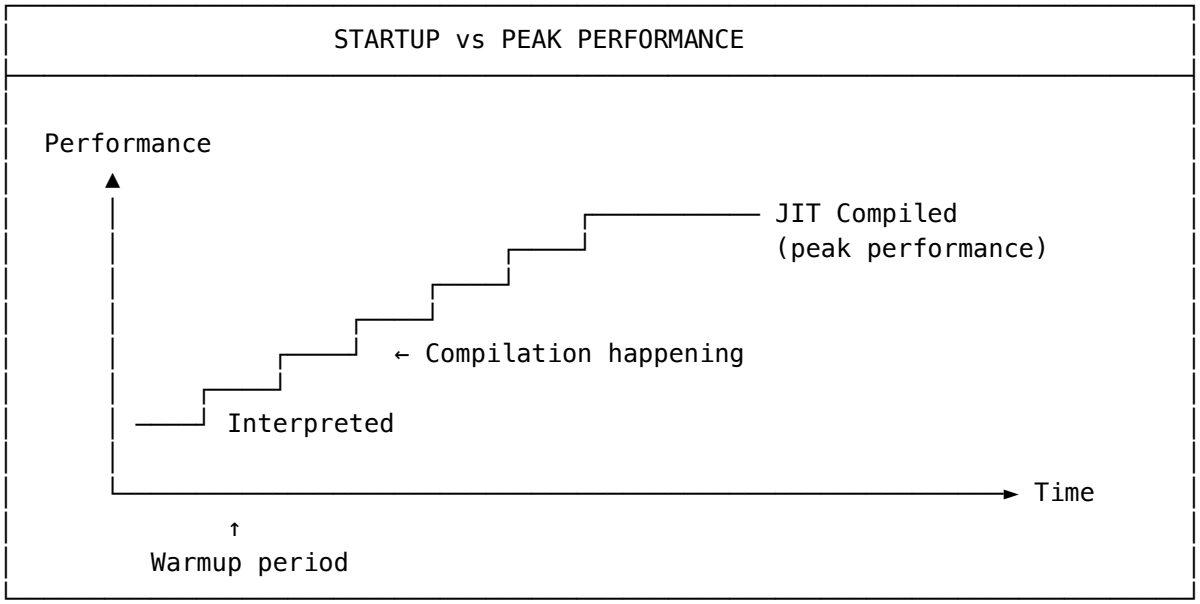
12 JIT Compilation - Deep Dive into HotSpot

12.1 JIT Compilation Overview

12.1.1 What is JIT Compilation?

```
Execution Pipeline:

┌──────────────────────────────────────────────────────────────────────┐
│                                                                      │
│  .java ──► javac ──► .class (bytecode) ──► JVM ──► Machine Code      │
│                                             │                        │
│                                             ▼                        │
│                                    ┌─────────────────┐               │
│                                    │   Interpreter   │ (slow, immediate)
│                                    └─────────────────┘               │
│                                             │                        │
│                                             ▼                        │
│                                    ┌─────────────────┐               │
│                                    │  JIT Compiler   │ (fast, delayed)
│                                    └─────────────────┘               │
│                                             │                        │
│                                             ▼                        │
│                                    ┌─────────────────┐               │
│                                    │  Native Code    │ (CPU executes)
│                                    │     Cache       │               │
│                                    └─────────────────┘               │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘

Why JIT instead of AOT (Ahead-of-Time)?
1. Platform independence (bytecode is portable)
2. Runtime profiling enables better optimizations
3. Speculative optimizations based on actual usage
4. Adaptive optimization (recompile hot paths)
```

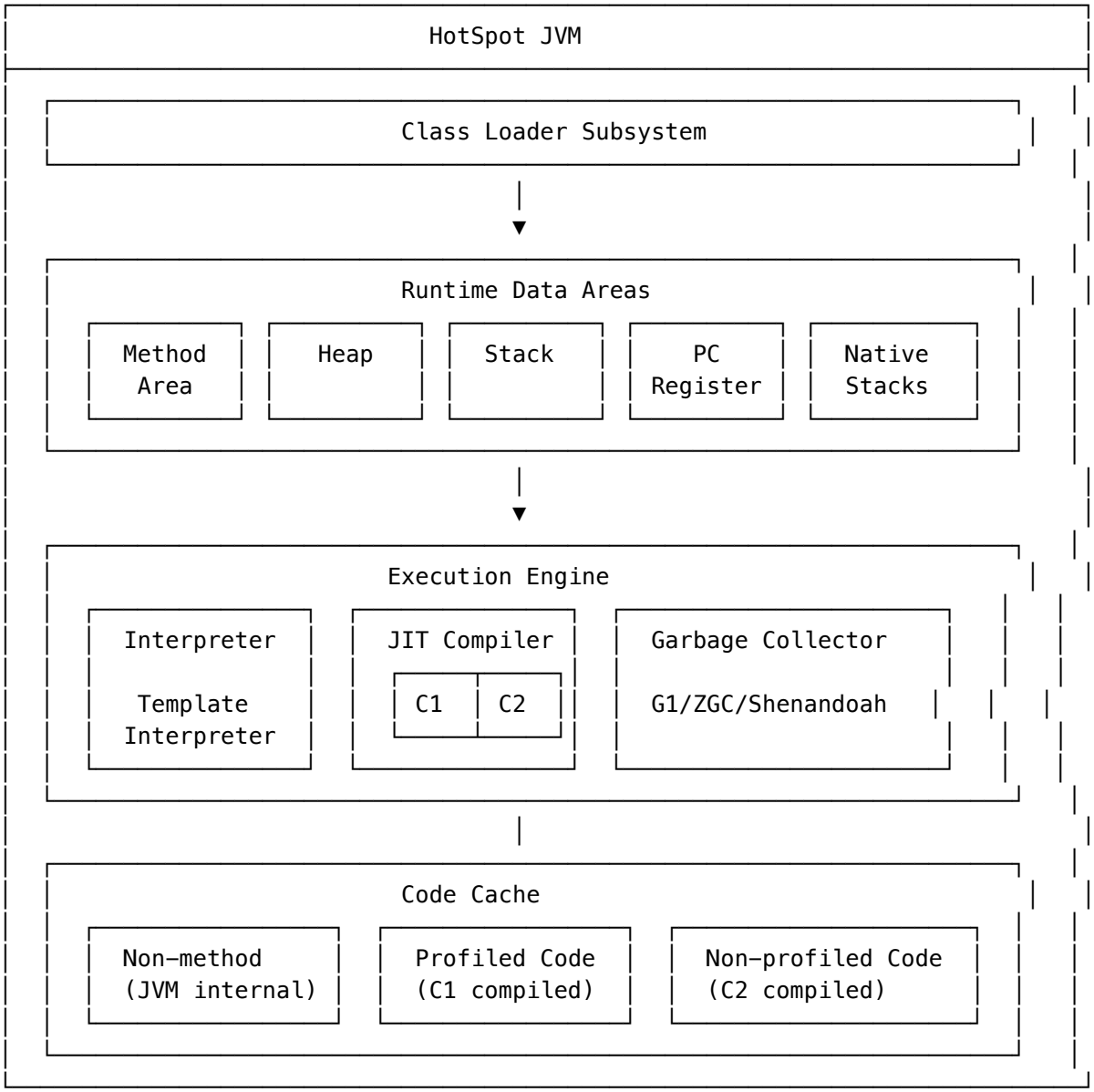
12.1.2 Compilation vs Interpretation Trade-off



12.2 HotSpot Architecture

12.2.1 HotSpot JVM Components

HotSpot JVM Architecture:



12.2.2 Method Invocation Counter

```
// HotSpot tracks method invocations and loop iterations
// When thresholds are reached, compilation is triggered

// Key counters per method:
// 1. Invocation counter: Times method called
// 2. Backedge counter: Loop iterations (backward branches)

// Compilation threshold (default with tiered compilation):
// - Tier 3 (C1): ~2000 invocations
// - Tier 4 (C2): ~15000 invocations

// Simplified counter logic:
// if (invocation_count + backedge_count > threshold) {
//     trigger_compilation(method);
// }
```

12.3 Tiered Compilation

12.3.1 Compilation Levels

```
Tiered Compilation Levels:
```

Level	Description	Profiling	Optimizations	Speed/Quality
0	Interpreter	Basic	None	Slow/None
1	C1 simple	None	Basic	Fast/Low
2	C1 limited profiling	Limited	Basic	Fast/Low
3	C1 full profiling	Full	Basic	Medium/Low
4	C2	None	Aggressive	Slow/High

```
Typical Compilation Path:

        ┌─────┐     ┌─────┐     ┌─────┐
        │  0  │ ──► │  3  │ ──► │  4  │   (Normal path)
        └─────┘     └─────┘     └─────┘
       Interp.     C1+Prof      C2

Alternative Paths:

        ┌─────┐     ┌─────┐
        │  0  │ ──► │  4  │   (Trivial method, skip C1)
        └─────┘     └─────┘

        ┌─────┐     ┌─────┐     ┌─────┐
        │  0  │ ──► │  2  │ ──► │  3  │   (C2 busy, use C1 limited)
        └─────┘     └─────┘     └─────┘
```

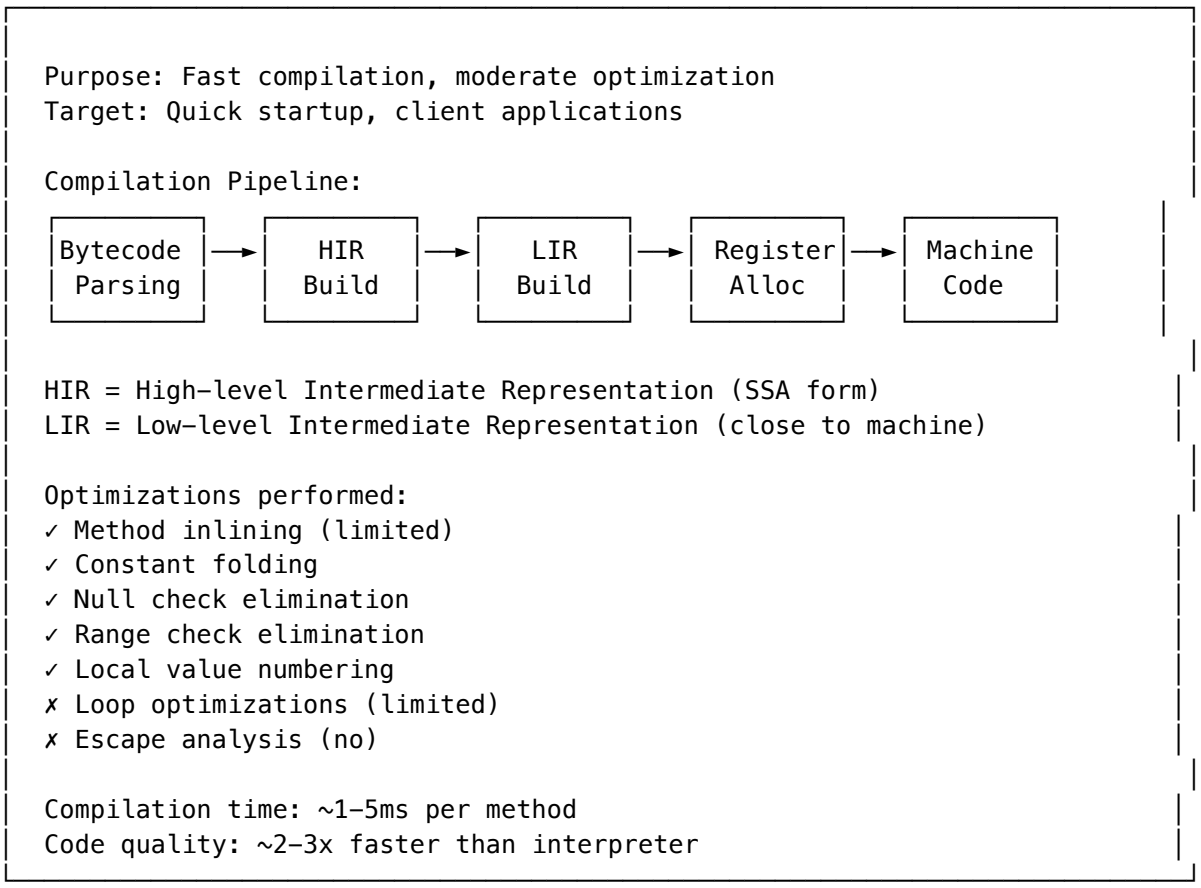
12.3.2 Compilation Thresholds

```
// Default thresholds (with tiered compilation):
// Tier 3 (C1 full): ~2000 invocations
// Tier 4 (C2): ~15000 invocations

// JVM flags to view/modify:
// -XX:Tier3InvocationThreshold=2000
// -XX:Tier4InvocationThreshold=15000

// View compilation events:
// -XX:+PrintCompilation

// Output format:
// timestamp compilation_id attributes method_name size deopt
// 123  456  %  3  java.lang.String::hashCode (55 bytes)
//
// Attributes:
// % = OSR (On-Stack Replacement)
// s = synchronized method
// ! = has exception handlers
// b = blocking compilation
// n = native method wrapper
```

12.4 C1 Compiler (Client)

12.4.1 C1 Characteristics

C1 Compiler Overview:



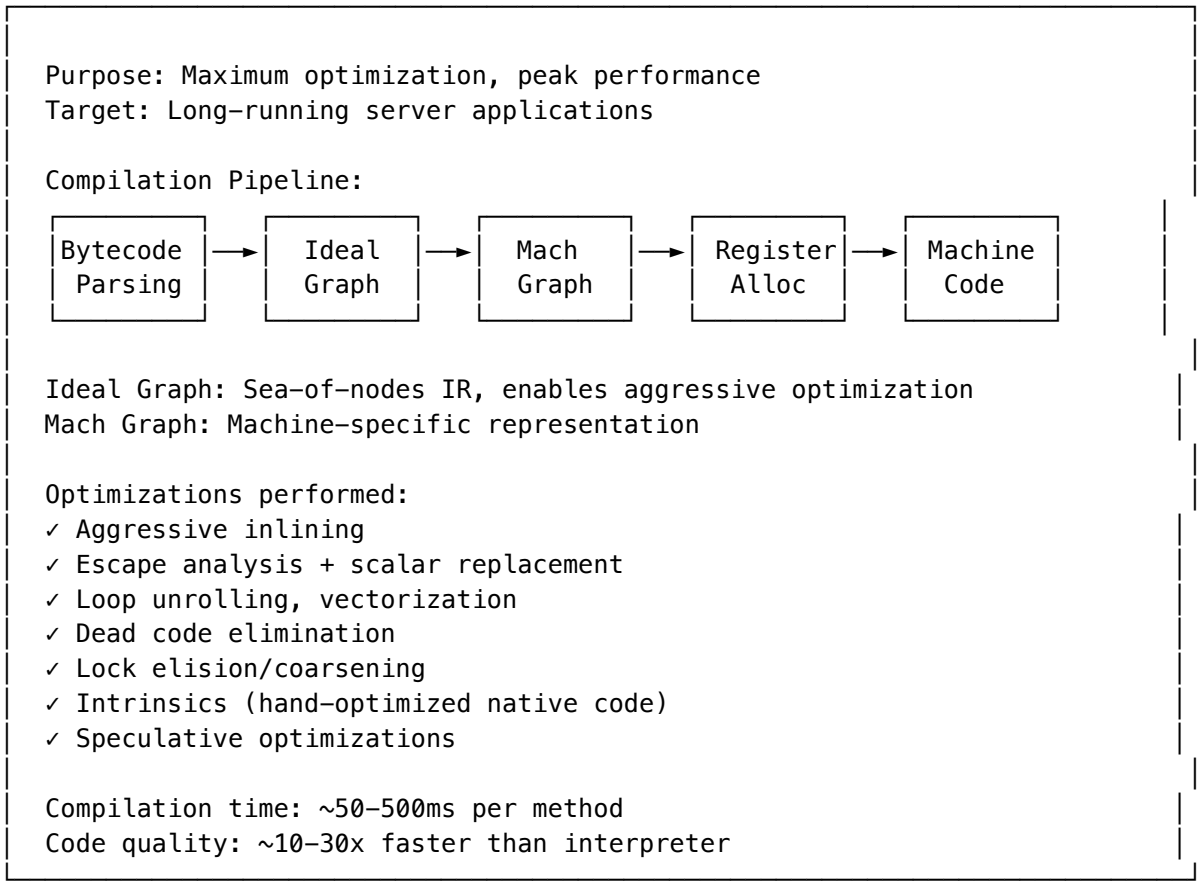
12.4.2 C1 Profiling

```
// C1 at Level 3 collects profiling data for C2:

// 1. Branch profiling
if (condition) {  // Track: how often true vs false?
    // ...
}

// 2. Type profiling
void process(Object obj) {
    obj.toString();  // Track: what types does obj have?
    // If always String, C2 can specialize
}

// 3. Call site profiling
interface Handler { void handle(); }
handler.handle();  // Track: which implementations called?
// If always ConcreteHandler, C2 can inline

// 4. Null profiling
if (obj != null) {  // Track: how often null?
    obj.method();
}
// If never null, C2 can eliminate null check
```

12.5 C2 Compiler (Server)

12.5.1 C2 Characteristics

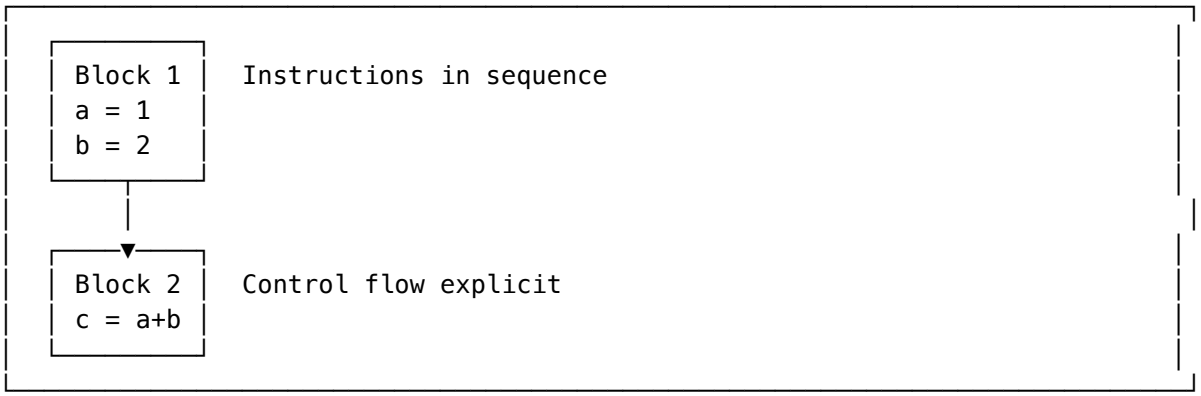
C2 Compiler Overview:



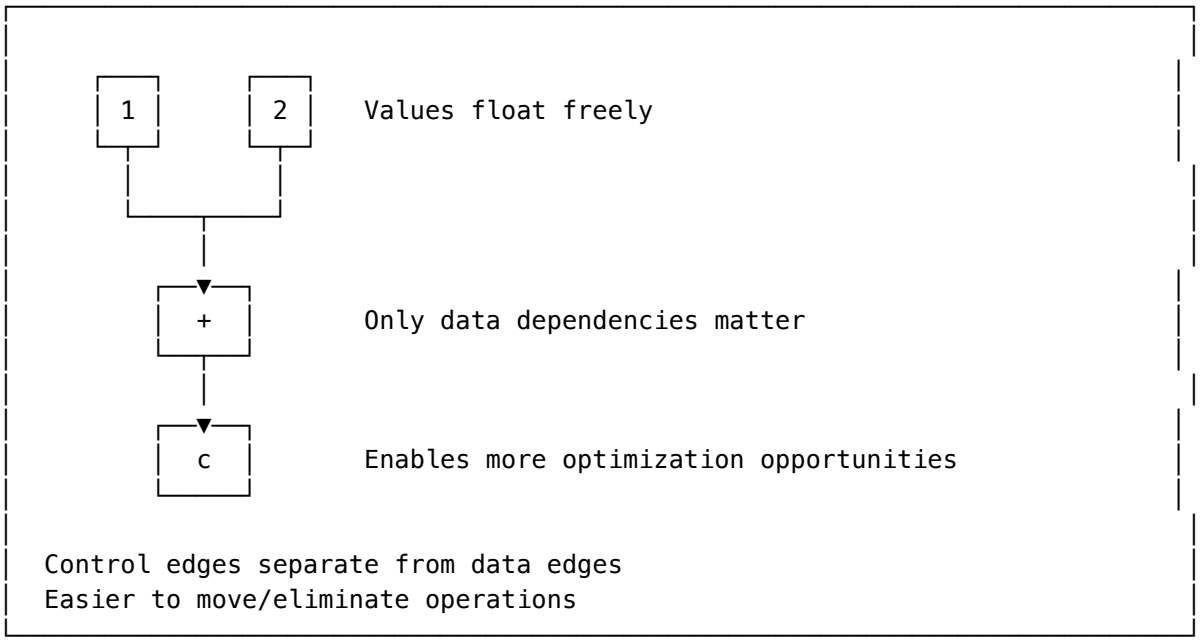
12.5.2 Ideal Graph (Sea of Nodes)

Traditional CFG vs Sea of Nodes:

Control Flow Graph (CFG):



Sea of Nodes (C2):



12.6 Key Optimizations

12.6.1 1. Method Inlining

```java
// Before inlining:
public int calculate(int x) {
    return square(x) + cube(x);
}

private int square(int n) { return n * n; }
private int cube(int n) { return n * n * n; }

// After inlining:
public int calculate(int x) {
    return (x * x) + (x * x * x);
}

// Benefits:
// - Eliminates method call overhead
// - Enables further optimizations (constant folding, etc.)
// - Most important optimization!

// Inlining decisions based on:
// - Method size (bytecode bytes)
// - Call frequency (hot methods)
// - Call site type profile (monomorphic preferred)

// JVM flags:
// -XX:MaxInlineSize=35        (max bytecode size for always inline)
// -XX:FreqInlineSize=325      (max size for hot methods)
// -XX:MaxInlineLevel=9        (max depth of inlining)
```

12.6.2 2. Escape Analysis

```java
public class EscapeAnalysis {

    // Object ESCAPES - must be heap allocated
    public Point createPoint() {
        return new Point(1, 2);  // Returned to caller
    }

    // Object does NOT escape - can be optimized
    public int sumCoordinates() {
        Point p = new Point(3, 4);  // Never leaves method
        return p.x + p.y;
    }

    // After escape analysis + scalar replacement:
    public int sumCoordinates_optimized() {
        // No Point object created!
        int p_x = 3;
        int p_y = 4;
        return p_x + p_y;
    }
}

// Escape states:
// 1. NoEscape: Object doesn't escape method
//    → Stack allocation or scalar replacement
// 2. ArgEscape: Object passed as argument but doesn't escape
//    → May skip synchronization
// 3. GlobalEscape: Object escapes (stored in field, returned, etc.)
//    → Normal heap allocation

// JVM flags:
// -XX:+DoEscapeAnalysis       (enabled by default)
// -XX:+EliminateAllocations   (scalar replacement)
// -XX:+EliminateLocks         (lock elision)
```

12.6.3 3. Loop Optimizations

```
// Loop Unrolling
// Before:
for (int i = 0; i < 100; i++) {
    sum += array[i];
}

// After unrolling (factor of 4):
for (int i = 0; i < 100; i += 4) {
    sum += array[i];
    sum += array[i + 1];
    sum += array[i + 2];
    sum += array[i + 3];
}
// Benefits: Fewer loop iterations, better instruction pipelining

// Loop Vectorization (SIMD)
// Before:
for (int i = 0; i < n; i++) {
    c[i] = a[i] + b[i];
}

// After vectorization (using AVX2):
// Process 8 integers at once using 256-bit registers
// vpaddd ymm0, ymm1, ymm2  (adds 8 ints in parallel)

// JVM flags:
// -XX:+UseSuperWord          (auto-vectorization)
// -XX:LoopUnrollLimit=60     (max unroll factor)
```

12.6.4 4. Null Check Elimination

```
// Before optimization:
public void process(String s) {
    if (s != null) {
        int len = s.length();      // Implicit null check
        char c = s.charAt(0);      // Implicit null check
        String upper = s.toUpperCase();  // Implicit null check
    }
}

// After null check elimination:
public void process(String s) {
    if (s != null) {
        // JIT knows s is not null in this block
        int len = s.length();      // No null check needed
        char c = s.charAt(0);      // No null check needed
        String upper = s.toUpperCase();  // No null check needed
    }
}

// Implicit null checks use SIGSEGV trap:
// - Access memory at object offset
// - If null, SIGSEGV caught and converted to NullPointerException
// - Zero overhead when not null!
```

12.6.5 5. Intrinsics

```
// Intrinsics: Hand-written assembly for critical methods
// JIT replaces bytecode with optimized native code

// Examples of intrinsified methods:
// - Math.sin(), Math.cos(), Math.sqrt()
// - System.arraycopy()
// - String.equals(), String.indexOf()
// - Object.hashCode()
// - Unsafe operations
// - CAS operations (compareAndSet)

// String.equals() intrinsic (x86-64):
// Uses SIMD instructions to compare 16/32 bytes at once
// Much faster than byte-by-byte comparison

// View intrinsics:
// -XX:+PrintIntrinsics (debug build only)
// -XX:+UnlockDiagnosticVMOptions -XX:+PrintInlining
```

12.6.6 6. Lock Optimization

```
// Lock Elision (remove unnecessary locks)
public void lockElision() {
    StringBuffer sb = new StringBuffer();  // Synchronized internally
    sb.append("Hello");
    sb.append(" ");
    sb.append("World");
    // If sb doesn't escape, locks can be eliminated!
}

// Lock Coarsening (merge adjacent locks)
// Before:
synchronized (lock) { op1(); }
synchronized (lock) { op2(); }
synchronized (lock) { op3(); }

// After coarsening:
synchronized (lock) {
    op1();
    op2();
    op3();
}
// Reduces lock/unlock overhead

// Biased Locking (deprecated in Java 15, removed in 18)
// Lock biased to first acquiring thread
// Subsequent acquisitions by same thread are very cheap
```

12.7 Deoptimization

12.7.1 What is Deoptimization?

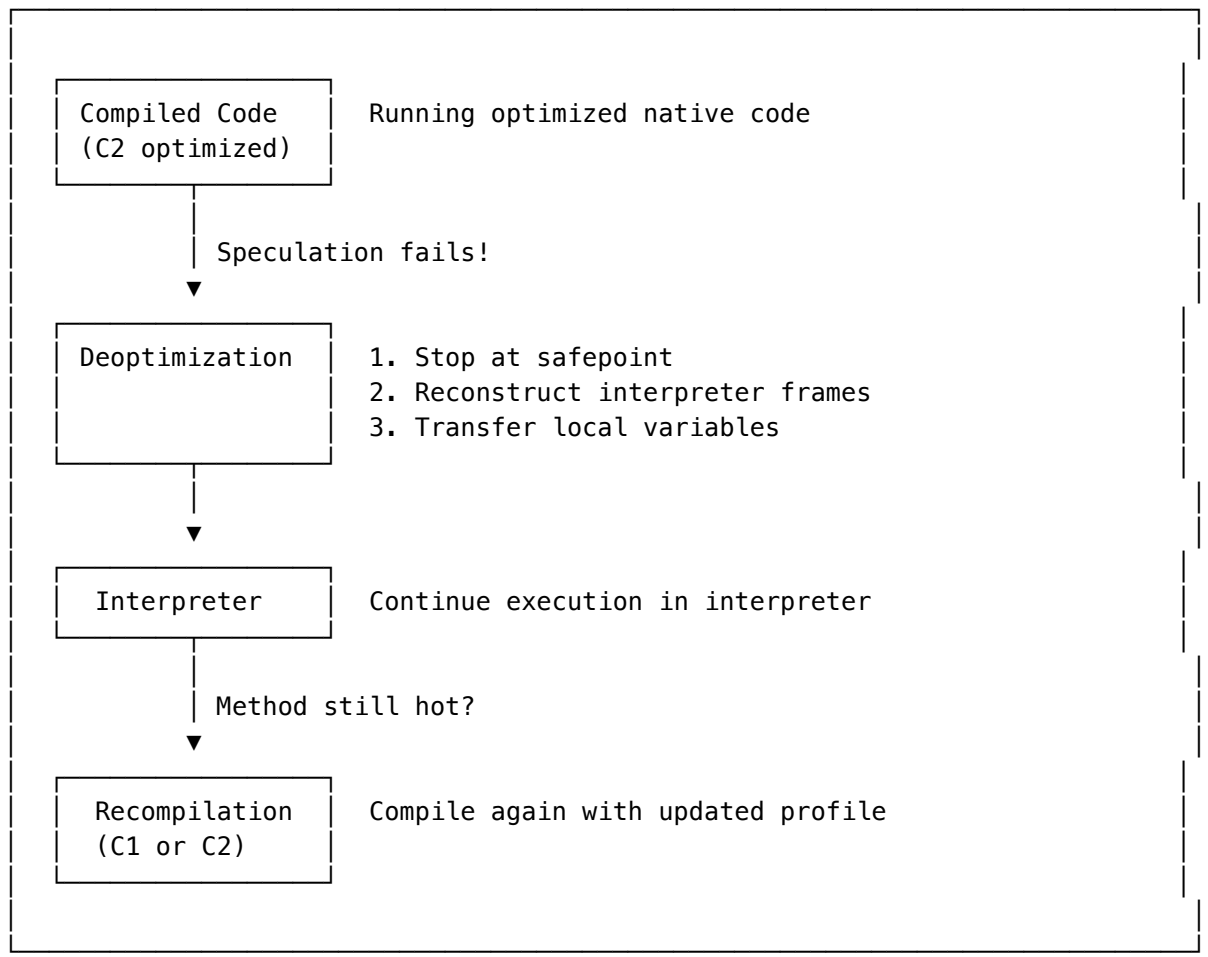
```
Deoptimization: Reverting from compiled code to interpreter

Why needed:
┌──────────────────────────────────────────────────────────────────────────────┐
│                                                                              │
│  JIT makes SPECULATIVE optimizations based on profiling:                     │
│                                                                              │
│  "This virtual call always goes to ConcreteHandler.handle()"                │
│  → Inline ConcreteHandler.handle() directly                                  │
│                                                                              │
│  But what if a NEW implementation is loaded?                                 │
│  → Speculation is WRONG                                                      │
│  → Must deoptimize and recompile                                             │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘

Deoptimization triggers:
1. Class loading invalidates type assumptions
2. Uncommon trap hit (unexpected branch taken)
3. Exception thrown in optimized code
4. Debugging/profiling attached
```

12.7.2 Uncommon Traps

```java
public void processPositive(int value) {
    if (value < 0) {
        // Profiling shows: never taken
        // JIT inserts "uncommon trap" instead of compiling this path
        handleNegative(value);
    }
    // Hot path optimized
    doWork(value);
}

// If value < 0 actually happens:
// 1. Uncommon trap triggered
// 2. Deoptimize to interpreter
// 3. Execute handleNegative() in interpreter
// 4. Recompile with updated profile (includes negative path)
```

12.7.3 Deoptimization in Action

Deoptimization Flow:




```
// View deoptimizations:
// -XX:+TraceDeoptimization
// -XX:+PrintDeoptimizationDetails
```

12.8 On-Stack Replacement (OSR)

12.8.1 What is OSR?

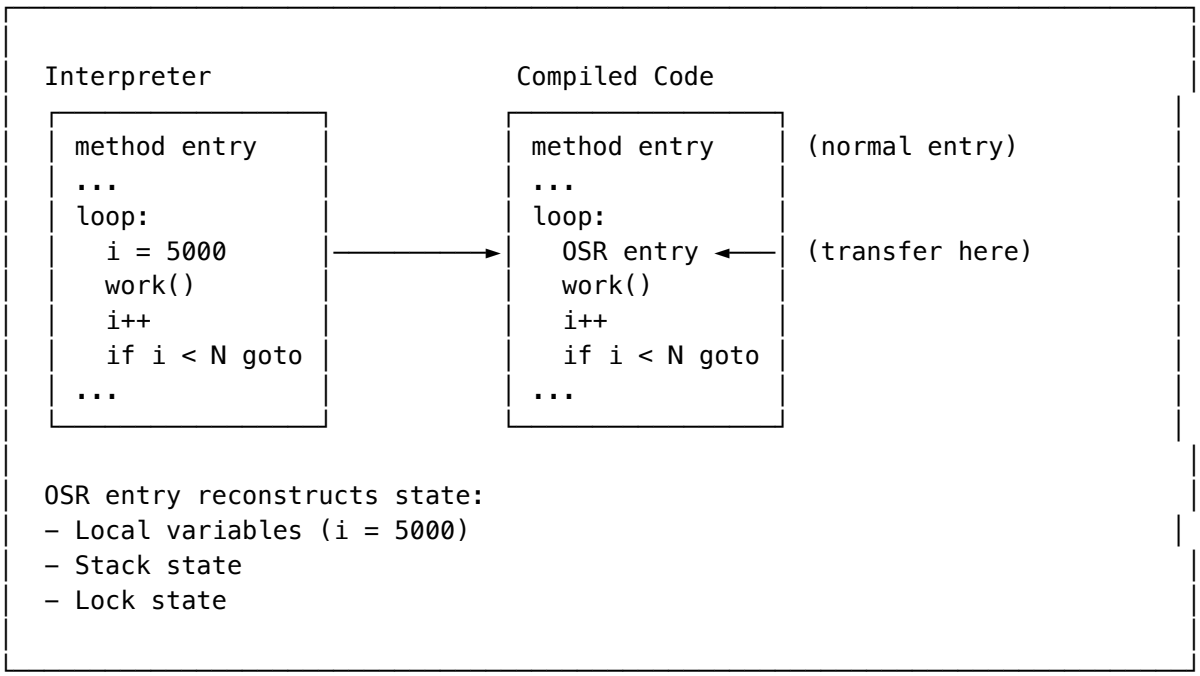
OSR: Compile and switch to optimized code WHILE method is running

Scenario:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                                                                          │
│  public void longRunningMethod() {                                       │
│      for (int i = 0; i < 1_000_000; i++) {  // Hot loop!                 │
│          // ... work ...                                                 │
│      }                                                                   │
│  }                                                                       │
│                                                                          │
│  Problem: Method invocation count = 1 (not hot)                          │
│           But loop iterations = 1,000,000 (very hot!)                    │
│                                                                          │
│  Solution: OSR                                                           │
│  1. Detect hot loop via backedge counter                                 │
│  2. Compile method with OSR entry point at loop header                   │
│  3. Transfer execution from interpreter to compiled code                 │
│  4. Continue loop in optimized code                                      │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

12.8.2 OSR Compilation

OSR Entry Point:




```
// OSR compilation shown in PrintCompilation:
// 123  456  %  4  MyClass::longRunningMethod @ 15 (100 bytes)
//            ^                                ^
//            OSR                              bytecode index of loop
```

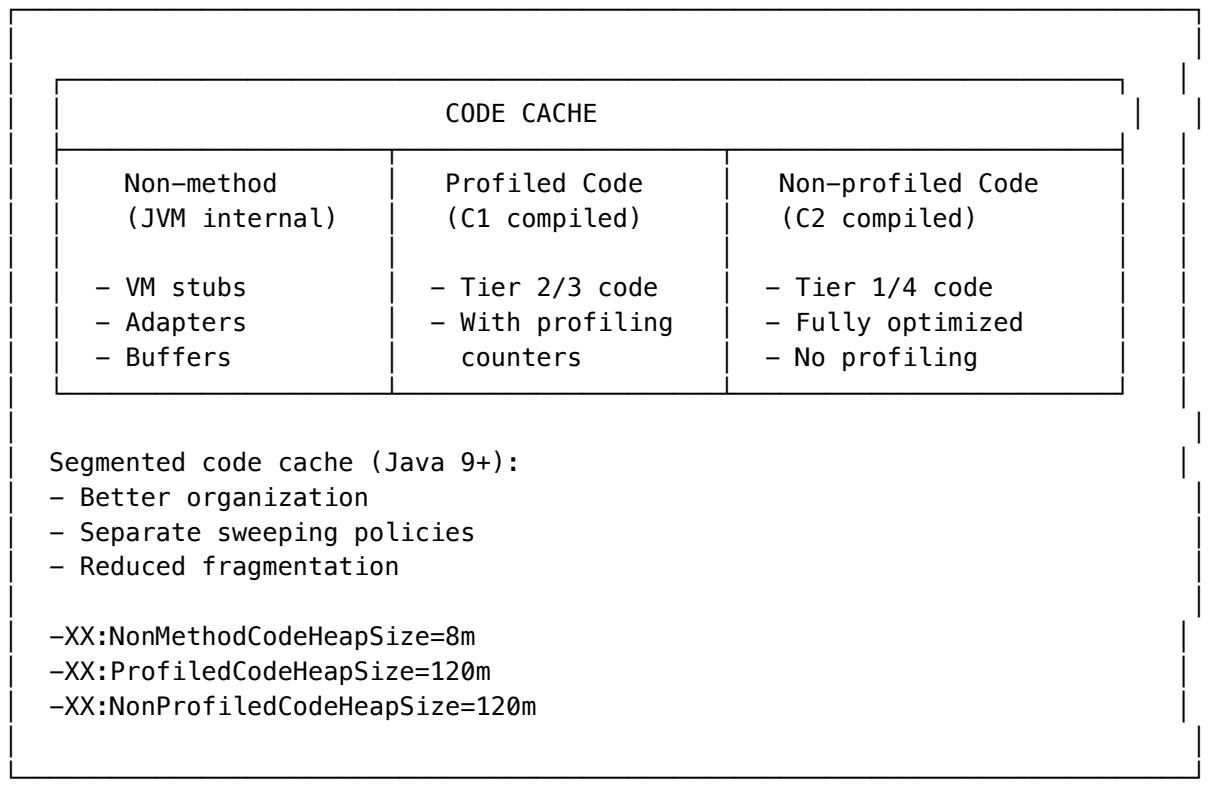
12.9 JIT Tuning Parameters

12.9.1 Common JIT Flags

```
# Compilation mode
-Xint                    # Interpreter only (no JIT)
-Xcomp                   # Compile everything (slow startup)
-Xmixed                  # Mixed mode (default)

# Tiered compilation
-XX:+TieredCompilation   # Enable tiered (default since Java 8)
-XX:-TieredCompilation   # Disable tiered (C2 only)
-XX:TieredStopAtLevel=1  # Stop at C1 (fast startup, lower peak)

# Compilation thresholds
-XX:CompileThreshold=10000        # Invocations before compile (no tiered)
-XX:Tier3InvocationThreshold=200  # Tier 3 threshold
-XX:Tier4InvocationThreshold=5000 # Tier 4 threshold

# Inlining
-XX:MaxInlineSize=35              # Always inline if smaller
-XX:FreqInlineSize=325            # Inline hot methods up to this size
-XX:MaxInlineLevel=9              # Max inline depth
-XX:InlineSmallCode=2000          # Inline if compiled code smaller

# Code cache
-XX:InitialCodeCacheSize=64m      # Initial code cache
-XX:ReservedCodeCacheSize=256m    # Maximum code cache
-XX:CodeCacheExpansionSize=64k    # Expansion increment

# Diagnostics
-XX:+PrintCompilation             # Print compilation events
-XX:+PrintInlining                # Print inlining decisions
-XX:+PrintAssembly                # Print generated assembly (needs hsdis)
-XX:+LogCompilation               # XML compilation log
```

12.9.2 Code Cache

Code Cache Structure (Java 9+):

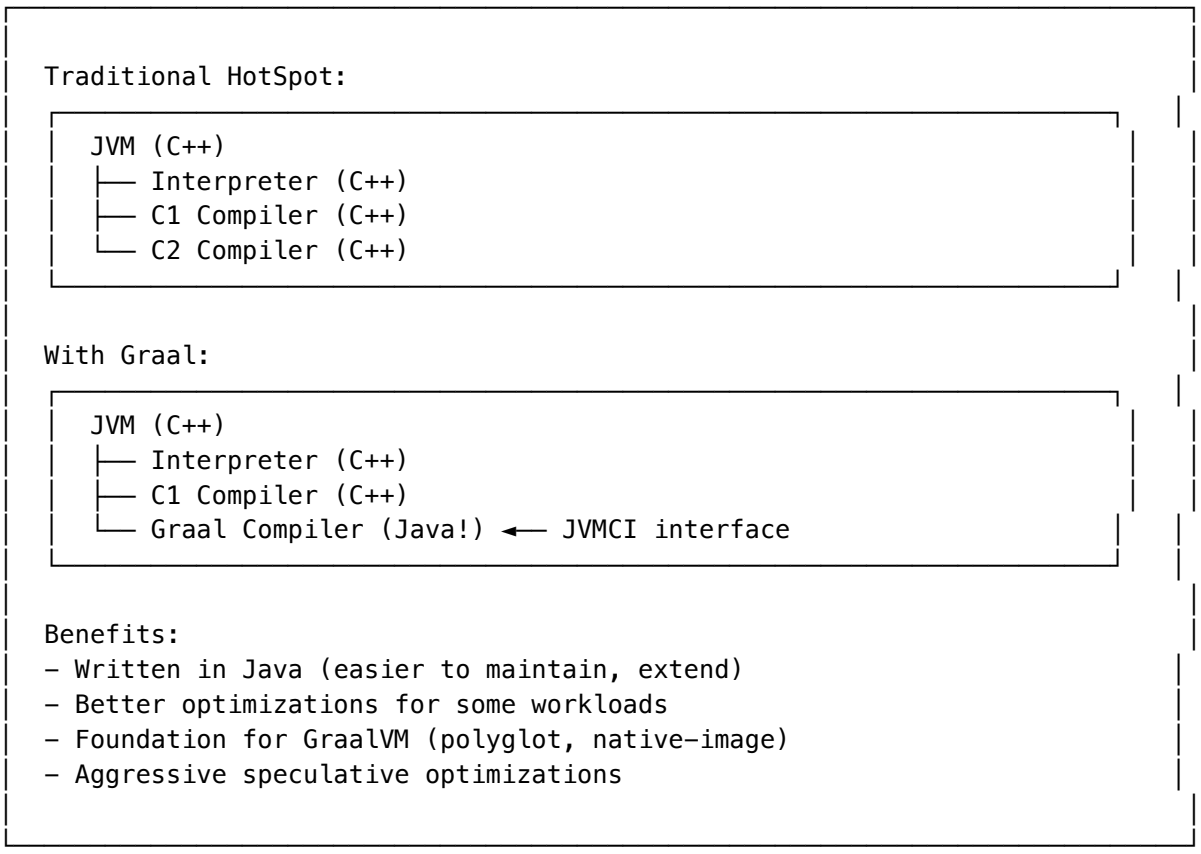



```
// Monitor code cache:
// jcmd <pid> Compiler.codecache
```

12.10 Graal Compiler

12.10.1 What is Graal?

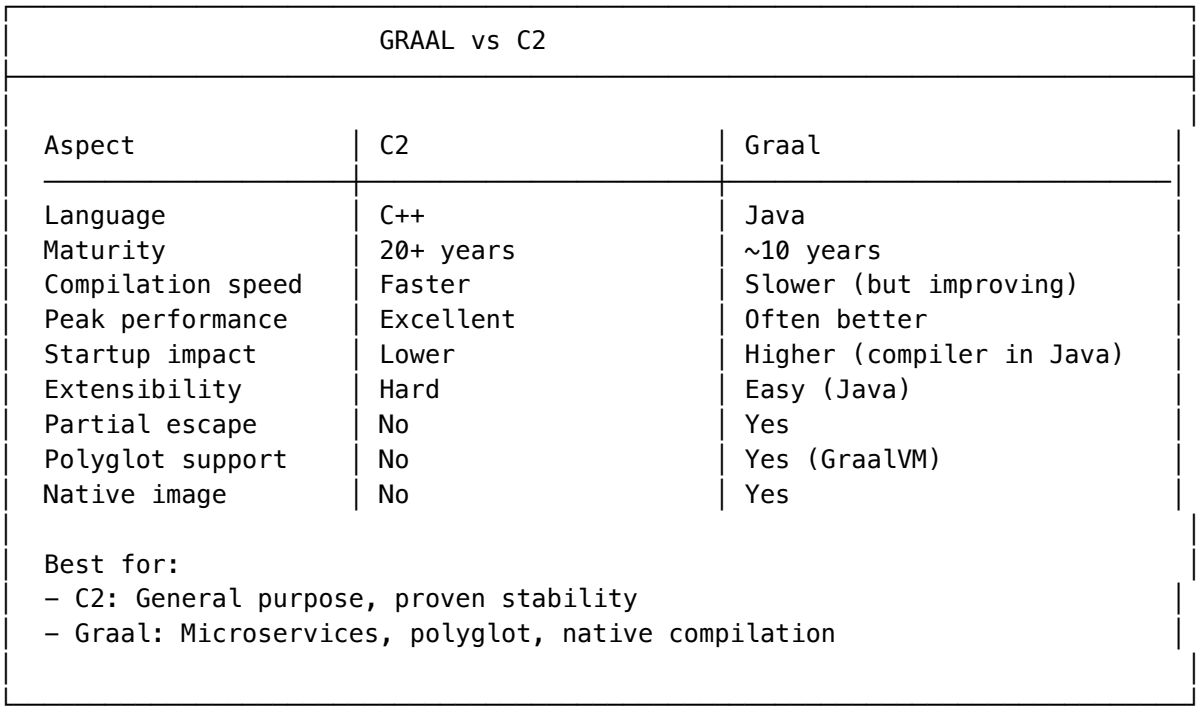
Graal: Next-generation JIT compiler written in Java



12.10.2 Using Graal

```
# Enable Graal as JIT compiler (Java 11+)
java -XX:+UnlockExperimentalVMOptions -XX:+UseJVMCICompiler MyApp

# GraalVM (includes Graal by default)
# Download from: https://www.graalvm.org/

# Graal-specific optimizations:
# - Partial escape analysis
# - Advanced inlining heuristics
# - Better loop optimizations
# - Improved speculative optimizations

# Native Image (AOT compilation)
native-image -jar myapp.jar
# Produces standalone executable with instant startup
```

12.10.3 Graal vs C2 Comparison



12.11 Interview Questions

12.11.1 Q1: Explain the difference between C1 and C2 compilers

Answer:

```
C1 (Client Compiler):
- Fast compilation, moderate optimization
- Compiles quickly to reduce startup time
- Collects profiling data for C2
- Optimizations: inlining, constant folding, null check elimination
- Code quality: ~2-3x faster than interpreter

C2 (Server Compiler):
- Slow compilation, aggressive optimization
- Uses profiling data from C1 for speculative optimizations
- Optimizations: escape analysis, loop unrolling, vectorization, lock elision
- Code quality: ~10-30x faster than interpreter

With tiered compilation (default):
- Methods start in interpreter
- Hot methods compiled by C1 (with profiling)
- Very hot methods recompiled by C2 (using C1's profile data)
- Best of both: fast startup + peak performance
```

12.11.2 Q2: What is escape analysis and how does JIT use it?

```java
// Escape analysis determines if an object "escapes" its creating method

public int calculate() {
    Point p = new Point(3, 4);  // Does p escape?
    return p.x + p.y;           // No! p is local only
}

// JIT optimizations enabled by escape analysis:

// 1. Scalar Replacement
// Instead of allocating Point object:
public int calculate_optimized() {
    int p_x = 3;  // Fields become local variables
    int p_y = 4;
    return p_x + p_y;
}

// 2. Stack Allocation (theoretical, JVM usually does scalar replacement)
// Allocate on stack instead of heap - no GC needed

// 3. Lock Elision
synchronized (new Object()) {  // Lock on non-escaping object
    // ... work ...
}
// Lock can be completely removed!

// Escape states:
// - NoEscape: Doesn't leave method → optimize
// - ArgEscape: Passed to method but doesn't escape globally → partial optimize
// - GlobalEscape: Stored in field, returned, etc. → normal allocation
```

12.11.3 Q3: What causes deoptimization?

```
Deoptimization: Reverting from compiled code to interpreter

Common causes:

1. Class Loading
   - New subclass loaded invalidates type assumptions
   - Example: JIT inlined ConcreteHandler.handle()
              New SubHandler loaded → must deoptimize

2. Uncommon Trap
   - Rarely-taken branch actually taken
   - Example: if (x < 0) { /* never happens */ }
              Then x = -1 → trap triggered

3. Type Check Failure
   - Speculative type assumption wrong
   - Example: JIT assumed obj always String
              Then obj = Integer → deoptimize

4. Null Check Failure
   - Assumed non-null was actually null

5. Array Bounds Check
   - Assumed in-bounds access was out of bounds

6. Debugging
   - Debugger attached, breakpoint set

After deoptimization:
- Method continues in interpreter
- If still hot, recompiled with updated profile
- New compilation avoids the failed speculation
```

12.11.4 Q4: How does JIT handle polymorphic calls?

```java
interface Shape { int area(); }
class Circle implements Shape { int area() { return πr²; } }
class Square implements Shape { int area() { return s²; } }

void process(Shape shape) {
    int a = shape.area();  // Virtual call - which implementation?
}

// JIT optimization based on call site profile:

// 1. Monomorphic (1 type seen)
// Profile: 100% Circle
// Optimization: Inline Circle.area() directly
//   if (shape.getClass() != Circle.class) deoptimize;
//   return shape.r * shape.r * PI;

// 2. Bimorphic (2 types seen)
// Profile: 70% Circle, 30% Square
// Optimization: Type check + inline both
//   if (shape instanceof Circle) return πr²;
//   else if (shape instanceof Square) return s²;
//   else deoptimize;

// 3. Megamorphic (many types)
// Profile: Circle, Square, Triangle, Pentagon, ...
// Optimization: Virtual call (vtable lookup)
//   No inlining possible, use standard dispatch

// JVM tracks call site types in MethodData
// -XX:TypeProfileWidth=2 (default: track 2 types per call site)
```

12.11.5 Q5: What is the code cache and why might it fill up?

```
Code Cache: Memory area storing JIT-compiled native code

Structure (Java 9+):
┌──────────────────────────────────────────────────────────────────────────┐
│  Non-method heap   │  Profiled code heap   │  Non-profiled code heap     │
│  (JVM internal)    │  (C1 + profiling)     │  (C2 optimized)             │
└──────────────────────────────────────────────────────────────────────────┘

Why it fills up:
1. Too many methods compiled
2. Large methods generating lots of code
3. Deoptimization/recompilation cycles
4. Code cache too small for application

Symptoms of full code cache:
- "CodeCache is full" warning
- Compilation disabled
- Performance degradation

Solutions:
-XX:ReservedCodeCacheSize=512m    # Increase size
-XX:+UseCodeCacheFlushing         # Enable flushing (default)

Monitor:
jcmd <pid> Compiler.codecache
jcmd <pid> VM.native_memory summary
```

12.11.6 Q6: Explain On-Stack Replacement (OSR)

```
OSR: Switching from interpreter to compiled code mid-execution

Scenario:
public void compute() {
    for (int i = 0; i < 10_000_000; i++) {
        // Hot loop, but method only called once
        heavyComputation(i);
    }
}

Problem:
- Method invocation count = 1 (not "hot")
- But loop runs 10 million times!
- Without OSR: Entire loop runs in interpreter

With OSR:
1. Backedge counter tracks loop iterations
2. When threshold reached (e.g., 10,000 iterations)
3. Compile method with OSR entry at loop header
4. Transfer execution: interpreter → compiled code
5. Remaining iterations run in optimized code

OSR entry point:
- Special entry into compiled code at loop header
- Must reconstruct: local variables, stack, locks
- Compiled code continues from current loop iteration

// PrintCompilation shows OSR:
// 123  456  %  4  MyClass::compute @ 15 (100 bytes)
//            ^                     ^
//            OSR marker            bytecode index
```

12.11.7 Q7: How do intrinsics work?

```
// Intrinsics: Hand-optimized native code for critical methods

// Example: System.arraycopy()
// Bytecode would be: loop copying element by element
// Intrinsic: Uses CPU's REP MOVSB or SIMD instructions

// How JIT handles intrinsics:
// 1. Recognize method signature
// 2. Replace bytecode with pre-written assembly
// 3. No interpretation or normal compilation

// Common intrinsified methods:
// - Math.sin(), cos(), sqrt(), abs()
// - String.equals(), indexOf(), hashCode()
// - Arrays.equals(), Arrays.fill()
// - Object.getClass(), hashCode()
// - Unsafe.compareAndSwap*()
// - Thread.currentThread()

// String.equals() intrinsic (x86-64):
// 1. Compare lengths (quick exit if different)
// 2. Compare 8 bytes at a time using 64-bit registers
// 3. Use SIMD (SSE/AVX) for longer strings
// 4. Much faster than byte-by-byte loop!

// View intrinsics:
// -XX:+PrintInlining shows "(intrinsic)" annotation
```

12.11.8 Q8: What JIT flags would you use to diagnose performance issues?

```
# Basic compilation logging
-XX:+PrintCompilation
# Shows: timestamp, compilation_id, method, size

# Detailed inlining decisions
-XX:+UnlockDiagnosticVMOptions -XX:+PrintInlining
# Shows: what was inlined, why something wasn't inlined

# Compilation queue
-XX:+PrintCompilationQueue
# Shows: methods waiting to be compiled

# Deoptimization events
-XX:+TraceDeoptimization
# Shows: why methods were deoptimized

# Full compilation log (XML)
-XX:+LogCompilation -XX:LogFile=compilation.log
# Analyze with JITWatch: https://github.com/AdoptOpenJDK/jitwatch

# Generated assembly (requires hsdis plugin)
-XX:+UnlockDiagnosticVMOptions -XX:+PrintAssembly
# Shows: actual machine code generated

# Code cache status
-XX:+PrintCodeCache
# Shows: code cache usage at shutdown

# Compilation statistics
-XX:+CITime
# Shows: time spent in compilation

# Example diagnostic session:
java -XX:+PrintCompilation \
     -XX:+UnlockDiagnosticVMOptions \
     -XX:+PrintInlining \
     -XX:+TraceDeoptimization \
     -jar myapp.jar 2>&1 | tee jit.log
```